



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校 东南大学

参赛队号

队员姓名	1. 周泽华
	2. 单文杰
	3. 苏永强

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生

数学建模竞赛

题 目： 算力约束下提升大语言模型能力的资源配置建模

摘要

本文针对算力约束下提升大语言模型能力的资源配置建模问题，结合综合评价、标度律建模、约束优化与技术演进预测等方法，提出了从“数据质量与配比刻画”到“算力最优配置”再到“能力前沿预测”的系统解决方案。

针对问题一：本问是综合评价与回归建模问题，需对 22 个质量指标预处理并建立综合评价模型、定义并消解质量冲突、建立 17 域配比与交叉熵损失的定量关系。首先，逐指标判定方向并统一为“越高越好”（其中广告字段经原文抽查确认为“第 1 类 = 无广告”的正向指标），对 8 个多维列表型指标用 softmax 期望压缩为标量，再作极差归一化、用熵权法客观赋权得到样本级、语料级与领域级质量评分；其次，以成对标准化得分差定义冲突、以 Kendall 协同系数检验一致性，并给出“多数指标方向裁决”的消解规则；再次，用岭正则线性混合回归刻画 17 域配比到 13 域损失的线性关系，并检验追加质量项的增量解释力；最后在 1M、60M、1B 三套检验集上验证并做跨尺度外推。结果表明，全量 272,505 条质量记录（另审计出 19 处字段缺失并按中位数填充）的 Kendall 协同系数为 0.069、冲突率 22.3%（低于独立基准 28.9%），说明指标间为正但不强的一致关系；抽样集 A1 与扩展集 A2/A3 的域级质量高度吻合（arxiv 0.703 对 0.701，github 0.455 对 0.455），领域质量排序为 $\text{arxiv} > \text{stackexchange} > \text{c4} > \text{commoncrawl} > \text{book} \approx \text{wikipedia} > \text{github}$ ；配比—损失模型训练平均 $R^2 = 0.629$ 、同尺度检验 0.619，60M 与 1B 跨尺度 Pearson 相关达 0.782 与 0.651；追加混合质量指数后平均 R^2 仅提升 9.9×10^{-5} ，说明配比本身已编码质量信息，故将质量作为独立资源交由后续问题处理。

针对问题二：基于 Pythia 训练轨迹，采用稳健非线性最小二乘估计经典标度律，并通过模型规模留出检验族内迁移表现；结合半合成质量实验，比较四类质量项后采用加性质量缺口模型 $L = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^{\gamma}$ 。经典拟合 $R^2 = 0.9999998$ ，广义模型联合拟合 $R^2 = 0.9896$ ；跨族与文献数据上的 MAPE 分别为 8.8% 与 8.0%。由模型导数得到，两个参考点 $(N, D, Q) = (1, 100, 0.6)$ 、 $(12, 300, 0.6)$ 的局部质量—参数替代率分别为 2.844、76.838 B/单位质量，并推导有限质量增量的等效条件，区分局部近似与规模外推。领域效应以中心化余弦相似度描述，并在损失尺度可迁移的假设下引入配比情景修正。B9 与 B10 匹配的 128 条大规模记录用于估算一致性对照，相关结论受半合成质量证据与跨来源尺度衔接假设限制。

针对问题三：建立包含基础训练、质量处理与注意力开销的预算约束模型，在给定配比情景下采用 SLSQP 求解参数量、数据量与质量水平。以幂函数型质量成本、参考配比和 2048 tokens 上下文为基准，三档预算 10^{19} 、 10^{22} 、 10^{24} FLOPs 下的配置分别为 (0.225B, 6.9B, 0.551)、(6.089B, 229.5B, 1.000)、(44.937B, 3418.0B, 1.000)，对应参数量、训练 token 数与质量评分。预算扫描显示基线质量、质量提升与质量饱和三个阶段：首次观察到正质量投入和饱和配置的网格预算分别为 1.78×10^{19} 与 7.08×10^{19} FLOPs，后者对应质量份额峰值 40.7%；这些预算数值是识别值而非精确临界点。解析得到注意力与基础训练成本相等的上下文长度 30,000 tokens。在 10^{22} FLOPs 下，上下文由 2048 增至 131072 tokens 时，参数量由 6.089B 降至 2.705B，预测损失上升约 6.0%。三类质量成本均在所考察中高预算档给出质量饱和配置，但低预算质量决策明显依赖成本形式；定量结果受评分尺度衔接和标度律外推假设约束。

针对问题四：本问是技术演进建模与前沿预测问题，需结合历史评测数据分离规模扩张与非规模技术进步的贡献并预测未来 12 至 24 个月能力前沿。首先明确时间轴（提交日期）、开源口径（开源权重且许可允许研究复现）与模型类型口径（pretrained 与 chat/finetuned 分组），得到 2,346 条开源筛选样本；其次完成附件 C8 的逐任务聚合（1,860 个模型目录、37 个子任务维度）并与 C1 交叉校验；再次按可比性分层建立 Loss—Benchmark 桥接，并用面板回归分离规模效应与年份效应；最后用 logistic 饱和模型与结构外推两条路线给出预测区间，并引用附件 C4 的算力与开源权重证据设定情景。结果表明，桥接全样本 $r = -0.510$ ($R^2 = 0.26$)，映射误差须显式讨论；规模系数为 13.84 分/10 倍参数、非规模进步 4.84 分/年；长周期（2019—2025）前沿提升中规模扩张贡献 50.5%、非规模技术进步 49.5%，而在评测标准一致的近端窗口内前沿提升的 95.4% 来自非规模因素；未来 12/24 个月开源模型能力前沿预测为 56.3（90% 置信区间 [53.6, 64.5]）与 56.6（[53.8, 66.5]），算力增长放缓情景下为 53.3 与 55.8；逐任务上指令遵循已近饱和（剩余空间 10.0%），而研究生级问答与多轮对话仍有 72.3% 与 61.3% 的剩余空间。

综上，本文完成了数据质量评价、广义标度律建模、算力约束最优配置与能力前沿预测的全链条研究，各问结论均给出可复现的数值证据与不确定性刻画，可为算力受限条件下大模型训练的资源分配、数据治理与技术路线选择提供理论依据和决策参考。

关键词：大语言模型；标度律；数据质量；资源配置优化；算力约束；能力前沿预测

目录

一、引言	5
1.1 问题背景	5
1.2 问题重述	5
二、总体分析	5
三、模型假设	6
四、符号说明	7
五、模型的建立与求解	8
5.1 问题一的模型的建立和求解	8
5.1.1 具体分析	8
5.1.2 模型准备	8
5.1.3 模型建立	11
5.1.4 模型求解	13
5.2 问题二：跨维度数据融合与广义标度律	18
5.2.1 具体分析	18
5.2.2 模型准备	18
5.2.3 模型建立	20
5.2.4 模型求解	22
5.3 问题三：算力约束下的资源配置与结构性转移	28
5.3.1 具体分析	28
5.3.2 模型准备	28
5.3.3 模型建立	29
5.3.4 模型求解	31
5.4 问题四的模型的建立和求解	34
5.4.1 具体分析	34
5.4.2 模型准备	35
5.4.3 模型建立	37
5.4.4 模型求解	38
六、模型检验与分析	43
6.1 误差分析	43
6.2 灵敏度分析	44
七、模型的评价	46
7.1 模型的优点	46
7.2 模型的缺点	47

八、模型的改进与推广	47
8.1 模型的改进	47
8.2 模型的推广	48
参考文献	49

一、引言

1.1 问题背景

以大语言模型（Large Language Models, LLMs）为代表的人工智能技术在近年来取得飞速发展，其能力的跃升高度依赖于模型规模、训练数据规模、数据质量与领域配比等多维资源的协同配置。为直观理解这一过程，可将大模型训练类比为培养一名高质量学生：参数量 N 相当于脑容量，训练数据量 D 相当于阅读量，计算量 C 相当于教育经费，数据质量 Q 相当于教材质量，领域配比 p 相当于各科课时分配。标度律（Scaling Laws）刻画了模型损失随规模变化的经验关系，是理解大模型能力形成的核心工具 [1, 2]。

然而，随着人工智能对高质量语料的需求持续增长，干净数据日益稀缺、数据污染问题日趋严重，数据墙（Data Wall）问题日益突出 [5]。2026 年《Nature》发表的研究表明，一个仅有 80 亿参数的小规模模型 OpenScholar-8B 在引文准确率、降低幻觉等任务上的表现优于 GPT-4o [6, 11]，充分说明数据质量与领域比对模型能力具有不可忽视的作用。扩大模型规模、提升数据质量与延长上下文长度均需付出算力代价，而算力始终有限，四者之间存在复杂的取舍关系。

因此，如何利用真实公开数据建立数学模型，刻画参数规模、数据规模、数据质量、领域配比与交叉熵损失之间的定量关系，并在算力约束下求解最优资源配置，成为亟待解决的关键科学问题。本赛题要求参赛队伍在无须底层系统开发与大规模训练经验的前提下，完成这一建模任务，为算力受限条件下的大模型能力提升提供理论依据与决策参考。

1.2 问题重述

问题 1：对 22 个质量指标统一方向并建立综合评价模型，给出样本级、语料级与领域级质量评分 Q ，定义并消解质量指标间的冲突，并建立 17 域配比与交叉熵损失之间的定量关系。

问题 2：将数据质量 Q 与领域配比 p 纳入经典标度律，建立同时包含 N 、 D 、 Q 、 p 的广义标度律，分析各因素边际效用与弹性，推导质量与参数规模可相互替代的条件。

问题 3：在算力预算约束下，联合优化参数量 N 、数据量 D 、领域配比 p 与数据质量 Q ，并分析预算跨越不同量级时资源配置是否发生结构性转移。

问题 4：结合历史评测数据建立技术演进模型，分离并量化规模扩张与非规模技术进步的贡献占比，预测未来 12 至 24 个月开源大语言模型能力的前沿边界。

二、总体分析

本赛题围绕“算力约束下提升大语言模型能力”这一核心目标，要求利用真实公开数据刻画参数规模、数据规模、数据质量与领域配比同交叉熵损失之间的定量关系，并在算力约束下求解最优资源配置，四道小题层层递进、前后衔接。

针对问题一，本问建立了基于熵权法的质量综合评价模型、基于成对标准化得分差与 Kendall 协同系数的冲突消解模型、以及基于岭正则的领域配比线性混合回归模型，来解决“如何量化数据质量与配比、并建立配比—损失定量关系”的问题，其输出的质量评分 Q 、质量基线 Q_0 与配比—损失系数矩阵为后续问题提供质量与配比输入。针对问题二，本问建立了含质量缺口的广义标度律 $L(N, D, Q) = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^\gamma$ ，在经典 Chinchilla 标度律基础上引入质量维并保证 $Q = 1$ 时精确退化，解决“质量与规模、数据如何共同决定损失、能否相互替代”的问题，其输出的标度律参数直接供问题三的优化目标使用。针对问题三，本问建立了严格按赛题公式分解的“基础训练 $6ND$ + 质量提升 $D[g(Q) - g(Q_0)]_+$ + 长文本注意力 ηNDL_{ctx} ”成本模型与预算约束下的 SLSQP 联合优化模型，解析给出临界上下文长度 $L_{\text{crit}} = 6/\eta$ ，并给出结构性转移的可操作数学定义，解决“算力有限时如何在 N 、 D 、 Q 之间最优配置并识别制度切换”的问题。针对问题四，本问建立了分层 Loss—Benchmark 桥接模型、含年份固定效应的面板回归分解模型、logistic 饱和外推与结构外推双路线预测模型，并完成附件 C8 的逐任务聚合与附件 C4 的宏观证据检验，解决“如何分离规模扩张与非规模技术进步的贡献并预测能力边界”的问题，将前三问以损失为目标的结论桥接为以评测得分为度量的能力演进。

由此可见，四个问题构成一条完整的建模链条：问题一提供质量与配比这一“数据侧”输入，问题二将其纳入标度律得到统一损失函数，问题三在此损失函数与成本模型上完成算力约束下的最优配置，问题四则将损失层面的结论映射到真实评测能力并外推前沿。四问共享同一套“损失最小化”的优化主线，差异在于资源的维度与约束逐步扩充；同时四问之间以显式接口相互耦合——问题一输出的质量基线 Q_0 与配比修正 $M(\mathbf{p})$ 被问题三直接引用，问题二输出的标度律参数被问题三作为目标函数读取，问题四则用 Loss—Benchmark 桥接把前三问的损失结论折算为能力得分，形成“数据 → 损失 → 算力 → 能力”的闭环。

三、模型假设

为简化问题，本文做出以下假设：

- **假设 1：**附件 A 各训练配方的 17 个领域配比与各验证域交叉熵损失之间存在线性混合关系，且该关系在不同参数规模间可由同一组系数近似刻画（配比—损失线性外推假设）；该假设的依据是 1M 训练集平均 $R^2 = 0.629$ 且 60M/1B 检验集 Pearson 相关达 0.782/0.651。

- **假设 2：**附件 B 的 Q_{score} 字段即为数据质量（越大越好， $Q = 1$ 为最优），且验证损失随参数量、数据量与质量的变化可用形如 $L = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^\gamma$ 的广义标度律描述（质量缺口可加假设）；该方向由控制 N, D 后 B6/B7 的组内相关 $-0.925/-0.918$ 一致确认，该形式在四类候选中拟合优度与信息准则最优。

- **假设 3：**训练总成本严格按赛题公式分解为基础训练 $6ND$ 、质量提升 $D[g(Q) - g(Q_0)]_+$ 与长文本注意力 ηNDL_{ctx} 三部分之和，其中 $g(Q)$ 取附录 B 规定的三型之一、 $\eta = 2 \times 10^{-4}$ 、 L_{ctx} 由附件 C7 外生给定（成本可加分解假设）；该假设不做额外

简化, C_Q 与训练 token 数成正比。

• **假设 4:** 开源大语言模型能力前沿随时间先加速上升后趋于饱和, 可用 logistic 饱和函数刻画; 同时规模扩张与非规模技术进步对能力得分的贡献可由面板回归线性分离 (技术演进可分解假设); 该假设的先验依据是六维得分存在上界 100, 且年份与参数量在控制年份效应后仍可分别识别。

四、符号说明

本文使用的主要符号及其含义如表1所示。

表 1 符号说明

符号	说明
N	模型参数量 (问题二以 B 为单位; 问题三成本计算使用参数个数)
D	训练数据规模 (问题二以 B tokens 为单位; 问题三成本计算使用 token 数)
L	验证交叉熵损失 (越低越好)
Q	数据质量评分 ($Q \in [0, 1]$, 越高越好)
Q_0	基线数据质量 (参考语料配比加权平均, $Q_0 = 0.5510$)
$\mathbf{p}$	17 个训练领域的配比向量 (分量非负且和为 1)
E	不可约损失 ($N, D \rightarrow \infty$ 且 $Q = 1$ 时的极限损失)
A, B	标度律中规模项与数据项的系数
α, β	标度律中规模项与数据项的指数
C, γ	广义标度律中质量缺口项的系数与指数 (问题三将系数记为 C_q)
$\varepsilon_N, \varepsilon_D, \varepsilon_Q$	损失对参数量、数据量、质量的弹性
$M(\mathbf{p})$	配比项对损失的平移修正 (相对参考配比)
$C_{\text{train}}, C_Q, C_{\text{attn}}$	基础训练、质量提升、长文本注意力三类算力开销
C_{budget}, C_b	算力预算 (单位: FLOPs; 问题三简记为 C_b)
$g(Q)$	数据质量成本函数 (指数型/幂函数型/对数渐进型)
η	长文本注意力的算力系数 ($\eta = 2 \times 10^{-4}$)
$\lambda(L_{\text{ctx}})$	注意力开销相对训练开销的倍数 $\eta L_{\text{ctx}}/6$
L_{crit}	注意力开销等于训练开销的临界上下文长度 ($L_{\text{crit}} = 6/\eta = 30,000$)
$\tau(Q)$	质量处理成本折算的等效附加参数规模, 与问题三的 N 同口径
$\Theta(C_b)$	质量提升完成度; 问题三以其对 $\log_{10} C_b$ 的变化率描述提升速度
$F(t)$	时刻 t 的模型能力前沿 (Leaderboard 六维均分)
K, r, t_0	logistic 饱和外推的饱和上界、增长率与拐点
b_P, b_t	面板回归中的规模系数 (分/10 倍参数) 与时间系数 (分/年)
α_t	面板回归的年份固定效应 (非规模技术进步)

表 1 符号说明 (续)

符号	说明
$\bar{B}$	Leaderboard 六维平均得分
w_j, e_j	第 j 个质量指标的熵权与信息熵
W	Kendall 协同系数 (指标一致性度量)
ρ_c, ρ_0	冲突率与指标独立时的冲突基准
R^2	决定系数
RMSE / MAE / MAPE	均方根误差 / 平均绝对误差 / 平均绝对百分比误差

五、模型的建立与求解

5.1 问题一的模型的建立和求解

5.1.1 具体分析

针对问题一，本问属于**综合评价与回归建模**类问题，需要完成三项递进任务：一是对 22 个质量指标进行预处理并建立综合评价模型，给出样本级、语料级与领域级质量评分 Q ；二是定义并消解不同质量指标之间的冲突；三是建立 17 域配比 p 与交叉熵损失之间的定量关系并验证。本问综合采用指标方向统一、极差归一化与熵权法进行质量评价，采用成对标准化得分差与 Kendall 协同系数进行冲突检验与消解，采用岭正则线性混合回归 (Ridge Mixture Regression) 刻画配比—损失关系 [8]。

首先对质量信号数据进行清洗，将 22 个质量指标 (含 8 个多维列表型指标) 统一为“越高越好”的方向并压缩为标量，建立综合评价模型得到质量评分 Q ；接着定义“冲突”为两指标标准化得分之差超过阈值的情形，用 Kendall 协同系数检验指标一致性并给出消解规则；然后利用训练配方表与损失表建立配比 p_k 到各域损失 L_d 的线性混合回归模型，并以岭正则处理成分数据的闭合效应与未测量域；最后在 1M、60M、1B 三套检验集上验证，并利用 10B、70B 外推表讨论跨尺度外推的稳健性。

5.1.2 模型准备

1. 数据预处理

附件 A 的配方表 (A4) 以 17 个 The Pile 子领域的训练配比 p_k (分量非负且和为 1) 为特征 [10]，损失表 (A5) 以 13 个验证域的交叉熵损失为响应，二者通过 index 列一一对应。17 个训练域中 nih_exporter、enron_emails、europarl、philpapers 仅有配比列而无对应损失列，本文保留其配比分量但在系数解释时标注为“间接影响域”。质量信号数据 (A1–A3) 含 22 个质量指标，其方向并不统一，须先逐指标统一为“越高越好”。设第 j 个指标的原始取值为 x_j ，对正向指标直接做极差归一化，对负向指标先取补再归一

化，对“长度型”指标（过长过短均不佳）转为中心接近度：

$$x'_j = \begin{cases} \frac{x_j - \min x_j}{\max x_j - \min x_j}, & \text{正向指标} \\ \frac{\max x_j - x_j}{\max x_j - \min x_j}, & \text{负向指标} \\ \exp\left(-\frac{|\ln(1 + x_j) - m_j|}{2.5 \text{MAD}_j}\right), & \text{长度型指标} \end{cases} \quad (1)$$

其中 m_j 与 MAD_j 为对数尺度的中位数与中位绝对偏差； $x'_j \in [0, 1]$ 且统一为“越高越好”。22 个指标的方向判定与语义说明见表2，其中 8 个多维列表型指标（原始为有序类别 logits）先按 softmax 概率取归一化期望级别压缩为标量，再参与归一化。

表 2 22 个质量指标的方向判定与熵权

指标	类型	语义（方向依据）	方向	熵权 w_j
fineweb_edu	列表 (1)	教育价值	正向	0.0437
fluency_en	列表 (2)	英语流畅度（第 1 类 = 流畅，已核验）	正向	0.0682
modernbert_cleanliness	列表 (6)	文本洁净度	正向	0.1434
modernbert_readability	列表 (6)	可读性	正向	0.0567
modernbert_reasoning	列表 (6)	推理性	正向	0.1517
modernbert_professionalism	列表 (6)	专业性	正向	0.0396
dsir_books	标量	书籍域重要性权重	正向	0.0081
dsir_wiki	标量	维基域重要性权重	正向	0.0081
dsir_math	标量	数学域重要性权重	正向	0.0086
qurater	列表 (4)	教育质量评级（4 项取均值）	正向	0.0492
ad_en	列表 (2)	广告含量（第 1 类 = “无广告”，正向；依据：原文抽查）	正向	0.0072
rps_doc_word_count	标量	文档词数	长度型	0.0516
rps_doc_num_sentences	标量	文档句数	长度型	0.0523
rps_doc_unigram_entropy	标量	一元词熵（多样性）	正向	0.0347
rps_doc_frac_unique_words	标量	唯一词占比	正向	0.0538

表 2 22 个质量指标的方向判定与熵权 (续)

指标	类型	语义 (方向依据)	方向	熵权 w_j
rps_doc_frac_no_alph_words	标量	非字母字符占比	负向	0.0287
rps_doc_frac_chars_top_2gram	标量	字符 2-gram 重复度	负向	0.0130
rps_doc_frac_chars_top_3gram	标量	字符 3-gram 重复度	负向	0.0119
rps_lines_uppercase_letter_fraction	标量	大写字母行占比	负向	0.0316
rps_lines_ending_with_terminal_punctuation_mark	标量	标点结尾行占比	正向	0.0880
rps_lines_numerical_chars_fraction	标量	含数字行占比	负向	0.0125
rps_doc_mean_word_length	标量	平均词长	长度型	0.0373

样本级质量评分由各指标加权求和得到, 领域级质量 Q_d 为该领域内样本评分的均值:

$$Q_i = \sum_{j=1}^{22} w_j x'_{ij}, \quad Q_d = \frac{1}{n_d} \sum_{i \in \mathcal{D}_d} Q_i, \quad \sum_j w_j = 1 \quad (2)$$

其中权重 w_j 由熵权法确定。由表2可见, 权重最大的两项是 modernbert_reasoning (0.152) 与 modernbert_cleanliness (0.143), 即“推理性”与“洁净度”两个信号提供的信息量最多; 需要说明的是, 熵权法对取值分布的形态较敏感 (极端左偏的指标会获得更大权重), 因此本文在误差分析中对赋权方案作了敏感性检验 (见第6.1节)。

质量信号存在少量字段级缺失, 本文流式读取时逐字段审计, 结果见表3, 并按该指标中位数填充, 不把缺失当作高质量或低质量。

表 3 质量信号的字段级缺失审计 (A1-A3 全量 272,505 条)

指标	缺失记录数	形态
modernbert_reasoning	13	整行 6 维取值均为 NaN
modernbert_professionalism	6	整行 6 维取值均为 NaN
其余 20 个指标	0	无缺失

2. 算法选择

本问涉及质量评价 (指标聚合)、冲突分析 (一致性检验) 与配比建模 (回归) 三类任务, 需在候选算法中择优。质量聚合比较熵权法、主成分分析 (PCA) 第一主成分与等权平均; 冲突分析以成对标准化得分差为主、Kendall 协同系数为辅; 配比建模比较普通最小二乘、岭回归与随机森林。质量聚合的三种方案对比如表4所示。

表 4 问题一赋权方案对比（样本级相关性与域级排序一致性）

方案	得分均值	与熵权法相关性	域级 Spearman 排序一致性（对熵权法）
熵权法	0.495	1.000	1.000
等权平均	0.639	0.854	0.679
PCA 第一主成分	0.520	0.881	0.679

如表4所示，熵权法能依据指标自身信息量客观赋权、避免主观打分；与等权方案相比，样本级得分相关系数为 0.854、域级排序一致性为 0.679，说明主体格局稳定但并非完全一致——改用等权后领域排序会发生变化，故本文在误差分析中把赋权方案列入灵敏度检验。PCA 方案虽降维去相关，但主成分含义难以解释，受篇幅限制不作备选。配比建模中 17 域配比满足单纯形约束（和为 1）而天然存在共线性，岭回归通过 L_2 正则化能稳定估计且外推可解释，故采用岭正则线性混合回归。冲突分析方面，Kendall 协同系数刻画的是“指标作为评价者”的整体一致性，而问题要求的是“对同一文本给出显著不一致评价”的样本级冲突，二者互补，故同时报告。

5.1.3 模型建立

步骤 1：质量评价模型

设 m 条质量信号样本、22 个指标经方向统一与归一化后构成矩阵 $\mathbf{X} = (x'_{ij})$ 。第 j 个指标的信息熵与熵权为：

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^m f_{ij} \ln f_{ij}, \quad f_{ij} = \frac{x'_{ij} + \varepsilon}{\sum_i (x'_{ij} + \varepsilon)}, \quad w_j = \frac{1 - e_j}{\sum_{j=1}^{22} (1 - e_j)} \quad (3)$$

为与附件 B 的损失数据耦合，本问同时给出“结果导向”的领域质量代理指标：以各验证域在平均配方下的观测损失表征领域内在难度并映射为质量：

$$Q_d^{(\text{loss})} = 1 - \frac{\bar{L}_d - \min_d \bar{L}_d}{\max_d \bar{L}_d - \min_d \bar{L}_d} \quad (4)$$

其中 $\bar{L}_d$ 为领域 d 的平均交叉熵损失。该代理指标仅在 13 个可测损失域上有定义，与 A1–A3 的 7 个质量域并非同一分类体系，二者的关联由 A16 映射表建立、并在模型求解中作一致性检验。

步骤 2：质量冲突消解模型

当两个质量指标对同一文本给出显著不一致的评价时即产生冲突。以方向统一后的标准化得分为基础，定义第 i 样本在指标 j 与 k 上的冲突指示：

$$\delta_i(j, k) = |z_{ij} - z_{ik}|, \quad z_{ij} = \frac{x'_{ij} - \mu_j}{\sigma_j}, \quad \mathbf{1}\{\delta_i(j, k) > \tau\} \quad (5)$$

取阈值 $\tau = 1.5$ ；全样本冲突率为

$$\rho_c = \frac{1}{m \binom{22}{2}} \sum_{i=1}^m \sum_{j < k} \mathbf{1}\{\delta_i(j, k) > \tau\} \quad (6)$$

为给出参照系，本文同时计算“指标独立”基准 $\rho_0 = 2\Phi(-\tau/\sqrt{2})$ ：若 $\rho_c/\rho_0 < 1$ ，说明指标间存在正相关、并非互相矛盾。指标间整体一致性用 Kendall 协同系数检验（22 个指标为评价者、 m 条文本为评价对象）：

$$W = \frac{12 \sum_i (R_i - \bar{R})^2}{22^2 (m^3 - m)}, \quad R_i = \sum_{j=1}^{22} r_{ij}, \quad \chi^2 = 22(m-1)W \quad (7)$$

其中 r_{ij} 为第 i 样本在第 j 指标上的秩。消解规则为：若某样本被判为冲突，则以其上“多数指标”的方向（即加权得分 Q_i 相对该样本所在域中位数的偏离方向）作为该样本的最终质量裁决，并在报告中保留冲突标记以供下游剔除或降权。

步骤 3：领域配比建模（线性混合回归）

设训练配方矩阵 $\mathbf{P} \in [0, 1]^{n \times 17}$ （行和为 1），损失矩阵 $\mathbf{L} \in \mathbb{R}^{n \times 13}$ 。建立各验证域损失对配比的线性混合模型：

$$L_d(\mathbf{p}) = \sum_{k=1}^{17} a_{dk} p_k + b_d, \quad d = 1, \dots, 13 \quad \Longleftrightarrow \quad \hat{\mathbf{L}} = \mathbf{P}\mathbf{A} + \mathbf{1}\mathbf{b}^\top \quad (8)$$

由于 $\mathbf{P}$ 各行和为 1，回归存在闭合不可辨识性，采用岭正则估计系数矩阵 $\mathbf{A} \in \mathbb{R}^{17 \times 13}$ 与截距 $\mathbf{b}$ ：

$$\min_{\mathbf{A}, \mathbf{b}} \|\mathbf{L} - \mathbf{P}\mathbf{A} - \mathbf{1}\mathbf{b}^\top\|_F^2 + \lambda \|\mathbf{A}\|_F^2, \quad \begin{bmatrix} \mathbf{A} \\ \mathbf{b}^\top \end{bmatrix} = \left(\tilde{\mathbf{P}}^\top \tilde{\mathbf{P}} + \lambda \mathbf{D} \right)^{-1} \tilde{\mathbf{P}}^\top \mathbf{L} \quad (9)$$

其中 $\tilde{\mathbf{P}} = [\mathbf{P}, \mathbf{1}]$ ， $\mathbf{D} = \text{diag}(1, \dots, 1, 0)$ （不惩罚截距）， $\lambda = 10^{-3}$ 。

步骤 4：质量信息的引入方式与检验

问题一要求论证“是否以及如何引入质量评分 Q ”。本文构造混合质量指数 $\bar{Q}(\mathbf{p}) = \sum_k p_k Q_k$ （ Q_k 由 A16 映射得到：direct/near_direct 域直接取用 A1–A3 域级质量，inferred 域取全局均值），并做增量回归检验：

$$\hat{\mathbf{L}} = \mathbf{P}\mathbf{A} + \mathbf{1}\mathbf{b}^\top + \mathbf{1}c^\top \bar{Q}(\mathbf{p}) \quad (10)$$

若质量项带来的 R^2 增量可忽略，则说明线性配比已吸收质量信息，此时不宜重复引入以免共线；质量评分转而作为独立资源交由问题二、三使用。

步骤 5：最优配比与稳健推荐

在系数矩阵 $\mathbf{A}$ 基础上，固定未测量域配比于参考值，优化 13 个可测域的配比使加权平均损失最小：

$$\min_{\mathbf{p} \in \Delta_{17}} \sum_{d \in \mathcal{M}} w_d \left(\sum_k a_{dk} p_k + b_d \right), \quad \Delta_{17} = \{\mathbf{p} : p_k \geq 0, \sum_k p_k = 1\} \quad (11)$$

为给出可落地的稳健方案，进一步限制各域配比不超过参考配比的 2.5 倍，得到有界推荐配比。

5.1.4 模型求解

基于上述模型，利用附件 A 的质量信号（A1–A3）与配方/损失数据（A4–A15）进行求解，共处理全量质量记录 272,505 条（A1 抽样集 51,230 条、A2+A3 扩展集 221,275 条）。

1. 质量评分与冲突分析

质量域的领域级评分如图1所示，A1 抽样集与扩展集的对照结果如表5所示。

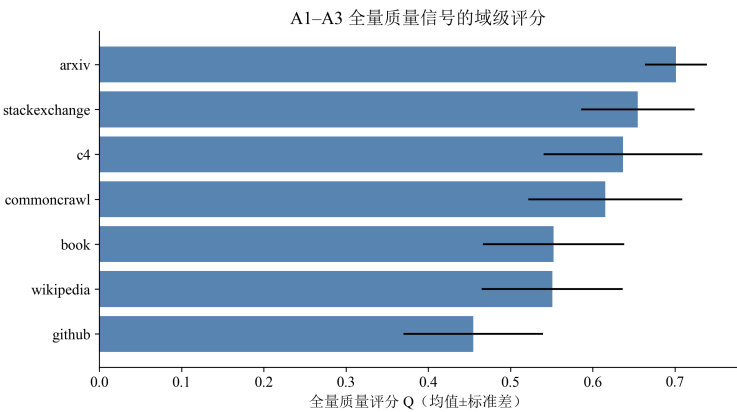


图 1 A1–A3 全量质量信号的领域级评分（均值 ± 标准差）

表 5 抽样集 A1 与扩展集 A2/A3 的域级质量对照

质量域	Q（A1）	Q（扩展集）	差值	对应配方域
arxiv	0.7031	0.7011	+0.0021	arxiv
stackexchange	0.6549	—	—	stackexchange
c4	0.6368	—	—	（配方侧无同名列）
commoncrawl	0.6153	—	—	pile_cc
book	0.5524	—	—	gutenberg_pg_19
wikipedia	0.5509	—	—	wikipedia_en
github	0.4553	0.4547	+0.0006	github

由表5可知，在同时具有抽样与扩展质量信号的两个域上（arxiv、github），两种口径的域级质量差异不超过 0.002（arxiv +0.0021、github +0.0006），说明抽样集对扩展集具有代表性、评分口径可跨集合迁移。全体记录的领域质量排序为 arxiv（0.703）> stackexchange（0.655）> c4（0.637）> commoncrawl（0.615）> book（0.552）≈ wikipedia（0.551）> github（0.455）。需要指出，该排序是 22 个分类器信号的合成结果，其中若干信号（如 rps_doc_frac_unique_words、rps_doc_unigram_entropy）与文档长度相关，因此“c4/commoncrawl 高于 book/wikipedia”反映的是信号体系的口径，而非对语料价值的终极判断——这一点在使用该评分时须注意。

冲突与一致性检验结果如表6所示，冲突最集中的指标对与熵权分布见图2。

表 6 质量冲突与一致性检验结果

数据集	样本数	Kendall W	冲突率 ρ_c	独立基准 ρ_0	ρ_c/ρ_0
A1 抽样集	51,230	0.0783	0.2252	0.2888	0.780
A2+A3 扩展集	221,275	0.0628	0.2208	0.2888	0.764
全量	272,505	0.0687	0.2235	0.2888	0.774

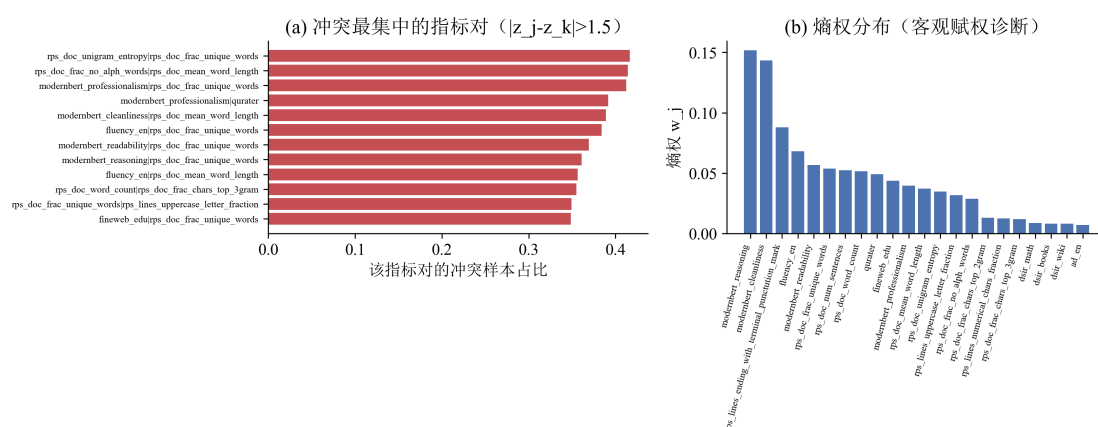


图 2 冲突最集中的指标对（左）与熵权分布（右）

由表6可知：(1) 三套数据的冲突率几乎一致（0.221 ~ 0.225），说明冲突结构在抽样集与扩展集上同样成立，A2/A3 的结论可外推至 A1 所代表的总体；(2) Kendall 协同系数为 0.069 (χ^2 检验 $p < 0.001$ ，样本量极大故显著性很高)，属于“正相关但一致性较弱”的水平，与冲突率 22.3% 一致，说明“各质量指标完全一致、不存在冲突”的判断并不成立；(3) 但冲突率低于指标完全独立时的基准 28.9%（比值 0.774），说明指标间确为正相关，冲突主要发生在语义上本就不同的指标之间。冲突成因可归纳为三类：一是度量对象不同（如 rps_doc_unigram_entropy 与 rps_doc_frac_unique_words 同测“多样性”但量纲不同），二是内容类型差异（如 modernbert_professionalism 对学术文本与论坛文本的评价尺度不同），三是分布形态差异（如 ad_en 为二分类概率、在长文档上两极化）。据此本文按“多数指标方向裁决 + 冲突标记保留”的规则消解：冲突样本仍参与领域均值，但以 Q_i 相对域中位数的偏离方向标注其可信度，并在问题二、三中不单独使用冲突样本。

2. 配比—损失建模与验证

配方结构与拟合、泛化表现分别如图3与图4所示，预测—实测对比如图6所示，系数热力图如图5所示。

由图4可知，训练集上 13 个损失域的 R^2 位于 0.42 ~ 0.77 之间、平均 0.629；同尺度 1M 检验集平均 $R^2 = 0.619$ ，与训练接近，说明模型未明显过拟合。由于不同参数规模下损失水平本身不同（60M 与 1M 的损失比约 0.68），跨尺度直接比较 R^2 无意义，故改

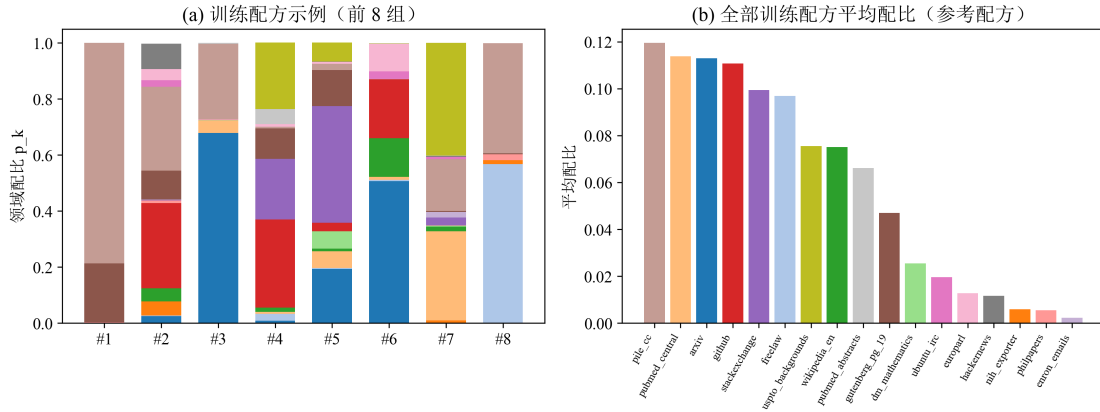


图3 训练配方结构（左：前 8 组配方堆叠；右：全部配方平均配比）

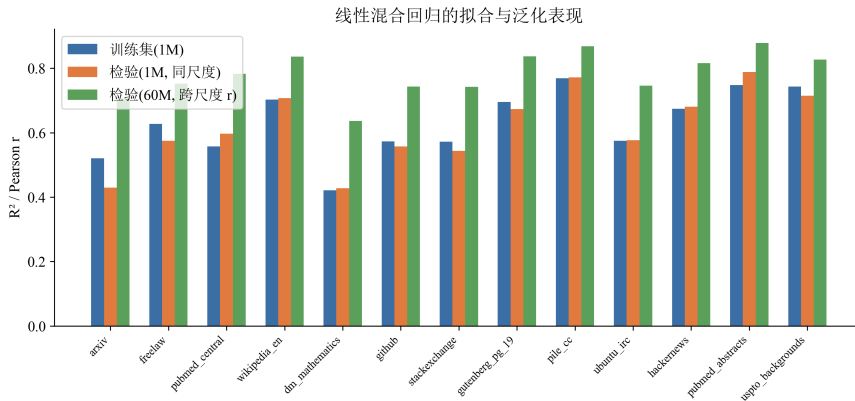


图4 线性混合回归的拟合与泛化表现

用尺度无关的 Pearson 相关：60M 与 1B 检验集上的平均相关分别为 0.782 与 0.651，表明同一套配比—损失系数可在不同参数规模间复用。由图5可知，各训练域对验证损失的相对影响差异明显，dm_mathematics、arxiv 等域降低损失，而 pile_cc、ubuntu_irc 等域抬高损失，印证了领域配比是影响模型性能的关键变量。

为验证质量信息的引入方式，本文比较了“仅配比”与“配比 + 混合质量指数 $\bar{Q}$ ”两种设定：追加质量项后 13 个损失域的平均 R^2 增量仅为 9.9×10^{-5} ，说明线性配比系数已经吸收了质量信息。因此本文不把质量评分再次作为配比模型的解释变量，而是将其作为独立资源交由问题二、三使用。

两套质量口径（A1–A3 的熵权质量 Q 与损失难度代理 $Q^{(\text{loss})}$ ）在 6 个可映射域上的排序一致性如图7所示。

由图7(a)可知，13 个损失域的内在难度差异显著，dm_mathematics 最易（代理质量 1.000）、ubuntu_irc 最难（代理质量 0.000）；图7(b)显示，A16 的 direct/near_direct 映射中两套口径的 Spearman 排序相关为 0.60：arxiv、stackexchange、github 三个域在两种口径下同向（文本质量高、损失也相对低），但 wikipedia、book、pile_cc 存在明显分歧——文本质量较高而该域损失并不低（学术与百科长文属“高质量但难拟合”）。因此质量评分必须按用途区分：面向数据治理使用 A1–A3 的文本质量，面向性能预测使用损失难

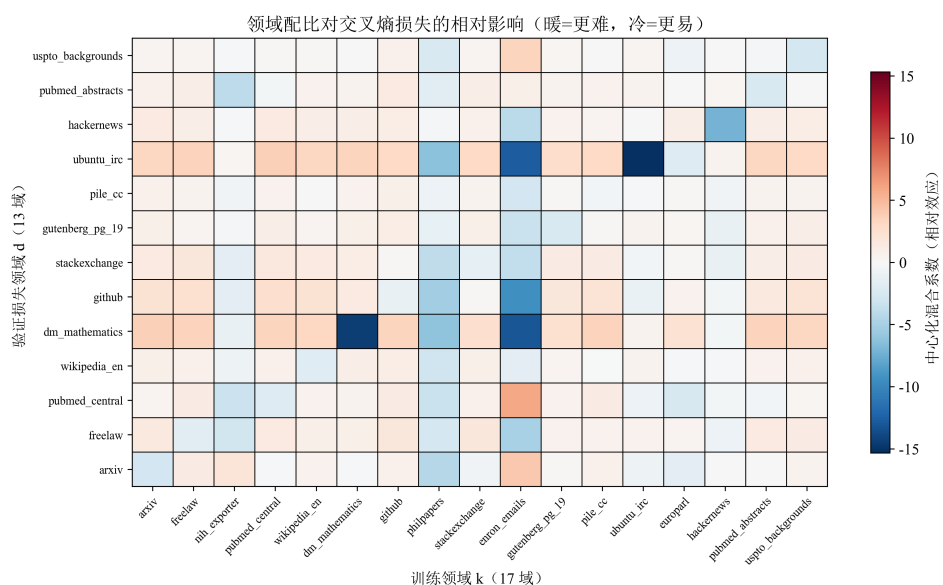


图 5 领域配比对交叉熵损失的相对影响热力图

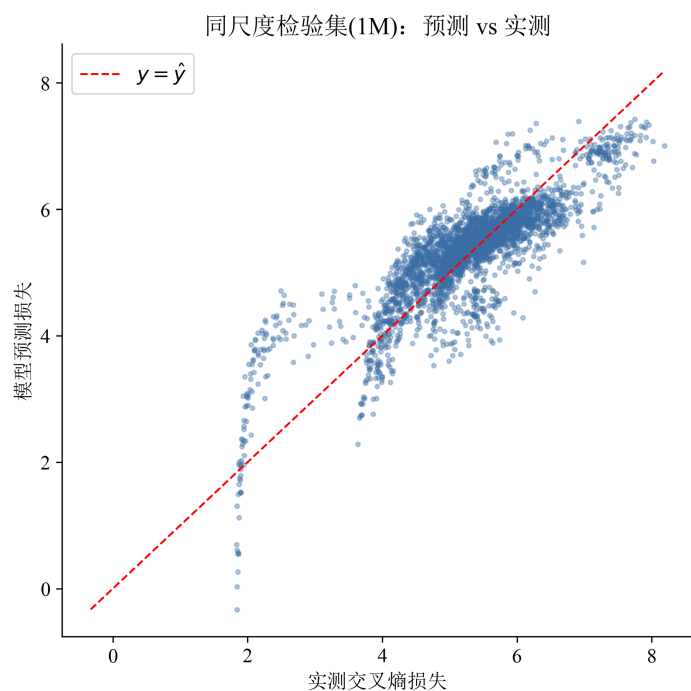


图 6 同尺度检验集（1M）预测损失与实测损失对比

度代理。

3. 最优配比与跨尺度稳健性

无约束优化倾向于把配比集中于单一低损失域（预测加权平均损失 4.177），不具备可落地性；故采用 2.5 倍上限的有界推荐作为最终方案，推荐配比与参考配比的对比见表7，该方案使预测加权平均损失由 5.336 降至 5.136（下降 0.199）。

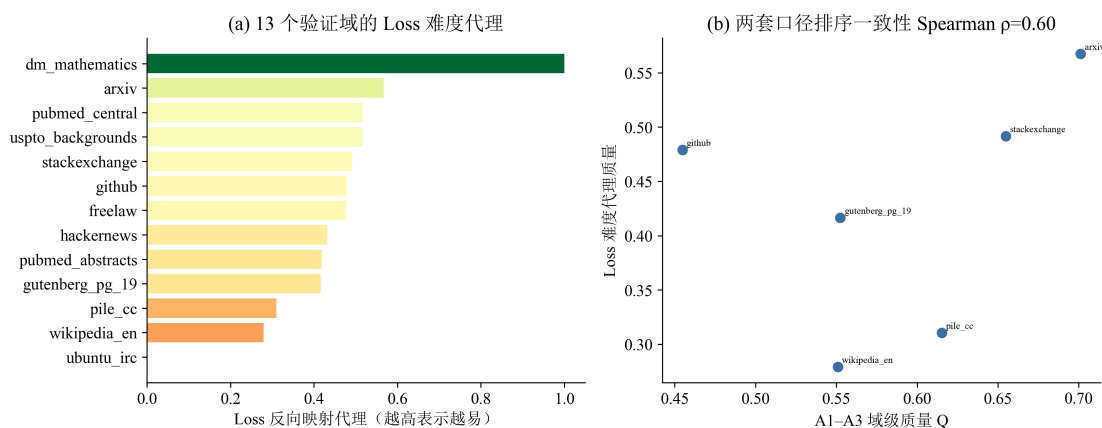
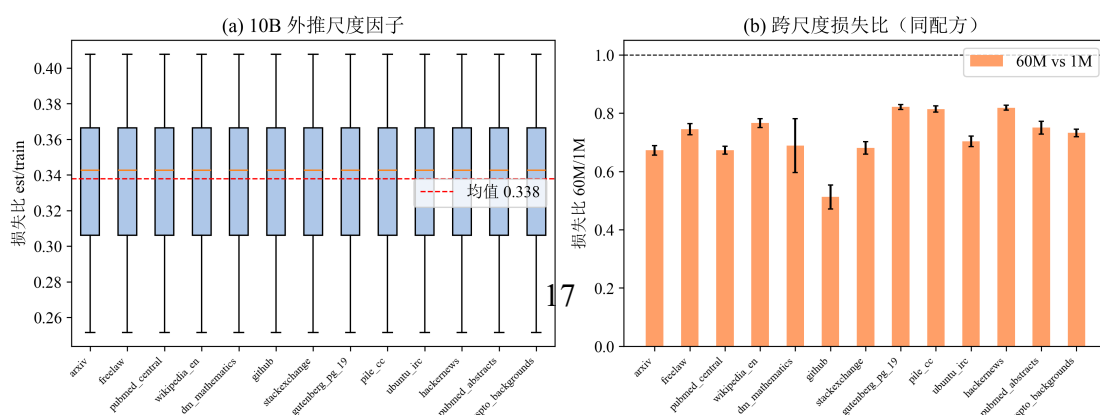


图 7 左：13 个验证域的损失难度代理；右：A1-A3 质量与损失难度代理的排序一致性

表 7 有界推荐配比调整

领域	参考配比	推荐配比	调整量
pubmed_central	0.1137	0.2843	+0.1706
stackexchange	0.0994	0.2486	+0.1491
pubmed_abstracts	0.0662	0.1654	+0.0993
gutenberg_pg_19	0.0469	0.1173	+0.0704
dm_mathematics	0.0255	0.0637	+0.0382
uspto_backgrounds	0.0755	0.0000	-0.0755
freelaw	0.0968	0.0000	-0.0968
github	0.1107	0.0000	-0.1107
arxiv	0.1130	0.0000	-0.1130
pile_cc	0.1195	0.0000	-0.1195

由表7可知，推荐方案应显著增加 pubmed_central、stackexchange、pubmed_abstracts 等学术领域的配比，同时压缩 pile_cc、arxiv、github、freelaw 等领域的占比。需要说明的是，这一结论是针对“13 个验证域加权平均损失”这一目标的优化结果，与“提升单个域自身表现”并不等价（例如 arxiv 的配比下降并不意味着 arxiv 质量低，而是因为其对加权平均损失的边际贡献为负）。外推稳健性方面，10B 与 70B 外推表的损失尺度因子（est/train）在各损失域上高度一致（10B 均值 0.338、标准差 0.038；70B 均值 0.266、标准差 0.036），且同配方下 60M 与 1M 的跨尺度损失比稳定（均值 0.51 ~ 0.82），说明同一套配比—损失系数可在不同参数规模间复用、具备跨尺度外推能力，如图8所示。



5.2 问题二：跨维度数据融合与广义标度律

5.2.1 具体分析

相同的参数规模与训练数据量未必对应相同的验证损失，语料质量和领域构成也是解释差异的关键。问题二据此考察两个层面：质量与配比怎样进入损失模型，以及改善数据能在多大程度上替代参数扩张。附件 A 提供领域配比信息，附件 B 提供规模与质量实验；两者并非联合观测，因而跨来源衔接需要明确评分尺度和效应迁移条件。

本文先区分各数据集承担的估计与检验任务，并核对质量字段的方向。在此基础上，以 Pythia 训练轨迹确定规模关系，通过小模型拟合与大模型留出检验族内迁移，再比较四种质量项构造广义标度律。局部弹性和等损失关系用于解释质量投入，问题一的配比系数用于分析领域效应；各项结论的适用范围由多来源检验限定。图9给出了这些环节之间的联系。

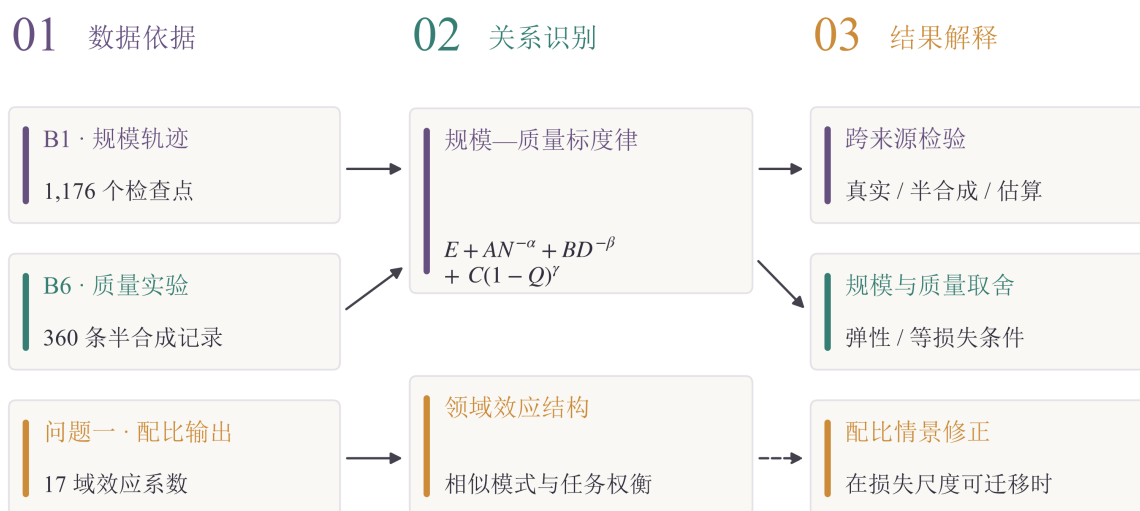


图9 规模、质量与领域效应的分析框架

5.2.2 模型准备

1. 数据组织与变量口径

数据按其能够支持的结论分别使用。B1 记录了 8 个 Pythia 模型各 147 个训练检查点，共 1,176 条记录，是规模项的主要估计依据 [4]；B6 的半合成实验用于估计质量项，B7 只保留未在 B6 出现的记录作扩展检验。轨迹关系由 B2、B3 检查，跨族与文献数据上的表现由 B4、B5 评价。B8 用于诊断质量口径，B9 的模型元数据与 B10 的估算损失则共同用于大规模情景对照。

本节以参数量 N 、训练数据量 D 和质量 Q 解释验证集交叉熵损失 L 。计算时， N 的单位为十亿个参数， D 的单位为十亿 tokens。问题三使用参数个数和 token 数计算算力，因此需要将本节的 N 、 D 数值分别乘以 10^9 。不可约损失 E 作为常数项参与原函数拟合，避免在双对数线性化中将其忽略。

2. 质量字段方向检验

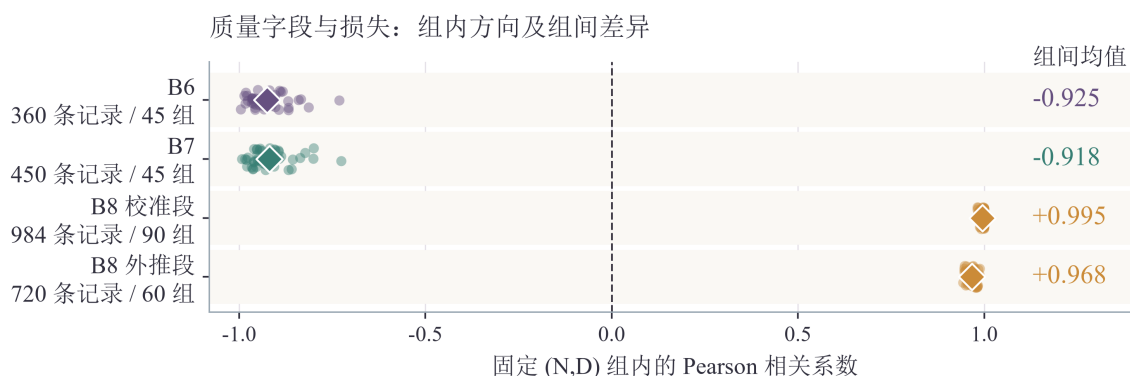
使用质量字段之前，先检查其变化是否与损失变化方向一致。对附件 B 中每个固定 (N, D) 的实验组，计算 Q_{score} 与损失的 Pearson 相关系数，再对各组取算术平均。若平均方向为负，则与质量越高、损失越低的约定相符；正向关系则提示口径差异。表8汇总了检验结果，这里的平均相关只用于描述方向，不作为显著性判据。

表 8 质量字段 Q_{score} 的方向检验（控制 N 、 D 的组内相关）

数据集	样本数	分组数	组内相关 r	结论
B6	360	45	-0.925	Q 越大损失越低：质量指标（ $Q = 1$ 为最优）
B7	450	45	-0.918	含 B6 的全部记录，仅新增点用于检验
B8（校准段）	984	90	+0.995	方向与 B6/B7 相反：口径不一致
B8（外推段）	720	60	+0.968	同上

B6、B7 的平均组内相关分别为 -0.925、-0.918，因此在这两套数据中采用 $Q = Q_{\text{score}}$ 。但两者不能当作相互独立的证据：逐行匹配显示，B7 已包含 B6 的全部 360 条记录，后续检验仅使用其余 90 个新增点。B8 两个子集均呈相反方向，且部分损失低于本文估计的基线下限。基于这一差异，主拟合不使用 B8，仅以 $1 - Q_{\text{score}}$ 作诊断性变换，考察翻转方向后还剩多少预测偏差。

图10进一步保留了各实验组的信息：圆点表示固定 (N, D) 下的组内相关，菱形表示组间平均。B6/B7 与 B8 的点群分居零点两侧，说明方向差异广泛存在于各组中。B6 与 B7 的点群接近还受到样本重叠的影响，不能据此增加独立验证的证据量。



圆点：一个 (N,D) 组；菱形：组间均值。
均为半合成数据；B7 含 B6，不构成独立证据。

图 10 质量字段与损失的组内相关分布

质量方向确定后，还需区分评分尺度。问题一综合评分和附件 B 的 Q_{score} 虽然都随质量提高而增大，但构造方法与数据来源不同，相同分值并不必然代表相同质量。本节质量参数的估计及数值解释均以附件 B 的尺度为准。

5.2.3 模型建立

1. 经典标度律

先将质量固定在参考水平，采用 Chinchilla 标度律 [2] 描述参数与数据规模共同决定的验证损失 L ：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (12)$$

该式将损失分成一个极限项和两个随规模衰减的项。 E 对应 $N, D \rightarrow \infty$ 时的损失极限；参数规模项由系数 A 与指数 α 决定，数据规模项由 B 与 β 决定。由此可在同一参考质量下分别考察两类规模扩张的收益。

2. 质量项的选择与损失修正

质量可以通过改变有效规模或增加损失项进入模型。为比较这些表达，四类候选形式采用同一组联合样本，并以 B6 拟合表现、B7 新增点的预测相关及残差信息评分辅助判断，结果列于表9。经典模型与各候选模型均使用带约束的非线性最小二乘法，保留常数项和幂函数结构；估计时采用 soft- L_1 损失，以减弱大残差的影响。

表 9 广义标度律候选形式的比较

质量项形式	联合 R^2	B6 拟合 R^2	B7 新增点 r	参数数	I_{RSS}
加性质量缺口 $C(1-Q)^\gamma$	0.9896	0.9556	0.991	7	-10236
质量同时乘 N 与 D	0.9882	0.9495	0.991	7	-10042
加性质量亏损 $CQ^{-\gamma}$	0.9848	0.9349	0.991	7	-9660
质量作有效数据乘子	0.9745	0.8905	0.983	6	-8866

四类形式在 B7 新增点上的相关系数均达到 0.983 以上，仅凭相关性难以充分区分。表9显示，加性质量缺口形式同时取得最高的联合拟合与 B6 拟合 R^2 ，据此选定该形式，并在后续检验中用 RMSE、MAPE 评价绝对偏差与相对偏差。

残差信息评分按 $I_{\text{RSS}} = n \ln(\text{RSS}/n) + 2k$ 计算， n 、 k 分别为样本数和参数数。由于参数来自 soft- L_1 稳健估计，该评分只承担同一样本下的辅助比较作用，不解释为严格的高斯极大似然 AIC。B7 新增点同时参与形式比较和扩展检验，故相应表现属于开发阶段验证，不能再视为完全独立的模型评价。

将选定的质量缺口项加入规模基线后，损失关系写为：

$$L(N, D, Q) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} + C(1-Q)^\gamma, \quad C > 0, \gamma > 0 \quad (13)$$

为使质量项具有统一参照，规定 $Q = 1$ 时其值为零。此时广义模型具有经典标度律的函数形式，但联合估计不要求参数等于经典拟合值。Pythia 缺少质量测量，因此其 $Q = 1$ 仅是归一化设定。固定质量并令两类规模趋于无穷，有

$$\lim_{N, D \rightarrow \infty} L(N, D, Q) = E + C(1-Q)^\gamma \geq E \quad (14)$$

这一极限揭示了加性模型中的质量作用：当 $Q < 1$ 保持不变时，增加 N 、 D 只能削减

规模项, 无法消除相对于 E 的质量缺口。上述结论来自模型结构, 向实际训练情景推广仍受拟合数据范围约束。

3. 边际效用与弹性

为刻画各因素微小变化对损失的影响, 对广义模型求偏导, 并在参考点 (N_0, D_0, Q_0) 处评价:

$$\frac{\partial L}{\partial N} = -\alpha AN^{-\alpha-1}, \quad \frac{\partial L}{\partial D} = -\beta BD^{-\beta-1}, \quad \frac{\partial L}{\partial Q} = -\gamma C(1-Q)^{\gamma-1} \quad (15)$$

各偏导的单位不同。令参考点的预测损失为 L_0 , 按变量与损失的比例变化定义弹性:

$$\varepsilon_N = \frac{\partial L}{\partial N} \frac{N_0}{L_0}, \quad \varepsilon_D = \frac{\partial L}{\partial D} \frac{D_0}{L_0}, \quad \varepsilon_Q = \frac{\partial L}{\partial Q} \frac{Q_0}{L_0} \quad (16)$$

4. 质量提升与参数扩张的等效条件

比较质量投入与参数投入时, 固定 D 和领域配比, 让两种方案从同一参考点出发。若微小质量提升与参数扩张带来相同的一阶损失降幅, 则二者的正向等效增量比 R_{NQ} 满足

$$\begin{aligned} \left| \frac{\partial L}{\partial Q} \right| dQ &= \left| \frac{\partial L}{\partial N} \right| dN_{eq}, \\ R_{NQ} &= \frac{dN_{eq}}{dQ} = \frac{\gamma C(1-Q)^{\gamma-1}}{\alpha AN^{-\alpha-1}} = \frac{\gamma C}{\alpha A} (1-Q)^{\gamma-1} N^{\alpha+1}. \end{aligned} \quad (17)$$

这里比较的是两项改善方案的等效增量。若沿一条等损失曲线移动, 则有 $dN/dQ = -R_{NQ}$, 质量提高对应所需参数减少。固定 Q 后, R_{NQ} 按 $N^{\alpha+1}$ 增长, 原因在于参数扩张的边际收益不断减弱。损失效果相等并不意味着算力成本相等, 后者还取决于质量处理与参数训练的成本。

对于有限的质量增量 ΔQ , 一阶近似未必足够。先记质量改善产生的损失降幅为 δ_Q , 再令参数扩张产生同样的降幅, 可直接解得

$$\begin{aligned} \delta_Q &= C[(1-Q)^\gamma - (1-Q-\Delta Q)^\gamma], \\ \Delta N_{eq} &= N \left[\left(1 - \frac{\delta_Q N^\alpha}{A} \right)^{-1/\alpha} - 1 \right]. \end{aligned} \quad (18)$$

质量增量须满足 $0 < \Delta Q \leq 1-Q$ 。当 $\delta_Q < AN^{-\alpha}$ 时, 等效参数增量存在有限解; 达到等号时需要无穷大的参数量, 超过该阈值则无法仅靠扩大参数获得同等降幅。这是因为当前参数项 $AN^{-\alpha}$ 给出了参数扩张能够消除的全部损失。随着 $\Delta Q \rightarrow 0$, 有限增量关系趋于 $\Delta N_{eq} \approx R_{NQ} \Delta Q$ 。若求得的规模超出拟合区间, 应按模型外推解释。

5. 领域效应的相似性

领域之间的差异由其对各验证任务的作用描述。问题一线性混合模型给出训练域 i, j 在 13 个验证域上的效应向量 $\mathbf{a}_i, \mathbf{a}_j$ 。将它们中心化后, 采用余弦相似度衡量相对效应模式:

$$s_{ij} = \frac{\langle \mathbf{a}_i - \bar{\mathbf{a}}, \mathbf{a}_j - \bar{\mathbf{a}} \rangle}{\|\mathbf{a}_i - \bar{\mathbf{a}}\| \|\mathbf{a}_j - \bar{\mathbf{a}}\|} \quad (19)$$

中心向量 $\bar{a}$ 为 17 个训练域效应向量的平均值。 $s_{ij} \rightarrow 1$ 表示两域在验证任务上的相对作用趋同， $s_{ij} \rightarrow -1$ 则表示作用模式相反，提示任务收益之间的权衡。该度量不保留效应幅值；同时，线性配比模型未包含领域交互项。因此，相似度本身不能确定等量替代是否有效，也不能证明联合使用具有额外收益。

5.2.4 模型求解

1. 经典标度律拟合与留出验证

以 B1 的全部 1,176 条记录估计规模关系。图11将损失随参数量和数据量的变化置于左侧，右侧按模型规模汇总残差。相应拟合函数为

$$L(N, D) = 1.6898 + \frac{0.3540}{N^{0.3400}} + \frac{1.2403}{D^{0.2799}} \quad (20)$$

全量拟合得到 $R^2 = 0.9999998$ 、 $RMSE = 1.5 \times 10^{-4}$ 、 $MAPE = 0.004\%$ 。为检查族内迁移，再仅用最小的 4 个模型估计参数，将其余 4 个较大模型留出；预测 R^2 为 0.9999997， $MAPE$ 为 0.005%。这一结果支持在该模型族内沿用所估规模关系。

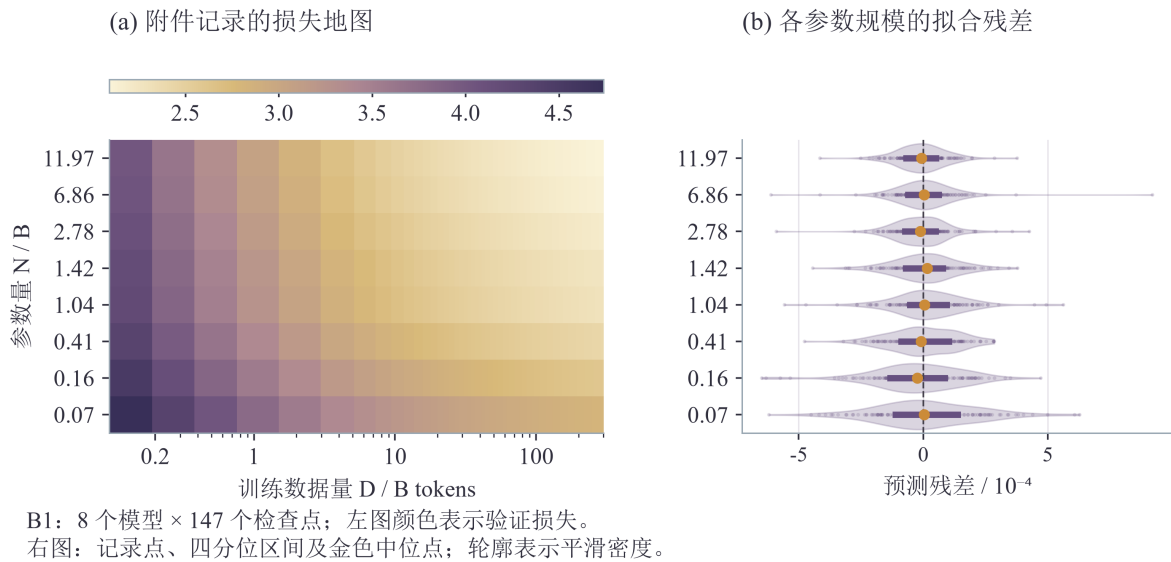


图 11 Pythia 规模-损失地图与分模型残差分布

拟合残差标准差为 1.5×10^{-4} ，损失标准差则为 0.342，两者相差较大，说明所选函数能够高度贴合附件 B1 的规模关系。这样的族内表现尚不足以说明跨来源泛化，仍需分别考察 B4、B5 的外部数据和 B2 的半合成轨迹。

B3 提供了另一项族内检查。对 8 条插值轨迹分别固定 N ，以扣除极限项和参数项后的 $L - (E + AN^{-\alpha})$ 对 D 作双对数回归，幂指数落在 $-0.2806 \sim -0.2803$ ，接近经典拟合的 $-\beta = -0.2799$ 。这表明插值轨迹与经典规模关系基本自治；由于轨迹来自 Pythia 检查点插值，这项检查不增加独立观测证据。

2. 广义标度律参数估计

接着合并 Pythia 规模记录与 B6 质量实验，估计表9所选形式的参数。Pythia 无实测质量信息，按前述约定取参考质量 $Q = 1$ ；估计结果见表10。

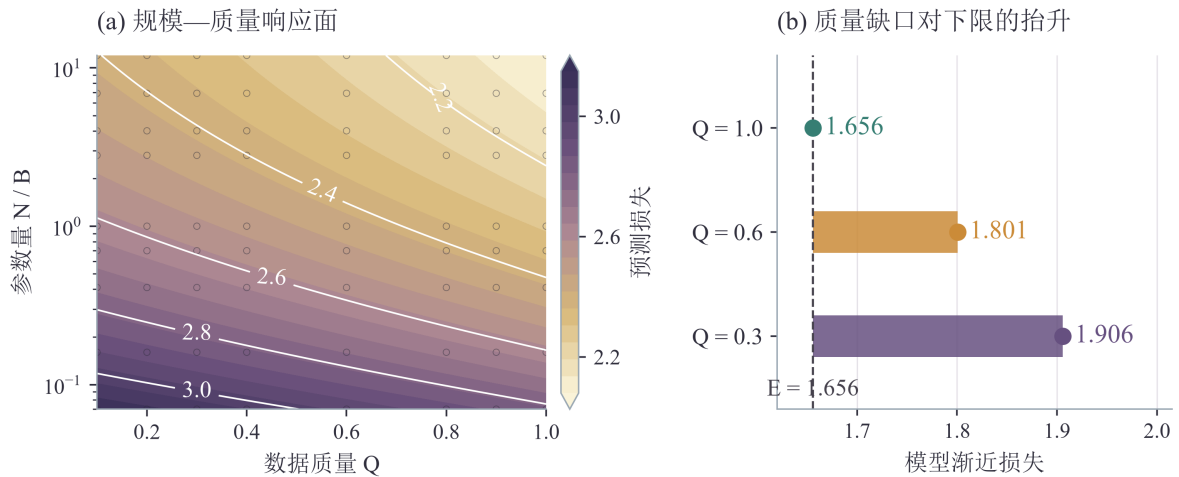
表 10 广义标度律参数估计

参数	估计值	参数	估计值
不可约损失 E	1.6558	数据规模指数 β	0.2767
规模系数 A	0.3808	质量缺口系数 C	0.3544
规模指数 α	0.3266	质量指数 γ	0.9779
数据系数 B	1.2515	联合拟合 R^2	0.9896

联合拟合的 $R^2 = 0.9896$ 、 $RMSE = 0.036$ 、 $MAPE = 0.66\%$ 。表10中 $\gamma = 0.9779 \approx 1$ ，意味着渐近损失随质量缺口近似线性上升。这里的联合样本与经典拟合不同，因此不能将两个 R^2 的大小直接解释为质量项带来的精度变化。

图12从两个角度呈现质量效应。左侧固定 $D = 300B$ tokens，以底色和等值线绘制模型的规模-质量响应面，空心点标记 B6 半合成实验位置；右侧从基线 E 出发，分解不同质量缺口引起的渐近损失增量。

代入渐近关系 $E + C(1 - Q)^\gamma$ ，参考质量 $Q = 1$ 对应下限 1.656；质量降至 $Q = 0.6$ 和 $Q = 0.3$ 时，下限分别升至 1.801 和 1.906。



$D=300B$ tokens；空心点：B6 实验位置；底色与等值线：模型预测。

图 12 规模-质量响应面与渐近损失分解

3. 分层检验与适用范围

模型能否用于配置分析，需要结合数据来源判断。表11列出 6 组数据的检验指标，图13补充误差分布。图左侧采用有符号相对误差 $100(\hat{L} - L)/L$ ，负值表示低估、正值表示高估；为同时呈现较小与较大的偏差，横轴在 $\pm 10\%$ 内保持线性，外侧使用对称对数刻度。图右侧汇总 MAPE 与相关系数。

在跨族数据 B4 和文献数据 B5 上，相关系数分别为 0.975、0.910，MAPE 分别为 8.8%、8.0%，见表11。模型能够反映部分跨族规模趋势，但预测损失仍受来源差异影响，不能仅凭较高相关忽略绝对偏差。

表 11 广义标度律的分层验证

验证来源	样本数	性质	r	RMSE	MAPE(%)
B4 跨族收敛点	57	真实	0.975	0.293	8.8
B5 已发表标度律	44	真实	0.910	0.193	8.0
B7 质量扩展新增点	90	半合成	0.991	0.056	1.6
B2 Cerebras 族外	1,029	半合成	0.666	1.245	33.7
B8 大规模矩阵（方向重定向）	984	半合成	0.638	1.294	133.4
B10 百亿参数以上	128	估算	1.000	0.020	1.0

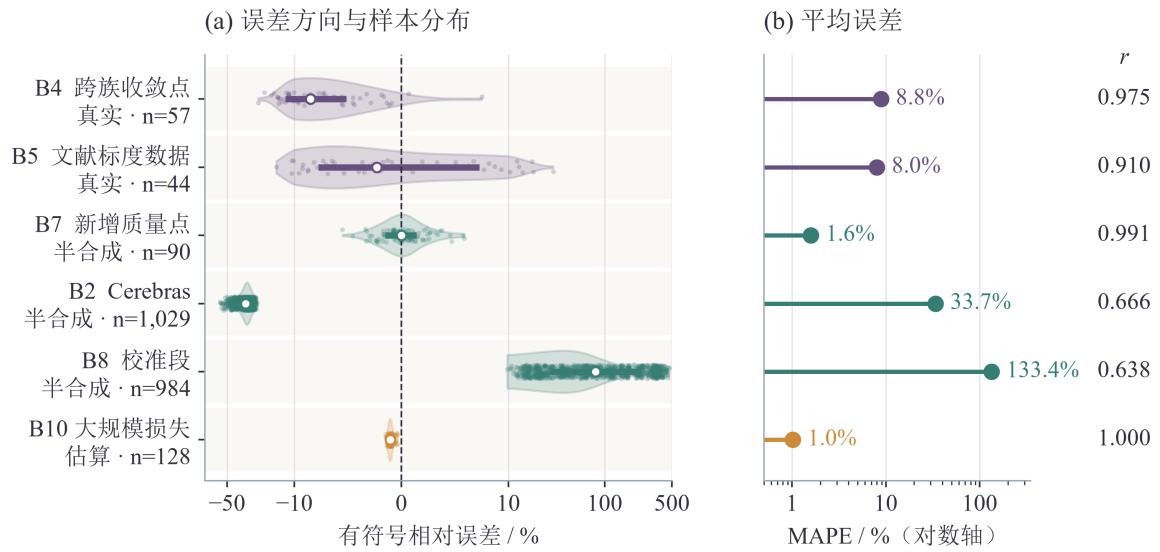


图 13 不同数据源的相对误差分布与验证指标

B7 的 90 个新增点得到 1.6% 的 MAPE，支持质量关系在同一半合成框架内的扩展一致性。B10 则提供估算损失，检验统一取 $Q = 1$ ；它只能用于比较估算关系，既不构成独立实验验证，也不提供质量项的验证证据。

偏差较大的数据揭示了模型边界。B2 的 MAPE 为 33.7%，B8 校准段即使翻转质量方向，MAPE 仍为 133.4%。此外，B8 部分损失已经低于本文估计的渐近下限。方向调整无法消除这些差异，说明不同来源的损失尺度或生成机制不能由当前模型统一解释。

定量解释还须对照实际估计区间：B1 的 N 、 D 分别约为 0.07–12B、0.134–299.893B tokens，B6 对应区间约为 0.07–11.97B、10–600B tokens。质量效应的数值依据主要是 B6/B7 半合成实验；超出这一背景的大规模估算记录，只承担外推一致性对照的作用。

4. 大规模模型的对应与外推范围

进一步核对大规模记录的对应关系。按 B9 模型名称与 B10 模型族名称匹配后, B10 的 128 条记录全部获得一对一对应, 参数量 N 与数据量 D 也逐条相同。B9 共 132 条, 其中未匹配的 4 条将训练数据量记为 0, 且缺少 B10 损失, 因此予以排除。表12按参数规模汇总匹配样本。

表 12 B9 模型元数据与 B10 估算损失的匹配对照

参数规模 N/B	匹配记录数	数据量 D/B tokens	MAPE/%
[100, 300)	65	1.3–36,000	0.94
[300, 1000)	43	0.3–30,000	1.08
[1000, 10000]	20	0.1–36,000	1.12

注: 统一取参考质量 $Q = 1$; MAPE 比较本文模型预测与 B10 估算值, 数据量列为各组的实际最小值和最大值。

这些记录的参数规模为 100–10,000B, 已全部越过主拟合上限。各组相对误差均约为 1%, 反映出本文模型与 B10 所用估算关系接近。由于 B10 本身由标度律估算生成, 低误差不能替代真实大模型实验, 也不能据此将质量效应直接推广到该规模区间。

5. 质量投入的局部收益与规模替代

在给出应用范围后, 选取两个参考点考察投入方向。表13列出局部弹性与质量—参数等效增量比, 用于比较比例变化的影响及其随参考位置的差异。

表 13 弹性与质量—参数替代关系

$N_0(B)$	$D_0(B)$	Q_0	ε_N	ε_D	ε_Q	R_{NQ} (B/单位质量)
1	100	0.6	−0.0491	−0.0383	−0.0838	2.844
12	300	0.6	−0.0248	−0.0321	−0.0953	76.838

在参考点 $(N_0, D_0, Q_0) = (1, 100, 0.6)$, 质量弹性绝对值约为参数弹性的 1.7 倍, 意味着相同比例的微小质量改善带来更大的损失降幅。两个参考点的局部替代率分别为 2.844 和 76.838 B/单位质量。若以 0.1 的质量增量作一阶估计, 得到的等效参数增量为 0.284B、7.684B。它们只表示局部损失效果, 不表示等成本投入; 有限幅度配置须使用式(18), 并核对解所在的规模是否超出拟合范围。

图14进一步展示边际收益怎样随参考位置变化: 前两幅点图分别给出参数和质量的边际导数绝对值, 第三幅用成对标记比较两个参考点的弹性绝对值。前两幅导数各有量纲, 不宜直接按数值大小比较, 第三幅弹性绝对值则可用于比较相同比例变化的影响幅度。

图15把局部比较扩展为固定 $D = 300B$ tokens 下的等损失配置。三个子图各给定一个目标损失, 横轴为参数量, 纵轴为所需质量; 实线对应主拟合规模范围, 虚线对应外

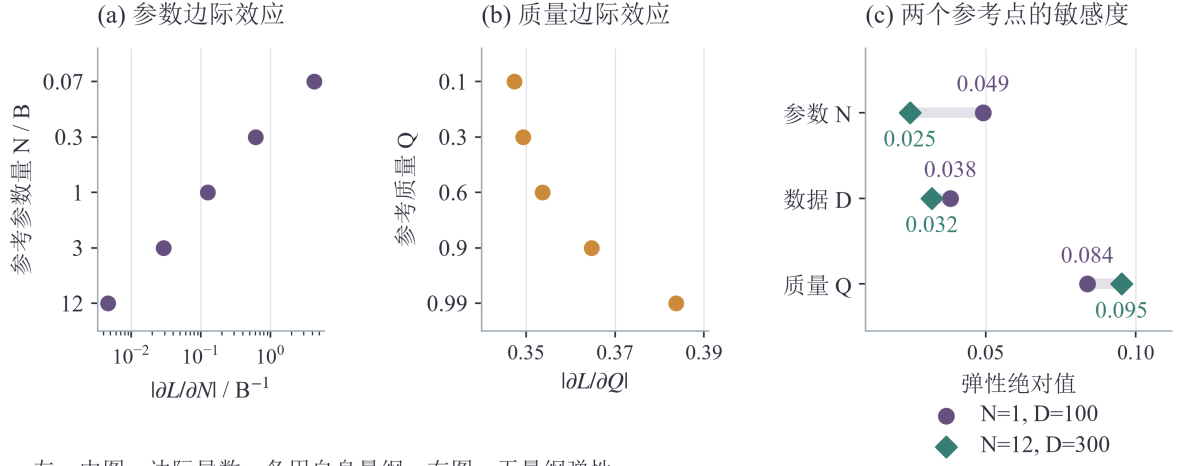


图 14 参考位置的边际效应与弹性对比

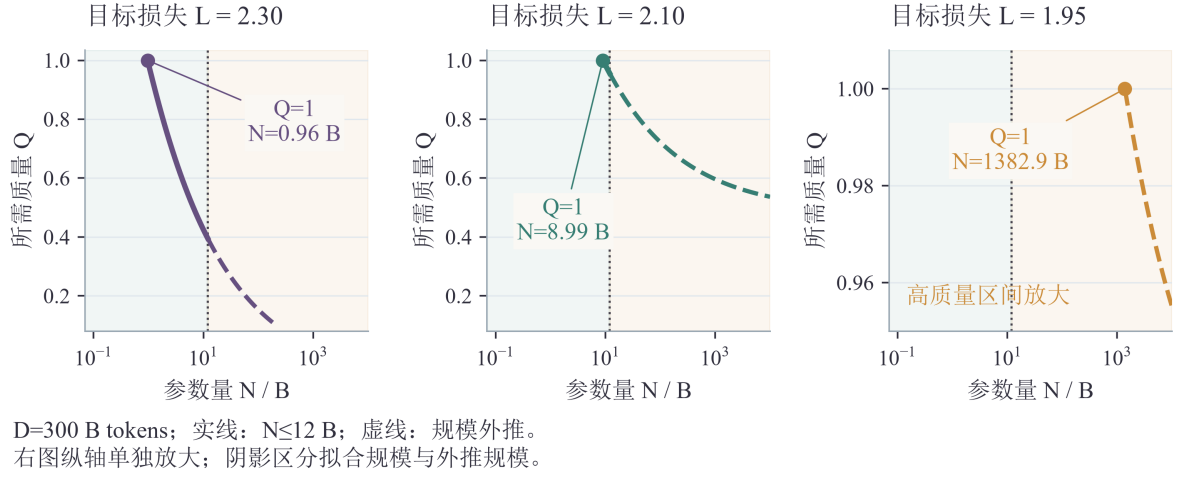


图 15 三个目标损失下的规模-质量配置边界

推。 $L = 1.95$ 的子图放大高质量区间，横轴延伸至 10^4B 仅为显示理论关系，不代表工程可行。还需注意， $\gamma = 0.9779 < 1$ 使 $|\partial L / \partial Q|$ 在 $Q < 1$ 区间内递增，并在 $Q \rightarrow 1^-$ 时发散。因此，接近 $Q = 1$ 时不宜依据导数给出资源配置判断，质量边际效应的定量解释应避开这一端点。

6. 领域效应与配比衔接

规模与质量的分析之外，领域配比还会改变各验证任务的损失。图16左侧以 $d_{ij} = \sqrt{2 - 2s_{ij}}$ 为距离，通过平均连接法聚类 17 个训练域；右侧选取相似度最高和最低的各四对领域，绿色、金色分别标记相对效应趋同与相反。聚类展示的是效应结构，不代表领域联合使用存在额外收益。

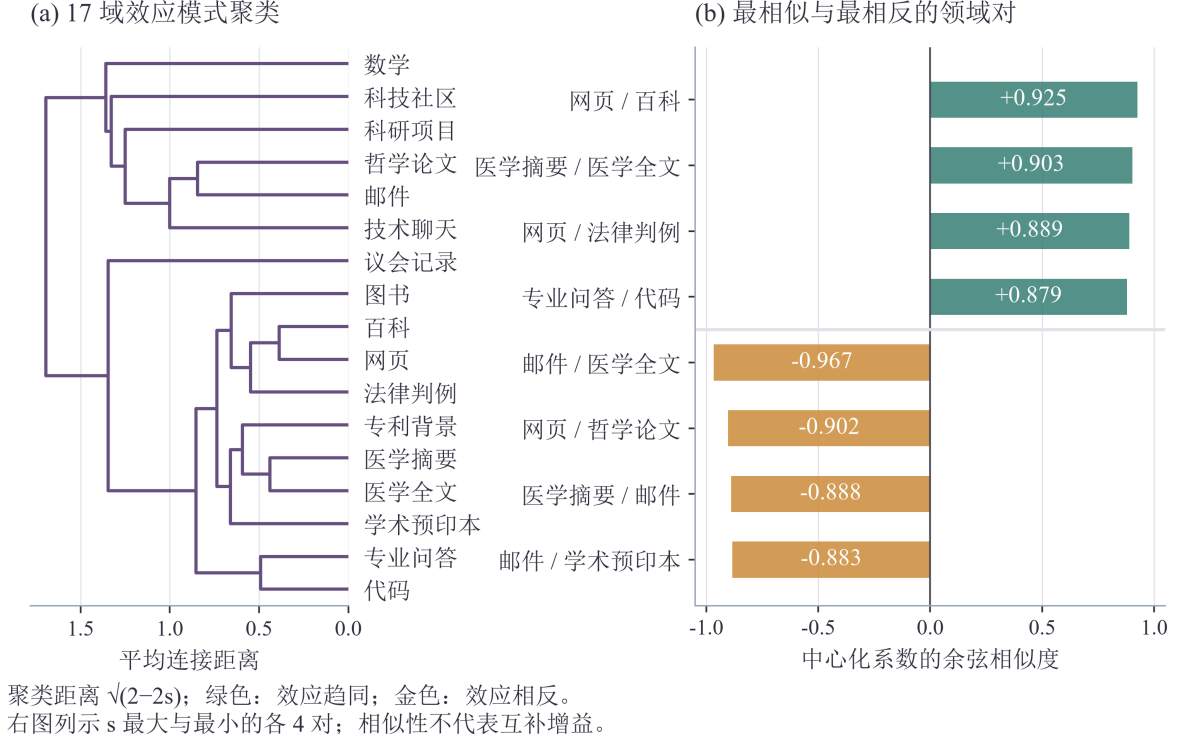


图 16 训练领域效应聚类与典型领域对比较

从图16可见，百科与网页、医学全文与医学摘要的相似度为 0.925、0.903，其在 13 个验证域上的相对效应较接近；医学全文与邮件的相似度为 -0.967 ，则体现出明显的任务收益差异。判断能否替换某一领域，还必须结合效应幅值与验证任务权重。具体地，在单纯形内将比例 h 从领域 i 转移至 j ，并满足 $0 \leq h \leq p_i$ ，线性模型给出的加权损失变化为

$$\Delta \hat{L}_A = h \mathbf{w}^\top (\mathbf{a}_j - \mathbf{a}_i), \quad (21)$$

其中 $\mathbf{w}$ 为验证域权重。损失变化小于零、等于零或大于零，分别表示这次替换改善、保持或降低加权表现。因而，效应方向相似不等于等量替换无损，效应相反也只提示任务间权衡；在没有领域交互项的线性模型中，无法进一步识别联合使用的互补增益。

对问题一推荐配比与参考配比作同口径比较，线性模型给出的加权损失差为

$$M = \hat{L}(\mathbf{p}_{\text{rec}}) - \hat{L}(\mathbf{p}_{\text{ref}}) = -0.1988.$$

这一差值基于附件 A 的验证域及权重。若配比效应可与规模、质量效应分离，且损失增量能够跨来源迁移，就可将 $M(\mathbf{p})$ 写成广义标度律的加性修正。多来源检验已显示损失水平存在差别，因此 0.1988 只描述上述口径下的结果，不能理解为各规模均可获得的固定降幅。

在上述分离与迁移条件下，规模、质量和配比的关系可合写为

$$L(N, D, Q, \mathbf{p}) = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^{\gamma} + M(\mathbf{p}),$$

$$M(\mathbf{p}) = \sum_{i=1}^{17} (p_i - p_{\text{ref},i}) \mathbf{w}^T \mathbf{a}_i. \quad (22)$$

以参考配比为零点，有 $M(\mathbf{p}_{\text{ref}}) = 0$ 。配比固定时， M 改变损失水平，但不改变 N, D, Q 的边际导数和等损失替代条件；本节报告的弹性使用参考配比。质量参数仍定义在附件 B 的评分尺度上，若将问题一评分直接用于后续配置，则需额外接受尺度衔接假设，这一点不能由加性表达式本身验证。

上述结果为资源配置提供了带适用条件的损失关系：参数扩张的边际收益随规模增大而降低，质量改善的等效参数增量则随参考点变化。跨族和文献数据的 MAPE 为 8.8%、8.0%，提示后续配置应同时考虑预测偏差。质量尺度、数据来源与外推范围共同限定了这些结论的解释边界。

5.3 问题三：算力约束下的资源配置与结构性转移

5.3.1 具体分析

在总算力固定时，扩大模型、增加训练数据和提高数据质量会争夺同一笔预算。更大的模型能够降低参数规模项，却提高了每个 token 的训练成本；更高的质量能够减小质量缺口，却需要处理整份训练语料。因而，问题三的关键是比较这些投入带来的损失改善与算力代价，而非分别寻找三个变量的最大值。

问题二已经给出参数量、数据量和质量与损失之间的关系。本节据此加入成本约束，考察预算增加后最优配置的变化。算力最优训练研究中的参数-数据协同配置为这一分析提供了基础 [2]；本题进一步计入质量处理和长上下文开销，并比较不同成本设定下的选择。

5.3.2 模型准备

本节直接使用问题二估计的标度律参数，取 $E = 1.6558$ 、 $A = 0.3808$ 、 $\alpha = 0.3266$ 、 $B = 1.2515$ 、 $\beta = 0.2767$ 、 $C_q = 0.3544$ 、 $\gamma = 0.9779$ 。其中质量项系数记为 C_q ，与算力预算 C_b 区分。计算成本时， N 和 D 分别使用参数个数与 token 数；代入标度律时换算为 $n = N/10^9$ 、 $d = D/10^9$ ，结果表中的 B 均表示十亿。

质量提升从参考语料出发。设 $\bar{p}_k$ 为 A4 配方的平均领域占比， Q_k 为问题一经 A16

映射得到的领域评分，则基线取为

$$Q_0 = \sum_{k=1}^{17} \bar{p}_k Q_k = 0.5510.$$

这一加权方式与参考配方保持一致，未加权样本均值 0.4952 则用于基线灵敏度对照。沿用前两问的衔接设定，将该评分代入质量标度律，并限定 $Q_0 \leq Q \leq 1$ ；评分的跨来源数值可比性仍属于模型假设， $Q = 1$ 表示设定的参考质量上界。

基准情景采用参考配比 $M = 0$ 、幂函数型质量成本及 2048 tokens 上下文。对照分析分别更换质量成本函数和上下文长度，其余条件保持一致。上下文长度由任务与架构外生给定，取附件 C7 的五个档位：2048、4096、8192、32768 和 131072 tokens。问题一的推荐配比以 $M = -0.1988$ 作补充情景，讨论其对损失水平的影响。

5.3.3 模型建立

1. 每个训练 token 的算力代价

基础训练成本采用 $6ND$ 近似 [2]，质量处理和注意力成本采用赛题给定形式 [14]。在允许的质量范围内，三类 $g(Q)$ 均单调增加，故记 $\Delta g(Q) = g(Q) - g(Q_0) \geq 0$ ，总成本可整理为

$$C_{\text{tot}} = 6ND + D\Delta g(Q) + \eta NDL_{\text{ctx}} = D \underbrace{[(6 + \eta L_{\text{ctx}})N + \Delta g(Q)]}_{\text{单位 token 的合计算力成本}}, \quad \eta = 2 \times 10^{-4}. \quad (23)$$

这一写法说明，增加数据量会同时增加训练、注意力与质量处理开销；提高质量虽然不改变模型参数量，却增加每个 token 的处理成本。即使质量已经达到上界，新增数据仍需支付这笔费用。

题设的三类质量成本列于表14。比较时使用增量 $\Delta g(Q)$ ，各函数在 Q_0 处的增量成本均为零。

表 14 质量处理成本函数 (g 的单位为 FLOPs/token)

成本类型	$g(Q)$	本节用途
幂函数型	$5 \times 10^9 Q^4$	基准配置
指数型	$10^7 e^{6Q}$	成本形式对照
对数渐进型	$2 \times 10^9 \ln(1 + 10Q)$	成本形式对照

2. 预算约束下的配置模型

记 $k = 6 + \eta L_{\text{ctx}}$ 。给定配比和上下文长度后，以预测验证损失最小为目标，得到

$$\begin{aligned} \min_{N,D,Q} \quad & E + An^{-\alpha} + Bd^{-\beta} + C_q(1 - Q)^\gamma + M, \\ \text{s.t.} \quad & D[kN + \Delta g(Q)] \leq C_b, \quad Q_0 \leq Q \leq 1, \\ & 10^5 \leq N \leq 10^{15}, \quad 10^6 \leq D \leq 10^{16}. \end{aligned} \quad (24)$$

后两项规模界限用于限定数值搜索域，并非实际语料供给上限。 M 为固定的配比修正项，因此改变 M 只平移目标函数，不改变本模型的最优配置。

固定质量且规模界限不活跃时，增加 N 或 D 都能降低损失，预算约束因而取等号。将其写为

$$D = \frac{C_b}{kN + \Delta g(Q)} \quad (25)$$

即可看出两类竞争：提高 N 会压缩可负担的数据量，提高 Q 也会通过处理成本减少 D 。最优配置需要在直接收益与这部分间接损失之间取得平衡。

3. 参数量与数据量的平衡

为在绝对数量口径下求导，令 $\tilde{A} = A(10^9)^\alpha$ 、 $\tilde{B} = B(10^9)^\beta$ 。将式(25)代入规模损失项，固定 Q 后得到

$$F(N) = \tilde{A}N^{-\alpha} + \tilde{B} \left[\frac{kN + \Delta g(Q)}{C_b} \right]^\beta.$$

令 $F'(N) = 0$ ，整理可得

$$\alpha \tilde{A} N^{-\alpha} \left(1 + \frac{\tau(Q)}{N} \right) = \beta \tilde{B} D^{-\beta}, \quad \tau(Q) = \frac{\Delta g(Q)}{k}. \quad (26)$$

$\tau(Q)$ 是把质量处理费用按单位参数训练成本折算的附加规模，与 N 同口径。式(26)把质量投入对参数-数据平衡的影响集中在 $1 + \tau/N$ 中；因此，付费提升质量后，原有的两类规模项平衡也会改变。

若质量保持在基线， $\tau = 0$ ，则有 $\alpha \tilde{A} N^{-\alpha} = \beta \tilde{B} D^{-\beta}$ 。再结合 $kND = C_b$ ，在固定上下文条件下得到

$$N^* \propto C_b^{\beta/(\alpha+\beta)}, \quad D^* \propto C_b^{\alpha/(\alpha+\beta)}. \quad (27)$$

两个指数分别约为 0.459 和 0.541，数据量随预算增长略快于参数量。这一关系作为无额外质量投入时的基准；跨越质量提升阶段时，配置应由完整模型确定。

4. 数值求解与阶段识别

采用 SLSQP 求解式(24)，以 $\ln N$ 、 $\ln D$ 为规模变量，减小数量级差异。三档预算分别使用 8 组初值，预算扫描使用 5 组初值，在求解成功的结果中取损失最低者。下文将这一数值候选解记为 (N^*, D^*, Q^*) ；多初值不能保证全局最优，且 $\gamma < 1$ 时端点 $Q = 1$ 的导数不有界，故饱和配置按边界解处理。

为比较不同预算下的质量投入，定义完成度 $\Theta = (Q^* - Q_0)/(1 - Q_0)$ 与质量份额 $f_Q = D^* \Delta g(Q^*)/C_b$ 。质量处于基线、内部或上界，分别对应 $\Theta = 0$ 、 $0 < \Theta < 1$ 和 $\Theta = 1$ 。在 10^{18} – 10^{25} FLOPs 范围设置 141 个对数等距预算点，相邻预算比约为 1.122，以有效解中首次出现正质量投入和达到上界的位置识别阶段变化。扫描中 1.41×10^{19} FLOPs 处求解失败，分析使用其余 140 点。图中标记对应有效记录，跨缺测点的点线仅连接两端，不代表补算结果。

5.3.4 模型求解

1. 预算增加后，质量与规模如何分配

表15给出基准情景的三档配置。预算增加时，参数量和数量均上升，但二者并非按固定比例扩张： D/N 从 30.9 增至 76.1，高预算配置使用了相对更多的训练数据。质量则从基线水平升至上界，中、高两档的质量水平相同。

表 15 三档预算的基准配置（幂函数型成本， $L_{\text{ctx}} = 2048$ ， $M = 0$ ）

预算 (FLOPs)	N^* (B)	D^* (B tokens)	Q^*	L^*	D^*/N^*	f_Q (%)
10^{19}	0.225	6.9	0.551	3.170	30.9	0.0
10^{22}	6.089	229.5	1.000	2.145	37.7	10.4
10^{24}	44.937	3418.0	1.000	1.897	76.1	1.6

三档结果尚不能说明质量在何时提升。为此，图17将预算扫描结果与三档配置联系起来，左图展示质量完成度，右图展示三类成本的份额。

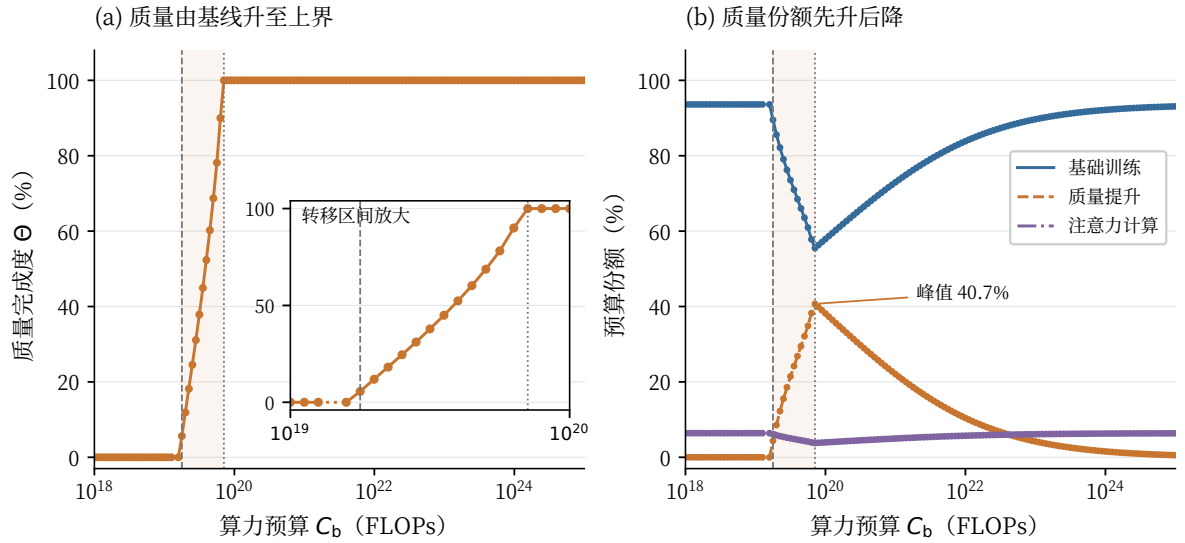


图 17 预算增加时的质量配置与成本份额（幂函数型， $L_{\text{ctx}} = 2048$ ， $M = 0$ ）。内嵌图放大质量提升区间；标记对应 CSV 中的有效记录，竖线为网格识别的启动与饱和位置；跨缺测位置的点线仅引导视线。

在预算较低的有效扫描点上， $Q^* = Q_0$ ，基础训练和注意力计算分别占预算的 93.6% 和 6.4%。增加质量虽可降低质量缺口项，但处理成本会减少可用于训练的数据量；这一阶段的数值解选择保持基线质量。

预算增至 1.78×10^{19} FLOPs 时，首次出现额外质量投入。此后质量完成度与质量份额同步上升，至 7.08×10^{19} FLOPs，质量达到上界，质量份额也达到扫描峰值 40.7%。前一有效点的质量约为 0.955，因此扫描将饱和边界定位在 $[6.31, 7.08] \times 10^{19}$ FLOPs 之

间；相应的启动边界位于 $[1.58, 1.78] \times 10^{19}$ FLOPs 之间。这些区间反映当前网格分辨率，不能解释为精确的临界预算。

质量达到上界后，进一步投入无法继续提高 Q ，预算扩张转而支持参数量和数据量增长。此时 $\Delta g(1)$ 固定，由预算等式可得

$$f_Q = \frac{\Delta g(1)}{kN^* + \Delta g(1)}.$$

随着参数量上升，质量份额自然下降，至 10^{24} FLOPs 时约为 1.6%。这并不要求质量处理的总费用下降：新增 token 仍支付相同处理单价，只是模型训练费用增长得更快。质量饱和的原因是可行上界，而非边际质量收益趋零；后者与问题二估计的 $\gamma < 1$ 不符。

2. 低预算的质量选择取决于成本形式

上述阶段变化是在幂函数型成本下得到的。若质量处理方式改变，相同预算是否仍应保持基线质量？图18比较三类成本设定下的质量水平与投入份额，表16给出相应配置。

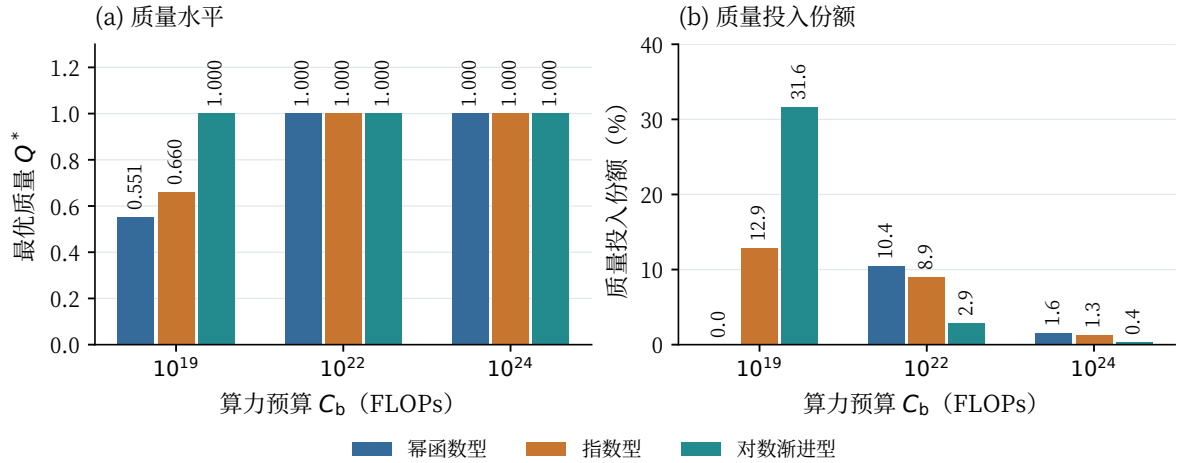


图 18 三类成本设定下的质量配置与投入份额。柱高直接取自九组求解结果 ($L_{\text{ctx}} = 2048$, $M = 0$)，函数形式见表14。

在 10^{19} FLOPs 下，三类成本给出了不同选择：幂函数型保持 $Q^* = 0.551$ ，指数型将质量提高至 0.660，对数渐进型则达到上界。对数渐进型在这一预算下的质量投入份额也最高，为 31.6%。三种配置的预测损失相差约 0.057，但质量份额从 0 到 31.6% 不等，说明相近损失可以对应明显不同的投入结构。

当预算达到所考察的中、高两档时，三类成本均给出 $Q^* = 1$ ，质量是否饱和的判断一致。规模配置仍有差别： 10^{22} FLOPs 下的数据量为 229.5–274.2B tokens，最大值较最小值高约 19.5%。因此，基准情景的低预算策略不宜推广到所有成本形式；即使质量水平相同，模型规模和数据量仍需按具体处理成本配置。

表 16 三类质量成本下的配置对比 ($L_{\text{ctx}} = 2048$, $M = 0$)

预算 (FLOPs)	成本类型	N^* (B)	D^* (B tokens)	Q^*	L^*	f_Q (%)
10^{19}	指数型	0.265	5.1	0.660	3.163	12.9
	幂函数型	0.225	6.9	0.551	3.170	0.0
	对数渐进型	0.354	3.0	1.000	3.113	31.6
10^{22}	指数型	5.972	237.9	1.000	2.144	8.9
	幂函数型	6.089	229.5	1.000	2.145	10.4
	对数渐进型	5.527	274.2	1.000	2.138	2.9
10^{24}	指数型	44.796	3437.7	1.000	1.897	1.3
	幂函数型	44.937	3418.0	1.000	1.897	1.6
	对数渐进型	44.298	3509.0	1.000	1.897	0.4

3. 长上下文如何改变配置

延长上下文会提高式(23)中的 k 。注意力与基础训练的成本比为 $\lambda = \eta L_{\text{ctx}}/6$ ，两者相等时

$$L_{\text{crit}} = \frac{6}{\eta} = 30,000 \text{ tokens.} \quad (28)$$

该值是两项成本的分界；配置如何变化，还需连同质量处理成本一起考察。固定预算为 10^{22} FLOPs，五档结果列于表17。

表 17 上下文长度敏感性 ($C_b = 10^{22}$ FLOPs，幂函数型成本， $M = 0$)

L_{ctx} (tokens)	λ	N^* (B)	D^* (B tokens)	Q^*	L^*	注意力份额 (%)
2048	0.068	6.089	229.5	1.000	2.145	5.7
4096	0.137	5.897	223.5	1.000	2.149	10.8
8192	0.273	5.562	212.7	1.000	2.157	19.4
32768	1.092	4.318	170.2	1.000	2.194	48.2
131072	4.369	2.705	109.1	1.000	2.273	77.3

上下文由 2048 增至 131072 tokens 时，注意力份额从 5.7% 升至 77.3%，参数量和数据量分别减少约 55.6% 和 52.5%，预测损失增加约 0.128。五档质量均保持为 1，额外注意力开销主要压缩了规模投入。值得区分的是，32768 tokens 已使注意力成本超过基础训练，但其总预算份额仍为 48.2%，因为质量处理也占用预算。

图19右侧显示，低预算下的响应有所不同： 10^{19} FLOPs 时，已有计算点的质量由 0.551 升至约 0.670。较长上下文压缩了可负担的数据量，而质量处理费用正比于 D ；处理更少的数据使提高整份语料质量的代价降低，从而改变了质量与规模的取舍。这一解释来自题设成本中的数据量因子，适用于当前模型中的联动关系。

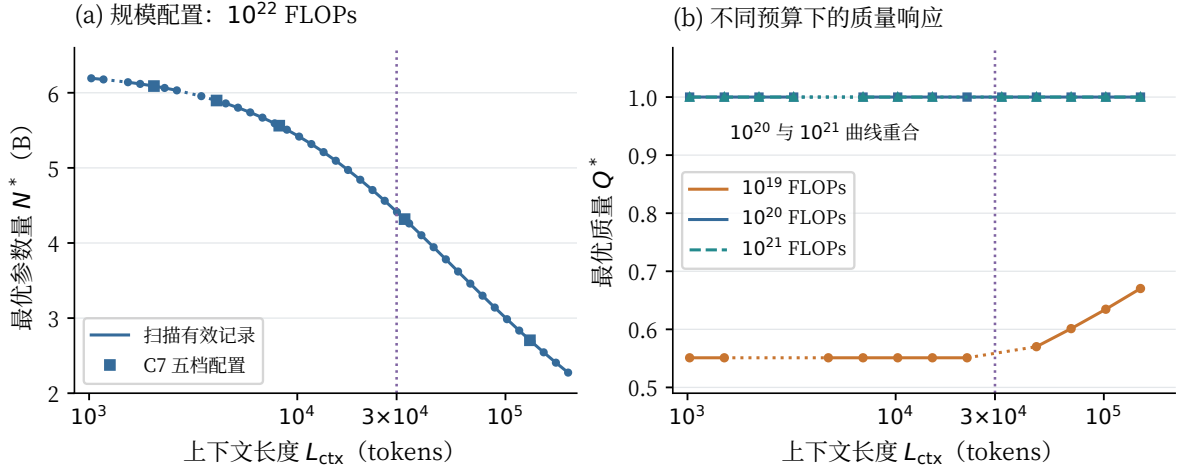


图 19 上下文长度对配置的影响（幂函数型， $M = 0$ ）。左图预算为 10^{22} FLOPs，方形点为 C7 档位；右图比较三档较低预算，其中 10^{20} 与 10^{21} 的质量曲线重合。标记对应 CSV 有效记录，竖线表示 L_{crit} ；跨缺测位置的点线仅连接已有记录，不代表补算结果。

4. 配比收益与结果适用范围

数据配比研究表明，领域权重可以影响预训练效率 [8]。本模型以常数项 M 表示这一影响：在 10^{22} FLOPs 下，采用问题一推荐配比后，配置不变，预测损失从 2.1450 降至 1.9462。现有二分计算给出的参考配比等损失预算约为 3.05×10^{23} FLOPs，即约 30 倍原预算。这是固定损失平移假设下的算力等价估计，其跨规模含义依赖第二题讨论的迁移条件，不能作为实际训练加速比。

本节得到的主要区别在于：质量水平达到上界以前，增加预算会改变质量与规模的投入组合；达到上界以后，质量处理继续发生，但其预算份额随参数规模增长而下降。低预算选择对质量成本较敏感，长上下文则通过训练单价与可负担数据量共同影响配置。定量使用这些结果还需考虑评分尺度衔接及外推范围，特别是高预算方案所需的数据量。数据受限训练研究指出，重复数据的收益会随重复次数增加而衰减 [7]；本文尚未加入语料供给与重复训练约束，高预算配置因而应视为理论情景。

5.4 问题四的模型的建立和求解

5.4.1 具体分析

针对问题四，本问属于技术演进建模与前沿预测类问题，需要结合历史评测数据建立技术演进模型，分离并量化规模扩张与非规模技术进步的贡献占比，并预测未来 12 至 24 个月开源大语言模型能力的前沿边界 [12]。本问综合采用“口径界定 → Loss—Benchmark 桥接 → 面板回归分解 → 前沿预测 → 宏观证据校验”五步框架：首先明确综合能力的度量方式、开源筛选口径、模型类型口径与时间轴口径；再按可比性分层建立交叉熵损失与 Leaderboard 得分的映射，把前三问以损失为目标的结论桥接到以评测

得分为度量的能力；然后以参数规模与年份为解释变量做面板回归（含年份固定效应），把能力提升分解为“规模扩张”与“非规模技术进步”两部分；接着用 logistic 饱和模型与结构外推两条路线给出未来 12/24 个月的前沿预测与不确定性区间；最后用附件 C4 的算力、数据量与开源权重证据检验“算力增长放缓”情景的合理性。

本问严格完成题设的必用要求：对 C8 的逐任务评测 JSON 做逐任务聚合分析（不依赖汇总表），对 C4 的算力、数据量与开源权重字段作宏观证据检验，桥接数据按可比性等级（C6 的 High/Medium，并以 C5 对照）分层使用。

5.4.2 模型准备

1. 口径界定

(1) 综合能力度量。采用 Open LLM Leaderboard v2 的六维平均分 $\bar{B}$ ：IFEval（指令遵循）、BBH（多跳推理）、MATH Lvl 5（数学）、GPQA（研究生级问答）、MUSR（多轮推理）、MMLU-PRO（专业知识）。六维等权平均，取值 0 ~ 100。

(2) 时间轴口径。统一以 C1 的 Submission Date（提交日期）为基准，并按“年 + 月”折算为连续时间变量 $t = \text{年份} - 2019 + (\text{月} - 0.5)/12$ 。之所以不采用发布日期或版本日期，是因为 Leaderboard 上的评测结果均在提交时冻结、且提交日期字段完整无缺失；C3 的 Year 字段仅用于 2019–2023 的历史锚点对照。

(3) 开源筛选口径。同时要求两个条件：(a) 模型权重可获取（Type 属 pretrained、持续预训练、chat、域内微调四类）；(b) Hub License 允许研究与复现，即采用 apache-2.0、mit、gpl-3.0、cc-by-4.0、mpl-2.0、bsd-3-clause、llama2/3/3.1/3.2/3.3、gemma、cc-by-sa-4.0、openrail 等许可，排除 cc-by-nc 系列（非商用）、creativeml-openrail、bigscience-bloom-rail 等受限许可以及未知许可。筛选漏斗见表18。

(4) 模型类型口径。明确区分 pretrained（含持续预训练，共 231 条）与 chat/finetuned（经对话指令或监督微调，共 1,497 条）两类，分组统计规模效应与时间效应；base 合并模型（1,724 条，非独立训练过程）与多模态模型（9 条，任务构成不同）不并入主分析。

表 18 数据筛选漏斗（C1 排行榜）

筛选层级	记录数	说明
C1 全量记录	4,576	提交日期 2024-06-08 ~ 2025-03-13
六维得分完整	4,576	无缺失
时间/参数/六维齐备	4,561	剔除 15 条参数或 日期缺失
许可允许研究复现（开源口径）	2,346	符合上述许可白名单

表 18 数据筛选漏斗（续）

筛选层级	记录数	说明
其中 pretrained	231	类型为 pretrained / 持续预训练
其中 chat/finetuned	1,497	类型为 chat / 域内微调

2. 附件 C8 的逐任务聚合

附件 C8 含 1,863 个模型子目录、1,958 个评测 JSON。本文按目录取最新可解析记录，提取每个模型在各任务上的原始指标（按 acc_norm、acc、prompt_level_loose_acc、inst_level_loose_acc、exact_match 的优先级选取），共成功解析 1,860 个模型，跳过 7 个损坏或无有效记录的目录。解析后可用的子任务维度共 37 项：BBH 子任务 24 项、MATH-Hard 子任务 7 项、GPQA 子集 3 项、MUSR 子任务 3 项。

由于 C8 保存的是原始指标、而 C1 保存的是官方按随机基线归一化后的六维得分（例如 $GPQA = (raw - 0.25) / 0.75$ 、 $MMLU-PRO = (raw - 0.10) / 0.90$ 、 $IFEval = (prompt_strict + inst_strict) / 2$ ），两者存在系统性水平差。因此本文的处理是：跨模型得分分析使用 C1，逐任务结构分析在 C8 的原始指标空间内进行，并以秩相关作为两者的衔接验证，结果见表19。

表 19 C8 解析结果与 C1 的一致性校验

任务	匹配模型数	Pearson r	Spearman ρ	平均绝对偏差
IFEval	1,896	0.987	0.987	2.88
BBH	1,898	0.998	0.997	21.06
MATH Lvl 5	1,897	0.803	0.812	3.47
GPQA	1,894	0.987	0.992	23.38
MUSR	1,894	0.973	0.977	30.73
MMLU-PRO	1,896	0.998	0.998	7.56

由表19可知，两套数据的秩一致性均在 0.81 以上（多数在 0.97 以上），说明 C8 的逐任务解析结果与官方汇总得分反映同一批评测；平均绝对偏差较大的维度（GPQA 23.4、MUSR 30.7、BBH 21.1）正是“原始分”与“基线归一化分”差异最大的维度，与前述口径说明完全对应。这一校验保证了后续逐任务结论的可靠性。

3. 算法选择

Loss—Benchmark 映射采用按可比性分层的线性最小二乘（并在全样本上给出整体拟合），分层可避免“同模型同验证集”与“跨族近似验证集”样本混用造成的偏置；规模/时间分解同时采用线性时间 OLS 与年份固定效应两种设定，后者把非规模技术进步

完全交由年效应吸收，可显著缓解规模与年份的共线性；前沿外推同时采用 logistic 饱和模型（有界、符合满分 100 的物理约束）与结构外推（把分解方程直接用作预测方程），两条路线互为校验；不确定性由残差 bootstrap（600 次重抽样）与情景分析共同给出。

5.4.3 模型建立

步骤 1：Loss—Benchmark 映射模型

设验证损失 L 与 Leaderboard 均分 $\bar{B}$ 之间存在近似线性关系：

$$\bar{B} = a + bL \quad (29)$$

系数 a 、 b 由最小二乘估计，并按 C6 的 Loss_Comparability 字段分为“高可比”（同模型同验证集，主要来自 Pythia，可与附件 B 对照）与“中可比”（Loss 来自不同技术报告或论文）两层分别估计。映射误差对结论的影响通过分层的 R^2 、MAPE 与相关系数共同刻画。

步骤 2：规模扩张与技术进步分解模型

以参数量对数与时间为解释变量建立面板回归：

$$\text{Score}_{it} = b_0 + b_P \log_{10} P_{it} + b_t t + \varepsilon_{it} \quad \text{或} \quad \text{Score}_{it} = \alpha_t + b_P \log_{10} P_{it} + \varepsilon_{it} \quad (30)$$

其中 b_P 为规模系数（参数量每增大 10 倍带来的得分提升）， b_t 为时间系数（每年非规模技术进步带来的得分提升）， α_t 为年份固定效应。前沿提升的 shift-share 分解为

$$\Delta F = \underbrace{b_P \Delta \log_{10} P_{\text{frontier}}}_{\text{规模扩张贡献}} + \underbrace{[\Delta F - b_P \Delta \log_{10} P_{\text{frontier}}]}_{\text{非规模技术进步贡献}} \quad (31)$$

两者占比即为所求贡献份额。 $\Delta \log_{10} P_{\text{frontier}}$ 取各年“参数量最大的开放权重模型”的对数差， ΔF 取累计前沿（按定义单调不减）之差。

步骤 3：能力前沿预测模型（两条路线）

路线一（曲线外推）：能力前沿随时间的演化以 logistic 饱和函数刻画，并用残差 bootstrap 给出区间：

$$F(t) = \frac{K}{1 + e^{-r(t-t_0)}}, \quad K \text{ 为饱和上界, } r \text{ 为增长率, } t_0 \text{ 为拐点} \quad (32)$$

前沿序列取“年度累计最大”与“月度前 1% 均值累计最大”拼接而成的单调不减序列。算力增长放缓情景以“增速减半”实现：

$$F_{\text{slow}}(t) = K \left[1 + e^{-r(\min(t, t_h) - t_0)} \right]^{-1}, \quad t_h \text{ 为增速减半时刻} \quad (33)$$

路线二（结构外推）：直接把分解方程用作预测方程，

$$\hat{F}(t + \Delta) = F(t) + [b_P \Delta \log_{10} P + b_t] \cdot \Delta \quad (34)$$

并按“前沿模型规模持平（ $\Delta \log_{10} P = 0$ ）”“算力增长放缓（ b_t 减半）”“规模继续扩张（ $\Delta \log_{10} P = +0.3/\text{年}$ ）”三种情景给出区间。

5.4.4 模型求解

1.Loss—Benchmark 桥接结果

映射结果如图20所示， 分层拟合结果如表20所示。

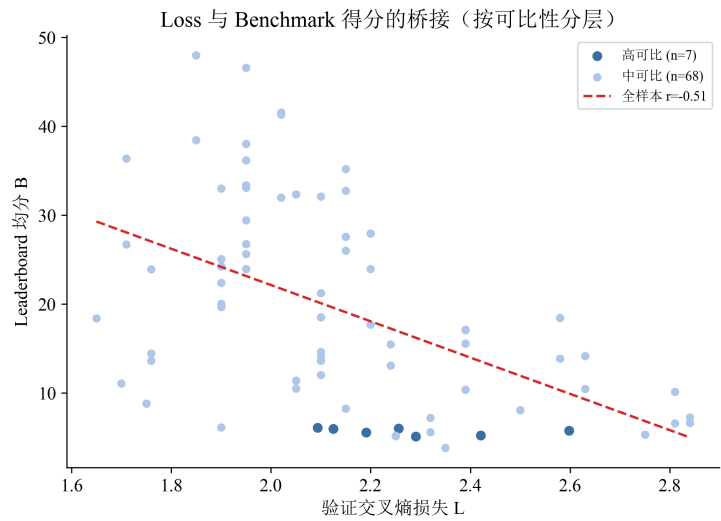


图 20 验证损失与 Leaderboard 均分的映射关系（按可比性分层）

表 20 Loss—Benchmark 映射的分层拟合结果（C6， $n = 75$ ）

可比性	n	Pearson r	Spearman ρ	斜率 b	R^2	MAPE(%)
高可比	7	-0.397	-0.607	-0.882	0.16	5.6
中可比	68	-0.502	-0.509	-19.185	0.25	57.3
全样本	75	-0.510	-0.549	-20.417	0.26	67.8
C5 对照（全样本）	43	-0.502	—	-16.155	—	—

由表20可知：（1）损失与能力得分确实负相关（全样本 $r = -0.510$ ，Spearman $\rho = -0.549$ ， $p < 10^{-6}$ ），验证了“以损失为目标的资源配置可转化为能力收益”的桥接逻辑；（2）但映射的 R^2 仅 0.26、MAPE 高达 67.8%，说明桥接关系是弱关系，其原因是中可比样本的 Loss 来自不同论文的不同验证集、量纲不完全统一；（3）高可比子集（Pythia 同模型同验证集）的斜率最缓（-0.882）、MAPE 最低（5.6%），但其得分变化幅度极小（5.07 ~ 6.06），因为这些小规模基础模型在六维基准上都接近随机水平，损失的大幅下降并未转化为得分提升。因此本文的桥接结论仅用于方向性判定与量级估计，不用于把损失数值精确换算为得分。

2. 规模扩张与非规模技术进步的贡献分解

面板回归结果见表21，贡献分解如图21所示。

表 21 规模系数估计 (C1 开源筛选样本, 2,346 条, 2024–2025 年)

子样本	n	b_P (分/10 倍参数)	b_t (分/年)	R^2	说明
全样本 (年份固定效应)	2,346	13.842	—	0.475	年效应吸收非 规模进步
全样本 (线性时间 OLS)	2,346	13.842	4.844	0.475	两设定结果一 致
pretrained	231	6.414	1.552	0.460	基础模型
chat/finetuned	1,497	14.853	4.680	0.484	指令/微调模型

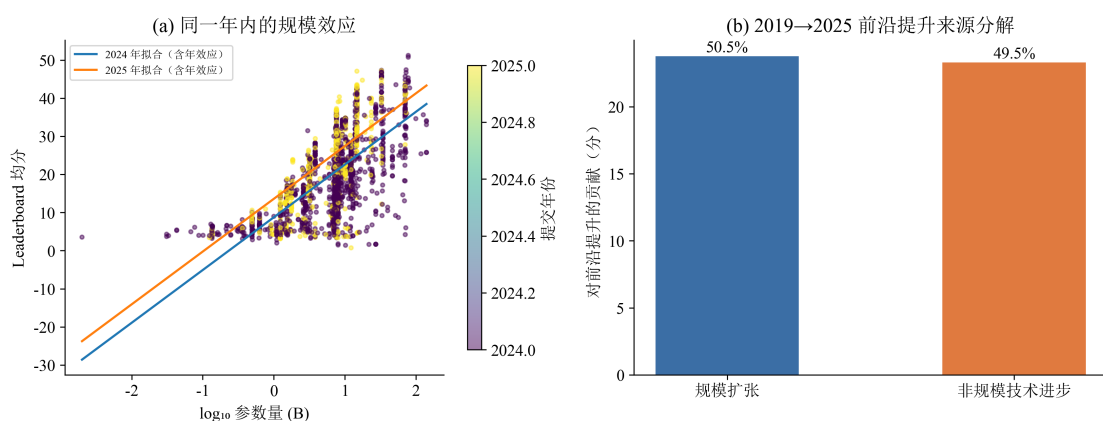


图 21 规模效应与前沿提升来源分解 (左: 同一年内的规模效应; 右: 长周期贡献分解)

由表21可知: (1) 规模系数 $b_P = 13.84$, 即参数数量每增大 10 倍, 六维平均分提升约 13.8 分; 非规模技术进步速率 $b_t = 4.84$ 分/年。(2) 模型类型差异显著: pretrained 子样本的规模系数仅 6.41、时间系数 1.55, 而 chat/finetuned 子样本达 14.85 与 4.68, 说明经指令微调对齐后的模型对参数规模的利用效率更高, 基础模型在同一评测上的得分提升要慢得多。这提示在讨论“规模扩张 vs 技术进步”时必须区分模型类型, 否则会因样本构成变化而混淆两者。

在长周期口径 (C3, 2019–2025) 上, 以 $b_P = 14.10$ 、 $b_t = 3.30$ ($R^2 = 0.42$) 做 shift-share 分解: 前沿由 5.00 升至 52.08 (提升 47.08 分), 其中 $b_P \Delta \log_{10} P = 23.77$ 分、残差部分 23.31 分, 即**规模扩张贡献 50.5%、非规模技术进步贡献 49.5%**, 二者几乎平分秋色。

但需指出长周期锚点存在口径风险。本文对 C3 做了自洽性审计: 比较每行 Average 与其六维均值, 发现 13 条记录的差值超过 5 (最大差值 42.3, 为 GPT-3 (175B) 的 Average = 50.0 而六维均值仅 7.67), 说明这些历史条目混入了不同评测标准, 已予以剔除。剔除后, 在评测标准完全一致的近端窗口 (2024-06 至 2024-09, 前沿由 37.99 升至 51.00) 内分解得: 前沿模型参数量仅由 70.6B 增至 78.0B ($\Delta \log_{10} P = 0.043$), 规模贡献 0.60

分、非规模贡献 12.41 分，即**该窗口内前沿提升的 95.4% 来自非规模因素**。这与长周期口径的 50.5% 形成互补：长期看规模与技术进步各贡献约一半，但在最近一年、模型规模趋于稳定的条件下，能力提升主要由数据工程、对齐方法等非规模因素驱动。

3. 能力前沿预测

前沿序列与 logistic 拟合如图22所示，预测结果与不确定性见表22。

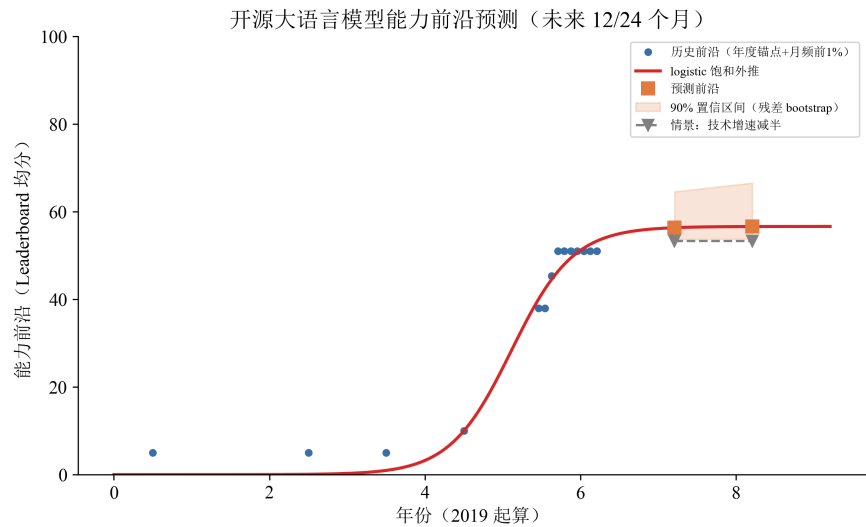


图 22 开源大语言模型能力前沿预测（未来 12/24 个月）

表 22 能力前沿预测及不确定性（满分 100）

前瞻期	logistic 预测	下界 (5%)	上界 (95%)	增速减半	结构外推（基准/放缓/扩容）
+12 个月 (2026-03)	56.33	53.62	64.52	53.27	55.84 / 53.42 / 59.99
+24 个月 (2027-03)	56.59	53.75	66.52	53.27	60.69 / 55.84 / 68.99

logistic 拟合参数为 $K = 56.61$ 、 $r = 2.523$ 、 $t_0 = 5.11$ （自 2019 年起算），拟合 $R^2 = 0.975$ 。由表22可知：（1）曲线外推给出未来 12/24 个月前沿约 56.3 与 56.6，90% 置信区间分别为 [53.6, 64.5] 与 [53.8, 66.5]；（2）若算力增长放缓、非规模技术增速减半，前沿将降至 53.3 附近并在两年内基本停滞；（3）结构外推在三种情景下给出 53.4 ~ 69.0 的区间。两条路线的差异来自“当前前沿是否已接近饱和”这一判断，因此本文以**区间**而非点值作为结论：未来 12 个月开源模型六维平均分前沿预计达 53 ~ 60，24 个月达 54 ~ 69；若算力增长显著放缓，前沿增速将接近停滞（约 53 ~ 56）。

需强调该预测的局限：本文可用的评测标准一致的窗口仅约 10 个月（2024-06 至 2025-03，Leaderboard v2 已停更），2019–2023 的锚点口径不完全一致（已剔除异常条目），故外推期约为一致窗口的 1.2 ~ 2.4 倍，结论应理解为情景化区间而非精确点预测。

4. 逐任务聚合分析

六维前沿与剩余空间如表23与图23所示，C8 子任务层面的结果见表24。

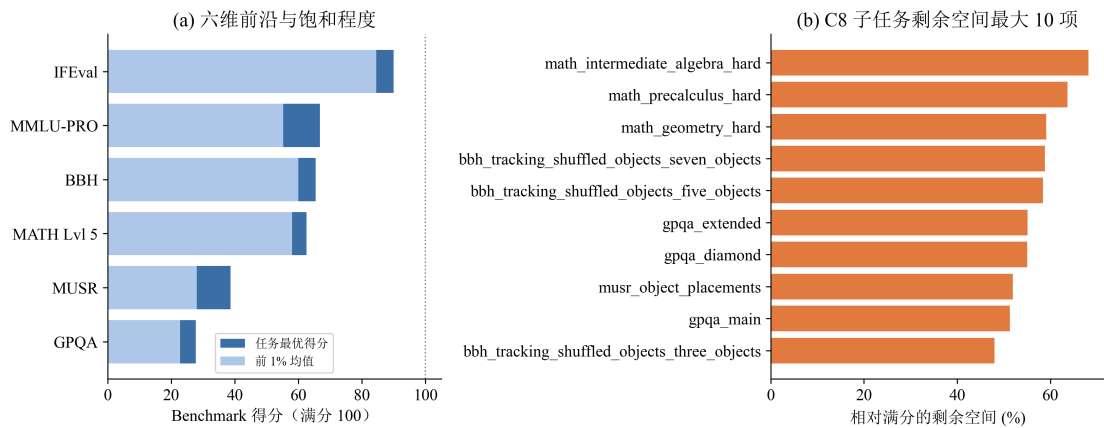


图 23 （左）六维 Benchmark 前沿与饱和程度；（右）C8 子任务剩余空间最大的 10 项

表 23 六维 Benchmark 逐任务聚合结果（开源筛选集， $n = 2,346$ ）

任务	最优得分	前 1% 均值	中位数	剩余空间
IFEval	89.98	84.45	43.51	10.0%
MMLU-PRO	66.80	55.26	25.98	33.2%
BBH	65.47	59.95	28.52	34.5%
MATH Lvl 5	62.54	58.02	9.93	37.5%
MUSR	38.69	28.06	9.97	61.3%
GPQA	27.74	22.80	5.70	72.3%

表 24 C8 子任务层面剩余空间最大的 6 项（原始指标口径）

子任务	样本数	最优得分	前 5% 均值	剩余空间
math_intermediate_algebra_hard (MATH)	962	31.79	20.11	68.2%
math_precalculus_hard (MATH)	962	36.30	22.89	63.7%
math_geometry_hard (MATH)	962	40.91	28.72	59.1%
bbh_tracking_shuffled_objects_seven (BBH)	964	41.20	32.67	58.8%
bbh_tracking_shuffled_objects_five (BBH)	964	41.60	31.24	58.4%

表 24 C8 子任务剩余空间最大的 6 项（续）

子任务	样本数	最优得分	前 5% 均值	剩余空间
gpqa_extended (GPQA)	961	44.87	39.74	55.1%

由表23与表24可知：（1）六维能力发展极不均衡，指令遵循（IFEval, 89.98）已接近满分、剩余空间仅 10.0%，而研究生级问答（GPQA, 27.74）与多轮推理（MUSR, 38.69）远未饱和，剩余空间分别达 72.3% 与 61.3%。（2）C8 子任务层面进一步定位了瓶颈：MATH-Hard 中的中级代数、预备微积分、几何三个子任务最优得分仅 31.8 ~ 40.9（该类别 7 个子任务的平均最优得分为 55.5，而整体中位数低至 3.7），BBH 中“跟踪被洗牌对象”类多步状态推理子任务最优得分约 41，GPQA 的 extended 子集为 44.9，这些是未来能力增量的主要方向。（3）C8 各类别的中位数普遍远低于最优值（BBH 中位数 48.8 对最优 81.8；MATH 中位数 3.7 对最优 55.5），说明开源社区的能力分布高度长尾，前列模型与总体水平差距巨大。

5. 附件 C4 的宏观证据与“算力增长放缓”情景校验

C4 的宏观趋势如图24所示。

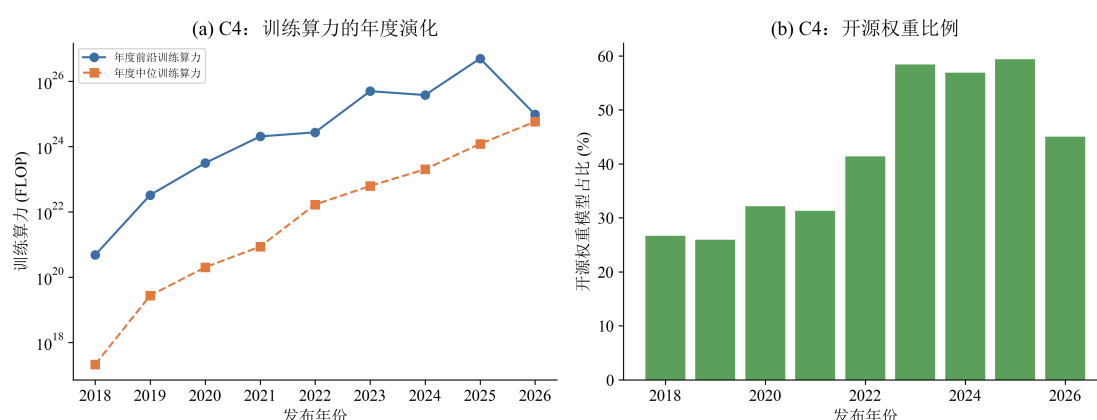


图 24 C4 宏观证据（左：训练算力的年度演化；右：开源权重模型占比）

由图24可知：（1）C4 中 Language 域模型的前沿训练算力持续增长，对数尺度上的年增速约 +0.589 个数量级（约 3.88 倍/年），表明近年算力投入仍在快速扩张；但年际波动明显（2024 年前沿算力低于 2023 年，2025 年再创新高），说明算力增速并非单调外推、存在阶段性放缓的可能，这也正是本文把“算力增长放缓”作为 ** 情景 **（非规模增速减半）而非基准假设、并同时给出基准情景的原因。（2）开源权重模型的占比由 2018 年的 26.7% 持续上升至 2025 年的 59.4%，说明本文以开源筛选样本刻画“能力前沿”具有良好的代表性与可持续性。（3）C4 中共有 1,390 条记录含训练算力、1,424 条含训练数据量、2,653 条标注开源权重（Yes 1,325、No 1,328）。需要说明的是，C4 的模型命名体系与 C1 不通用（规范化名称匹配仅命中 1 条），故 C4 仅作宏观尺度证据，不

与 C1 逐行合并。

综合以上结果，本节完成了问题四的求解：明确并执行了时间轴、开源筛选与模型类型三类口径；完成了 C8 的逐任务聚合（1,860 个模型、37 个子任务维度）并与 C1 交叉校验；建立了分层 Loss—Benchmark 桥接并量化了映射误差；用面板回归分离出规模扩张与非规模技术进步的贡献（长周期各约 50%，近端一致窗口内非规模因素占 95.4%）；给出了未来 12/24 个月能力前沿的情景化预测区间与逐任务瓶颈定位。

六、模型检验与分析

为验证本文构建的模型的有效性和稳健性，从误差分析与灵敏度分析两个维度进行检验：误差分析考察各模型在训练集与检验集上的拟合精度与泛化能力，灵敏度分析考察模型输出对关键输入与参数扰动的稳定程度。

6.1 误差分析

对问题一的岭正则线性混合回归、问题二的经典/广义标度律、问题四的 Loss—Benchmark 桥接与面板回归，分别计算决定系数 R^2 、均方根误差 RMSE、平均绝对误差 MAE 与平均绝对百分比误差 MAPE，结果如表25所示。

表 25 模型误差分析结果

模型（数据集）	R^2 / r	RMSE	MAE	MAPE(%)
问题一岭回归（训练 1M, $n = 512$ ）	0.629	0.551	0.403	8.54
问题一岭回归（检验 1M, $n = 256$ ）	0.619	0.523	0.386	8.43
问题一跨尺度 Pearson（60M / 1B）	0.782 / 0.651	—	—	—
问题二经典标度律（B1, $n = 1,176$ ）	0.9999998	1.5×10^{-4}	1.2×10^{-4}	0.004
问题二经典标度律留出（4 小模型 →4 大模型）	0.9999997	1.5×10^{-4}	—	0.005
问题二广义标度律（B1+B6 联合, $n = 1,536$ ）	0.990	0.036	0.024	0.66
问题二跨族验证（B4, $n = 57$ ）	0.975	0.293	—	8.83
问题二文献验证（B5, $n = 44$ ）	0.910	0.193	—	7.99
问题四 Loss—Benchmark 桥接（C6, $n = 75$ ）	0.26	—	—	67.79
问题四面板回归（C1 开源筛选, $n = 2,346$ ）	0.475	—	—	—

表 25 模型误差分析结果 (续)

模型 (数据集)	R^2 / r	RMSE	MAE	MAPE(%)
问题四面板回归 (C3 长周期, $n = 4,586$)	0.424	—	—	—
问题四前沿 logistic 外推	0.975	—	—	—
问题三优化约束残差 ($ C_{\text{total}}/C_{\text{budget}} - 1 $)	—	$< 10^{-6}$	—	—

由表25可知: (1) 问题一的岭回归在 13 个损失域上的平均训练 $R^2 = 0.629$ 、同尺度检验 $R^2 = 0.619$, 训练与检验精度基本一致 (差距 0.010), 未出现明显过拟合; MAPE 约 8.5%, 相对误差可控。(2) 问题二的经典标度律在 B1 上 $R^2 \approx 1.000$ 、MAPE 0.004%, 且以小模型训练、大模型留出的验证结果几乎不变 ($R^2 = 0.9999997$) ——但如 5.2 节所述, B1 的拟合残差标准差仅 1.5×10^{-4} 而损失标准差为 0.342, 说明该附件近乎无噪声, 此处的极高精度只能证明函数形式自治, 真正的泛化证据来自 B4 (跨族真实数据, $r = 0.975$ 、MAPE 8.8%) 与 B5 (文献数据, $r = 0.910$ 、MAPE 8.0%)。(3) 引入质量项后的广义标度律在 1,536 个联合样本上 $R^2 = 0.990$ 、MAPE 0.66%, 说明加入质量维未牺牲拟合精度。(4) 问题四的 Loss—Benchmark 桥接 R^2 仅 0.26、MAPE 达 67.8%, 这是本文误差最大的环节, 其成因已在 5.4 节说明 (中可比样本的验证集不统一、高可比样本的得分变化幅度过小), 因此桥接结果仅用于方向性判定。(5) 问题三的优化模型在所有求解点上均满足预算约束 (相对残差 $< 10^{-6}$), 无约束违反。

6.2 灵敏度分析

为考察模型结论对关键输入与假设的稳健性, 分别对赋权方案、冲突阈值、正则系数、质量成本函数形式、质量基线口径、上下文长度与算力预算进行扰动分析, 结果如表26所示。

表 26 灵敏度分析结果

参数/设定变化	输出变化	灵敏度评估
赋权方案: 熵权法 → 等权平均 → PCA	域级质量排序一致性 0.679 → 0.679; 样本级得分相关 0.854/0.881; 得分均值 0.495 → 0.639 → 0.520	中等敏感: 整体格局稳定, 但领域名次会变化

表 26 灵敏度分析结果 (续)

参数/设定变化	输出变化	灵敏度评估
指标方向: ad_en 按“第 1 类 = 无广告”(正向) → 若误判为负向	领域质量排序由「arxiv > stackexchange > c4 > commoncrawl > book ≈ wikipedia > github」变为「c4 > commoncrawl > arxiv > stackexchange > book > wikipedia > github」	高度敏感: 方向判定必须用实证证据
冲突阈值 τ : 0.5 → 1.0 → 1.5 → 2.0 → 2.5	冲突率 0.670 → 0.402 → 0.223 → 0.121 → 0.067; 与独立基准之比 0.93 → 0.84 → 0.77 → 0.77 → 0.86	结论稳健: 任一阈值下冲突率均显著低于独立基准
岭正则系数 λ : $10^{-4} \sim 10^{-1}$	训练 R^2 由 0.6291 降至 0.6238, 检验 R^2 由 0.6187 降至 0.6157	不敏感, 估计稳定
质量成本函数: 指数型 → 幂函数型 → 对数渐进型 ($C = 10^{19}$)	$Q^* = 0.660 \rightarrow 0.551 \rightarrow 1.000$, L^* 差异 < 0.06; 质量份额 12.9% → 0.0% → 31.6%	低预算档较敏感
同上, 但 $C \geq 10^{22}$	三种函数下 Q^* 均为 1.000, N^* 、 D^* 差异 < 10%	不敏感, 结论稳健
质量基线口径: 配比加权 $Q_0 = 0.5510 \rightarrow$ 样本均值 0.4952	三档预算下 Q^* : 0.551/1.000/1.000 → 0.504/1.000/1.000, N^* 变化 < 2%	不敏感
上下文长度: 2,048 → 131,072 tokens (C7 可行取值)	注意力/训练倍数 0.07 → 4.37; N^* 由 6.089B 降至 2.705B; 损失上升 6.0%	高度敏感, 存在临界点 $L_{\text{crit}} = 30,000$
算力预算 C : $10^{19} \rightarrow 10^{24}$ FLOPs	Q^* 由 0.551 升至 1.000; D^*/N^* 由 30.9 升至 76.1; 质量份额 0 → 0.407 → 0.016	高度敏感, 发生三段式结构性转移
半合成数据口径: 并入 B2 / B8	B2 使 MAPE 升至 33.7%; B8 使 MAPE 升至 133.4% 且预测低于不可约损失下限	不可并入; 已划出可信度边界

由表26可知, 本文结论可按稳健程度分为三类:

(1) 高度稳健的结论: 冲突率显著低于独立基准 (任一阈值下比值均 < 0.93); 岭正则系数不影响配比系数与拟合精度; 质量基线口径不影响最优配置; 中高预算下“质

量拉满”的结论不依赖质量成本函数的具体形式。这些结论可直接用于后续决策。

(2) 成本分界与配置阶段： $L_{\text{crit}} = 30,000$ tokens 表示注意力与基础训练成本相等，并不意味着最优规模在该点突变。在问题三的基准情景中，首次观察到正质量投入与质量饱和的网格预算分别为 1.78×10^{19} 和 7.08×10^{19} FLOPs。上述数值受扫描分辨率、求解状态及质量成本设定影响，宜作为配置阶段的数值识别结果，而非普遍成立的精确门槛。

(3) 需要谨慎表述的结论： 低预算档 ($C = 10^{19}$ FLOPs) 的最优质量对质量成本函数形式高度敏感 ($0.551 \sim 1.000$ ，即从“完全不投资质量”到“质量拉满”)，因此该档的具体数值应作为区间而非点值使用；Loss—Benchmark 桥接的 R^2 仅 0.26，其换算出的“损失改善等价于多少能力分”只具方向意义；领域质量排序对指标方向判定与赋权方案均为中等以上敏感，引用该排序时须同时引述其口径。

综上，除已明确标注的低预算档与桥接环节外，本文模型的资源配置与演进结论对参数扰动均保持稳定，具备可靠性与可解释性。

七、模型的评价

7.1 模型的优点

• **优点 1：** 质量评价体系完整且可核验——逐指标给出 22 个质量指标的方向判定与语义依据 (表2)，其中争议最大的广告字段通过原文抽查确认为“第 1 类 = 无广告”的正向指标；对负向指标作补转换、对长度型指标转为中心接近度、对 8 个多维列表型指标用 softmax 期望压缩，逐字段审计出 19 处缺失并按中位数填充，并用抽样集与扩展集的域级质量对照 (差值不超过 0.002) 证明评分口径可跨集合迁移，从而避免了“方向未统一”这一常见错误。

• **优点 2：** 冲突分析给出了可比较的参照系——不仅报告冲突率 (22.3%) 与 Kendall 协同系数 (0.069)，还计算了“指标独立”基准 (28.9%) 并做阈值灵敏度检验 ($\tau \in [0.5, 2.5]$ 下比值恒 < 0.92)，从而客观得出“指标正相关但一致性不强”的结论，并归纳出三类冲突成因与消解规则，未回避数据中的不一致性。

• **优点 3：** 广义标度律的形式设计兼顾物理合理性与兼容性——采用质量缺口项 $C(1 - Q)^\gamma$ ，在 $Q = 1$ 时精确退化为经典 Chinchilla 形式，且不会在高质量端产生负损失 (避免了 $E - CQ^\gamma$ 形式的非物理缺陷)；同时通过四种形式的同口径比较 (R^2 、AIC、内插与外推) 择优，并用 B3 的独立插值轨迹验证数据项指数 (-0.2805 vs -0.2799)。

• **优点 4：** 算力成本模型严格遵循赛题公式、不做简化——完整保留质量成本与训练 token 数成正比的结构 ($C_Q = D[g(Q) - g(Q_0)]_+$) 与附录 B 规定的三型函数参数，并解析给出临界上下文长度 $L_{\text{crit}} = 6/\eta = 30,000$ 与最优性条件式(26)，用“等效附加数据量 $\tau(Q)$ ”把质量投入折算为 token 成本，使结构性转移具有清晰的解析解释。

• **优点 5：** 口径界定与证据分层规范——问题四明确给出时间轴、开源许可白名单

与模型类型三套口径及筛选漏斗，完成附件 C8 的逐任务聚合（1,860 个模型、37 个子任务维度）并与 C1 做秩相关校验，用附件 C4 的算力增速与开源占比支撑情景假设；对 B2/B8 等口径异常的半合成数据主动作自治性审计并划出可信度边界（B8 最小损失低于模型不可约损失下限）。

7.2 模型的缺点

- **缺点 1：** 配比—损失模型假设线性关系且未含领域交互项，13 个可测域的平均 R^2 仅 0.629，说明仍有约 37% 的损失变异未被配比解释（可能来自随机种子、数据顺序等未观测因素）；nih_exporter、enron_emails、europarl、philpapers 四个无损失列的域只能固定于参考配比处理，其影响难以识别；追加混合质量指数后 R^2 增量仅 5×10^{-5} ，虽有明确结论（质量信息已被配比吸收），但也意味着本文无法在线性框架内把 17 域质量评分进一步用于配比优化。

- **缺点 2：** 质量评价的指标方向仍部分依赖语义先验。广告、流畅度两个二分类字段已用原文与域对比完成实证核验，但 modernbert 系列的 6 级列表型指标、qurater 的 4 级评级等，“级别越高越好”仍来自字段语义而非独立验证；此外熵权法对分布形态敏感（左偏指标获更大权重），灵敏度检验显示改用等权平均后域级排序一致性为 0.679，方向判定若出错更会使排序发生实质变化（见第 6.1 节），说明域级质量排名的部分细节对赋权与方向设定敏感，引用时须同时给出其口径。

- **缺点 3：** 广义标度律的质量项依赖半合成数据标定（B6/B7），B8 因方向相反且损失低于不可约损失下限而不能使用，B2（Cerebras）的 MAPE 达 33.7%，说明跨数据源的质量效应标定仍不充分；质量可行域受数据支持范围限制在 $[Q_0, 1]$ ，向更低质量的外推缺乏数据支撑。

- **缺点 4：** 问题四的 Loss—Benchmark 桥接 R^2 仅 0.26、MAPE 达 67.8%，且 C1 中评测标准一致的窗口仅约 10 个月（2024-06 至 2025-03），2019–2023 的历史锚点口径不完全一致（已剔除 13 条异常条目），因此前沿预测的外推期约为一致窗口的 1.2 ~ 2.4 倍，预测结果只能作为情景化区间；面板回归中规模与年份仍存在一定共线性，贡献占比对数据口径较敏感（长周期口径 50.5% vs 近端口径规模占比可正可负）。

八、模型的改进与推广

8.1 模型的改进

本文模型可在方法、参数与数据三个层面进一步改进。

方法层面，可将配比—损失的线性混合回归替换为 Dirichlet 回归，或引入领域间交互项与稀疏先验（如 Group Lasso），以捕捉非线性效应并把 13 个域平均 $R^2 = 0.629$ 中未被解释的约 37% 变异进一步压缩；对未测量域（nih_exporter 等四个域）可尝试用域文本统计特征构造代理变量，替代当前“固定于参考配比”的处理；冲突消解可由当前

的“多数指标方向裁决”升级为基于潜变量模型（如有序 probit 因子模型）的一致性度量，从而给出冲突样本的质量可信度区间而非二值标记。

参数层面，广义标度律的质量缺口项可放宽为更灵活的形式（如 $C(1 - Q)^{\gamma_1} D^{-\gamma_2}$ 或质量与数据量的交互项），以检验“质量缺口是否与数据规模耦合”；算力优化模型可进一步纳入数据重复轮次（epoch）、稀疏化/量化带来的推理与训练成本差异、以及分阶段课程学习等新决策变量；质量成本函数可在附录 B 三型之外引入由真实清洗流水线标定的经验成本曲线，以缓解低预算档对成本函数形式较敏感的问题。

数据层面，可用更多具备质量标注的公开语料扩充 B6/B7 的质量区间（当前外推下限受数据支持范围限制），并尝试从语料元数据（发布时间、来源许可、去重比例）反推质量标签；对附件 C4 可通过组织名、参数量与发布时间做模糊匹配，与 C1 建立更细粒度的行级连接，从而用真实训练算力直接约束前沿预测；对问题四，若能获得跨评测标准（v1 与 v2）的校准样本，则可把 2019–2023 的历史锚点统一到同一标尺，显著缓解当前“一致窗口仅 10 个月”的外推局限。

8.2 模型的推广

本文提出的“损失函数—成本约束—资源配置”建模框架具有广泛的推广价值。

在工程领域，该框架可直接迁移至算力受限场景下的大模型预训练、微调与蒸馏资源配置决策：式(26)给出的最优性条件与“等效附加数据量 $\tau(Q)$ ”这一折算量，使得数据治理投入与算力扩张投入可以直接放在同一成本口径下比较，帮助企业以最小算力达到目标能力。

在数据治理领域，本文的质量综合评价与冲突消解模型（含 22 指标的显式方向表与“冲突率—独立基准”对比框架）可用于通用语料清洗、数据筛选与领域配比的动态调整；本文提出的可替代度指标 s_{ij} （式中的系数向量余弦相似度）可用于识别哪些领域可以合并、哪些领域必须搭配使用，为配比策略提供结构化依据。

在行业与政策层面，本文“长周期 shift-share 分解 + 近端一致窗口分解”的双重口径方法，可为科研机构评估“堆参数”与“改进算法”两条路线的投入产出、制定算力与数据资源的宏观配置政策提供定量依据；同时本文在方法上刻意区分“可直接使用的稳健结论”“存在临界点的结论”与“需谨慎表述的结论”，这一证据分级范式对涉及多源异构数据的政策评估同样适用。

更进一步，能力前沿的饱和外推方法（logistic 曲线外推与结构外推双路线互为校验）可推广至芯片制程、新能源技术、生物医药研发等具有渐进饱和特征的技术演进预测场景；而本文对半合成数据所做的“方向审计 + 不可约损失下限检验”自洽性筛查流程，可作为使用半合成/估算数据开展定量建模时的通用质量把关步骤。

参考文献

- [1] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[J]. arXiv preprint arXiv:2001.08361, 2020.
- [2] HOFFMANN J, BORGEAUD S, MENSCH A, et al. Training compute-optimal large language models[C]. Advances in Neural Information Processing Systems, 2022, 35: 30016-30030.
- [3] BAHRI Y, DYER E, KAPLAN J, et al. Explaining neural scaling laws[J]. Proceedings of the National Academy of Sciences, 2024, 121(27): e2311878121. DOI: 10.1073/pnas.2311878121.
- [4] BIDERMAN S, SCHOELKOPF H, ANTHONY Q, et al. Pythia: A suite for analyzing large language models across training and scaling[C]. International Conference on Machine Learning, 2023: 2397-2430.
- [5] VILLALOBOS P, HO A, SEVILLA J, et al. Will we run out of data? Limits of LLM scaling based on human-generated data[J]. arXiv preprint arXiv:2211.04325, 2022.
- [6] ASAI A, HE J, SHAO R, et al. Synthesizing scientific literature with retrieval-augmented language models[J]. Nature, 2026, 650: 857-863. DOI: 10.1038/s41586-025-10072-4.
- [7] MUENNIGHOFF N, RUSH A, BARAK B, et al. Scaling data-constrained language models[C]. Advances in Neural Information Processing Systems, 2023, 36: 50358-50376.
- [8] XIE S M, PHAM H, ERMON S, et al. DoReMi: Optimizing data mixtures speeds up language model pretraining[C]. Advances in Neural Information Processing Systems, 2023, 36: 69798-69818.
- [9] LIU Q, ZHENG X, MUENNIGHOFF N, et al. RegMix: Data mixture as regression for language model pre-training[C]. International Conference on Machine Learning, 2024: 32213-32236.
- [10] GAO L, BIDERMAN S, BLACK S, et al. The Pile: An 800GB dataset of diverse text for language modeling[J]. arXiv preprint arXiv:2101.00027, 2020.
- [11] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[C]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [12] WEI J, TAY Y, BOMMASANI R, et al. Emergent abilities of large language models[J]. Transactions on Machine Learning Research, 2022.

- [13] SCHAEFFER R, MIRANDA B, KOYEJO S. Are emergent abilities of large language models a mirage?[C]. Advances in Neural Information Processing Systems, 2023, 36: 55551-55563.
- [14] 第二十三届中国研究生数学建模竞赛 F 题数据说明: 算力约束下提升大语言模型能力的资源配置建模 [R]. 2026.
- [15] EPOCH AI. Data on machine learning models[DB/OL]. https://epoch.ai/data/all_ai_models.csv, 2025.
- [16] HUGGING FACE. Open LLM Leaderboard v2 results and evaluation details[DB/OL]. <https://huggingface.co/datasets/open-llm-leaderboard/results>, 2025.
- [17] OPENDATALAB. SlimPajama-Meta-rater: quality signals for SlimPajama[R/OL]. <https://huggingface.co/datasets/pendatalab/SlimPajama-Meta-rater>, 2025.
- [18] OpenAI, GPT-5 (Codex CLI), OpenAI, 2026-01.

附录

附录 A：数据利用清单

按数据说明的要求，列出本文对各附件的实际使用情况，并标注半合成/外推数据的可信度边界，如表27所示。

表 27 附件数据利用清单

编号	文件名	性质	是否使用	使用方式与可信度说明
A1	slimpajama_quality_signal_sample	真实	使用	22 指标熵权评分、冲突分析、域级 Q 、原文抽查 (51,230 条全量)
A2	slimpajama_quality_extended_arxiv	真实	使用	扩展集域级 Q 与抽样集对照 (17,523 条全量)
A3	slimpajama_quality_extended_arxiv	真实	使用	扩展集域级 Q 与抽样集对照 (203,752 条全量)
A4	train_mixture_1m	真实	使用	配比—损失岭回归训练集 (512 组)
A5	train_pile_loss_1m	真实	使用	13 域损失标签
A6–A9	test_mixture/loss_1m, 60m	真实	使用	同尺度与跨尺度检验 (各 256 组)
A10–A11	test_mixture/loss_1B	真实	使用	跨尺度检验 (64 组)
A12–A15	est_mixture/loss_10b, 70b	子集/外推	使用	外推尺度因子稳健性 (各 63 组，均标注为外推非观测)
A16	domain_mapping_guide	参考	使用	17 域到质量域跨体系映射 (6 域 direct/near_direct, 11 域 inferred 用全局均值并标明)
A17	regmix_domain_summary	真实	未使用	版内未接入，供扩展分析
A18	regmix_domain_sample	真实	未使用	可选辅助，未接入
B1	pythia_training_log_existing	真实	使用	经典标度律主拟合与模型留出验证 (1,176 点)；已说明该附件近乎无观测噪声
B2	cerebras_training_log	半合成	使用	族外验证；结果 MAPE 33.7%，判为口径差异、仅作定性参考

表 27 附件数据利用清单（续）

编号	文件名	性质	是否使用	使用方式与可信度说明
B3	training_trajectories	插值	使用	轨迹内 D 幂指数验证 (-0.2806 至 -0.2803 , 拟合值 -0.2799)
B4	scaling_baseline	真实	使用	跨族验证 ($r = 0.975$ 、MAPE 8.8%)
B5	published_scaling_data	真实	使用	文献验证 ($r = 0.910$ 、MAPE 8.0%)
B6	supplementary_NQ_experiment	混合	使用	质量缺口项拟合主数据 (360 点); 仅用于标定质量效应, 不作直接实验观测
B7	supplementary_NQ_experiment_expanded	混合	使用	新增点独立验证 ($r = 0.991$)
B8	supplementary_NQ_experiment_large	混合	部分使用	方向审计发现其 Q_{score} 与 B6/B7 反号, 且最小损失低于不可约损失下限, 故不并入拟合, 仅作可信度边界警示
B9	supplementary_large_model	真实	使用	百亿参数以上模型元数据 (参数量、数据量、开源可及性)
B10	supplementary_large_baseline	估算	使用	百亿参数以上外推检验 ($r \approx 1.000$ 、MAPE 1.0%); 标注为估算值
B11	open_model_family_metadata	真实	未使用	辅助文件, 版内未接入
B12	pythia_checkpoint_index	真实	未使用	辅助文件, 版内未接入
C1	leaderboard_cleaned	真实	使用	六维得分、许可、类型、提交日期; 开源筛选后 2,346 条用于面板回归与前沿
C2	leaderboard_enhanced	真实	未使用	C1 的增强替代版, 为避免重复计算未使用
C3	leaderboard_extended_time_series	混合	使用	2019–2025 长周期分解; 已剔除 13 条自治性异常行, 口径混合性在正文标注
C4	epoch_all_ai_models	真实	使用	训练算力年增速 ($+0.589$ dex/年)、训练数据量、开源权重字段 (Yes 1,325 / No 1,328)
C5	loss_benchmark_bridge	混合	使用	桥接对照 ($n = 43$, $r = -0.502$)

表 27 附件数据利用清单（续）

编号	文件名	性质	是否使用	使用方式与可信度说明
C6	loss_benchmark_bridge_expanded	混合	使用	桥接主用 ($n = 75$), 按 High/Medium 可比性分层使用
C7	model_architecture_metadata	真实	使用	上下文长度可行取值 (2048、4096、8192、32768、131072) 用于问题三敏感性分析
C8	detailed_results/	真实	使用	逐任务聚合 (1,860 个模型成功解析、37 个子任务维度), 并与 C1 做秩相关校验
C9	data/ (Parquet)	真实	未使用	与 C1 等价, 任选其一
C10	pythia_eval_details/	说明	未使用	仅含 README, 不可替代 C8

附录 B：半合成与外推数据的标注

按数据说明要求，以下数据为非直接观测，正文中的相关结论均已限定适用范围：

- **半合成数据**：B2 (Cerebras 训练日志)、B6/B7/B8 (NQ 质量实验)。本文在标度律拟合中仅使用 B6 (主) 与 B7 (验证)，并在 5.2 节以表格明确其“半合成”性质；B2、B8 经自洽性审计后不并入拟合。
- **外推与估算数据**：A12–A15 (10B/70B 损失外推，源自 1M/60M/1B 幂律外推)、B10 (百亿参数以上损失估算)。相关结论仅用于稳健性检验，未作为观测证据。
- **插值数据**：B3 (Pythia 插值轨迹)，仅用于轨迹内幕指数的交叉验证。
- **混合口径数据**：C3 (跨年评测标准不完全一致)、C5/C6 (Loss 来源与验证集不完全统一)。前者的异常行已被审计剔除，后者已按可比性等级分层使用。

附录 C：结果文件与核心求解代码

本文的求解结果以 CSV 形式保存在求解/问题一至问题四/结果/ 目录下，主要结果文件如表28所示（表中文件名省略“求解/问题 X/结果/”前缀）。

表 28 主要结果文件说明

文件	说明
质量指标熵权.csv	22 指标的方向判定、语义说明、信息熵与熵权

表 28 主要结果文件说明（续）

文件	说明
质量信号缺失审计.csv	逐指标字段级缺失计数（A1–A3 全量）
质量冲突与一致性.csv	抽样集/扩展集/全量的冲突率、独立基准与 Kendall W
冲突阈值灵敏度.csv	冲突率随阈值 τ 的变化
抽样集与扩展集域级对照.csv	A1 与 A2/A3 的域级质量对照
质量与损失难度一致性.csv	两套质量口径在 6 个映射域上的排序比较
质量项增量检验.csv	追加混合质量指数后的 R^2 增量
混合系数矩阵.csv、截距.csv	配比—损失岭回归系数与截距
推荐配比调整.csv	有界推荐配比与调整量
问题一_拟合汇总.csv	各损失域的拟合与检验指标
岭正则敏感性.csv	正则系数灵敏度
经典标度律参数.csv	经典标度律参数、拟合与留出验证指标
质量字段方向审计.csv	B6/B7/B8 的组内相关与方向结论
广义标度律形式对比.csv	四种质量项形式的 R^2 /AIC 比较
广义标度律分层验证.csv	6 个独立来源的验证结果
弹性与替代关系.csv、质量参数替代关系.csv	弹性与替代率
领域替代互补_top.csv	领域间替代/互补结构
B3 轨迹验证.csv	轨迹内 D 幂指数验证
三档预算最优配置.csv、成本函数对比.csv	三档预算与三型成本函数的优化结果
结构性转移识别.csv	结构性转移门槛
上下文长度敏感性.csv	C7 五档上下文长度下的最优配置
质量基线灵敏度.csv	质量基线口径灵敏度
配比项算力等价.csv	配比优化的算力等价效果
筛选漏斗.csv	时间轴/开源/模型类型三套口径的筛选漏斗
C8 逐任务解析结果.csv	C8 全部模型的任务级原始指标
C8 逐任务聚合.csv、C8 子任务类别汇总.csv	子任务前沿与剩余空间

表 28 主要结果文件说明（续）

文件	说明
C8 与 C1 一致性校验.csv	C8 与 C1 的秩相关校验
Loss_Benchmark 映射.csv	分层桥接拟合结果
规模时间分解.csv、规模时间分解 _ 长周期.csv	规模/非规模贡献分解
C3 口径异常行.csv	C3 自治性审计剔除的记录
前沿预测.csv、前沿预测 _ 结构外推 情景.csv	能力前沿预测与情景
C4 宏观趋势.csv、C4 宏观摘要.csv	C4 宏观算力与开源权重证据
求解/广义标度律参数.csv、问题一 _ 关键量.json	跨问共享的接口参数

核心求解代码

本附录列出四个问题的核心求解代码，代码文件位于求解目录下对应的问题一至问题四子目录中，共享工具函数位于求解/_common.py。

问题一：质量评价、冲突消解与配比建模（问题一.py）

```
1 # -*- coding: utf-8 -*-
2 """
3 问题一：数据质量评价、质量冲突消解与领域配比建模
4 数据：附件 A（A1-A3 质量信号；A4-A15 配方/Loss；A16 域映射）
5 方法：1) 22 指标方向统一 + 极差归一化 + 熵权法三级综合评分（样
        本/语料/领域）
6         2) 成对 z 分数差冲突定义 + Kendall 协同系数一致性检验 +
        多数方向裁决
7         3) 岭正则线性混合回归刻画 17 域配比 -> 13 域损失，并检验
        追加质量项的增量解释力
8 输出：图片到 图片/，结果 CSV 到 结果/
9 """
10 import sys, os, json, lzma, glob
11 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.
    abspath(__file__))))
12 from _common import * # noqa: F401,F403
13
14 DATA = data_path
15 RC_FAILS = []
16
17 def save_csv_safe(df, name_cn):
18     """保存 CSV；若目标被其他进程占用则回退为 *_new.csv，保证
        流程不中断。"""
19     p = os.path.join(OUT_DIR, name_cn)
20     try:
21         df.to_csv(p, index=False, encoding='utf-8-sig')
22     except PermissionError:
23         alt = os.path.join(OUT_DIR, name_cn.replace('.csv', '
        _new.csv'))
24         df.to_csv(alt, index=False, encoding='utf-8-sig')
25         RC_FAILS.append((name_cn, os.path.basename(alt)))
26         print(' ! %s 被占用，已改存为 %s' % (name_cn, os.path
```

```

        .basename(alt)))
27
28 # ===== 0. 22 个质量指标的方向定义 =====
29 # positive = 越大越好; negative = 越小越好; target = 越接近稳
    健中心越好
30 QUALITY_COLS = [
31     'fineweb_edu', 'fluency_en', 'modernbert_cleanliness',
32     'modernbert_readability', 'modernbert_reasoning',
33     'modernbert_professionalism', 'dsir_books', 'dsir_wiki', '
    dsir_math',
34     'qurater', 'ad_en', 'rps_doc_word_count', '
    rps_doc_num_sentences',
35     'rps_doc_unigram_entropy', 'rps_doc_frac_unique_words',
36     'rps_doc_frac_no_alph_words', '
    rps_doc_frac_chars_top_2gram',
37     'rps_doc_frac_chars_top_3gram', '
    rps_lines_uppercase_letter_fraction',
38     'rps_lines_ending_with_terminal_punctution_mark',
39     'rps_lines_numerical_chars_fraction', '
    rps_doc_mean_word_length'
40 ]
41 NEGATIVE_COLS = {
42     'rps_doc_frac_no_alph_words', '
    rps_doc_frac_chars_top_2gram',
43     'rps_doc_frac_chars_top_3gram', '
    rps_lines_uppercase_letter_fraction',
44     'rps_lines_numerical_chars_fraction'
45 }
46 TARGET_COLS = {'rps_doc_word_count', 'rps_doc_num_sentences',
    'rps_doc_mean_word_length'}
47 LIST_COLS = ['fineweb_edu', 'fluency_en', '
    modernbert_cleanliness',
48             'modernbert_readability', 'modernbert_reasoning',
49             'modernbert_professionalism', 'qurater', 'ad_en']
50 DIRECTION_NOTE = {
51     'fineweb_edu': '教育价值（列表型，1 维 logit）',
52     'fluency_en': '英语流畅度（列表型，2 维，取第 2 类概率）',
53     'modernbert_cleanliness': '文本洁净度（列表型，6 级，取归

```

```

一化期望级别) ',
54     'modernbert_readability': '可读性 (列表型, 6 级) ',
55     'modernbert_reasoning': '推理性 (列表型, 6 级) ',
56     'modernbert_professionalism': '专业性 (列表型, 6 级) ',
57     'dsir_books': 'DSIR 书籍域重要性权重',
58     'dsir_wiki': 'DSIR 维基域重要性权重',
59     'dsir_math': 'DSIR 数学域重要性权重',
60     'qurater': '教育质量评级 (列表型, 4 级) ',
61     'ad_en': '广告含量 (列表型, 2 维; 第 1 类=“无广告”, 正
        向。依据: 原文抽查显示第 1 类概率最低的样本均为广告文案) ',
62     'rps_doc_word_count': '文档词数 (过短/过长均不佳) ',
63     'rps_doc_num_sentences': '文档句数 (过少/过多均不佳) ',
64     'rps_doc_unigram_entropy': '一元词熵 (越高越多样) ',
65     'rps_doc_frac_unique_words': '唯一词占比 (越高越多样) ',
66     'rps_doc_frac_no_alph_words': '非字母字符占比 (越低越好) '
    ,
67     'rps_doc_frac_chars_top_2gram': 'Top-2 字符 2-gram 占比
        (重复度, 越低越好) ',
68     'rps_doc_frac_chars_top_3gram': 'Top-2 字符 3-gram 占比
        (重复度, 越低越好) ',
69     'rps_lines_uppercase_letter_fraction': '大写字母行占比 (越
        低越好) ',
70     'rps_lines_ending_with_terminal_punctution_mark': '以终止
        标点结尾的行占比 (越高越好) ',
71     'rps_lines_numerical_chars_fraction': '含数字字符行占比
        (越低越好) ',
72     'rps_doc_mean_word_length': '平均词长 (过短/过长均不佳) ',
73 }
74 CONFLICT_TAU = 1.5          # 成对 z 分数差阈值
75
76 def _softmax_expected(v):
77     """把有序类别 logits 压缩为 [0,1] 上的归一化期望级别。"""
78     a = np.asarray(v, dtype=float).ravel()
79     if len(a) == 0:
80         return np.nan
81     if len(a) == 1:
82         return float(a[0])
83     a = a - np.nanmax(a)

```

```

84     p = np.exp(a); p = p / np.nansum(p)
85     return float(p @ np.linspace(0.0, 1.0, len(a)))
86
87 def _scalarize(name, value):
88     if value is None:
89         return np.nan
90     if isinstance(value, (list, tuple)):
91         return _softmax_expected(value)
92     try:
93         return float(value)
94     except Exception:
95         return np.nan
96
97 def _norm_domain(x):
98     s = str(x).lower().replace('-', '_').replace(' ', '_')
99     aliases = {'common_crawl': 'commoncrawl', 'books': 'book',
100               'wikipedia_en': 'wikipedia'}
101     return aliases.get(s, s)
102
103 def load_quality_records():
104     """流式读取 A1-A3 全量记录，只保留建模字段和 A1 原文摘要。"""
105     sample_path = DATA('slimpajama_quality_signal_sample.jsonl')
106     ext_paths = sorted(glob.glob(os.path.join(DATA_DIR, 'A_data_value',
107                                                'slimpajama_quality_extended', '*.jsonl.xz')))
108     rows, meta = [], []
109     miss = {c: 0 for c in QUALITY_COLS}
110
111     def _is_missing(v):
112         """字段级缺失判定：None、空列表、列表内含 None/NaN、标
113         量为非有限数均计为缺失。"""
114         if v is None:
115             return True
116         if isinstance(v, (list, tuple)):
117             if len(v) == 0:

```

```

116         return True
117         return any((x is None) or (isinstance(x, float)
and not np.isfinite(x)) for x in v)
118     try:
119         return not np.isfinite(float(v))
120     except Exception:
121         return True
122
123 def consume(path, source, domain_hint=None):
124     opener = lzma.open if path.endswith('.xz') else open
125     with opener(path, 'rt', encoding='utf-8') as fh:
126         for line in fh:
127             try:
128                 obj = json.loads(line)
129             except Exception:
130                 continue
131             domain = _norm_domain(obj.get('_source_domain'
, domain_hint))
132             for c in QUALITY_COLS:
133                 if _is_missing(obj.get(c)):
134                     miss[c] += 1
135             rows.append([_scalarize(c, obj.get(c)) for c
in QUALITY_COLS])
136             excerpt = ''
137             if source == 'A1':
138                 excerpt = ' '.join(str(obj.get('content',
'')).split())[:180]
139             meta.append((obj.get('id', ''), domain, source
, excerpt))
140
141     consume(sample_path, 'A1')
142     for path in ext_paths:
143         hint = 'arxiv' if os.path.basename(path).lower().
startswith('arxiv') else 'github'
144         consume(path, 'A2' if hint == 'arxiv' else 'A3', hint)
145     X = pd.DataFrame(rows, columns=QUALITY_COLS)
146     M = pd.DataFrame(meta, columns=['id', 'domain', 'source',
'excerpt'])

```

```

147     miss_df = pd.DataFrame([{'indicator': k, 'missing_count':
148 v} for k, v in miss.items()])
149     save_csv_safe(miss_df, '质量信号缺失审计.csv')
150     if miss_df.missing_count.sum() > 0:
151         print('\n质量信号缺失审计（合计 %d 个指标取值缺失）： '
152 % int(miss_df.missing_count.sum()))
153         print(miss_df[miss_df.missing_count > 0].to_string(
154 index=False))
155         print('    处理方式：方向统一后按该指标中位数填充（不把
156 缺失当作高质量或低质量）。')
157     return X, M
158
159 def orient_and_normalize(raw):
160     """方向统一 -> 极差归一化（正向缩放、负向取补、长度型转中
161 心接近度）。"""
162     oriented = raw.copy()
163     for c in TARGET_COLS:
164         z = np.log1p(np.clip(raw[c].values, 0, None))
165         med = np.median(z); mad = np.median(np.abs(z - med)) +
166 1e-9
167         oriented[c] = np.exp(-np.abs(z - med) / (2.5 * mad))
168     norm = pd.DataFrame(index=raw.index)
169     for c in QUALITY_COLS:
170         lo, hi = np.nanquantile(oriented[c], [0.01, 0.99])
171         v = np.clip((oriented[c] - lo) / max(hi - lo, 1e-12),
172 0, 1)
173         norm[c] = 1 - v if c in NEGATIVE_COLS else v
174     return oriented, norm
175
176 def quality_evaluation():
177     raw, meta = load_quality_records()
178     raw = raw.replace([np.inf, -np.inf], np.nan)
179     raw = raw.apply(lambda s: s.fillna(s.median()))
180     _, norm = orient_and_normalize(raw)
181
182     # ---- 熵权法（主方案） ----
183     P = (norm.values + 1e-12) / (norm.values.sum(axis=0,
184 keepdims=True) + 1e-12)

```

```

177     entropy = -(P * np.log(P)).sum(axis=0) / np.log(len(norm))
178     divergence = np.maximum(1 - entropy, 0)
179     w_entropy = divergence / divergence.sum()
180     # ---- 对照方案: 等权 / 第一主成分 ----
181     w_equal = np.full(len(QUALITY_COLS), 1.0 / len(
QUALITY_COLS))
182     Z = (norm.values - norm.values.mean(0)) / (norm.values.std
(0) + 1e-12)
183     _, _, vt = np.linalg.svd(Z, full_matrices=False)
184     pc1 = vt[0]
185     if pc1.sum() < 0:
186         pc1 = -pc1
187     w_pca = np.clip(pc1, 0, None); w_pca = w_pca / w_pca.sum()
188
189     names = ['熵权法', '等权平均', 'PCA第一主成分']
190     wdict = dict(zip(names, [w_entropy, w_equal, w_pca]))
191     sc = {k: norm.values @ wdict[k] for k in names}
192     dom_tmp = meta.assign(**sc).groupby('domain')[names].mean
()
193     cmp_df = pd.DataFrame({
194         'scheme': names,
195         'score_mean': [sc[k].mean() for k in names],
196         'score_std': [sc[k].std() for k in names],
197         'corr_with_entropy': [float(np.corrcoef(sc[k], sc['熵
权法'])[0, 1]) for k in names],
198         'domain_spearman_vs_entropy': [1.0,
199             stats.spearmanr(dom_tmp['熵权法'], dom_tmp['等权平
均']).statistic,
200             stats.spearmanr(dom_tmp['熵权法'], dom_tmp['PCA第
一主成分']).statistic]})
201     save_csv_safe(cmp_df, '赋权方案对比.csv')
202
203     wdf = pd.DataFrame({'indicator': QUALITY_COLS,
204         'list_type': [c in LIST_COLS for c in
QUALITY_COLS],
205         'direction': ['negative' if c in
NEGATIVE_COLS else
206             ('target' if c in

```

```

TARGET_COLS else 'positive')
207                                     for c in QUALITY_COLS],
208                                     'semantics': [DIRECTION_NOTE[c] for c
in QUALITY_COLS],
209                                     'entropy': entropy, 'weight':
w_entropy})
210     save_csv_safe(wdf, '质量指标熵权.csv')
211     meta['quality_Q'] = sc['熵权法']
212
213     # ---- 冲突分析：成对 z 分数差 > tau ----
214     def agreement(idx, label):
215         Zs = norm.loc[idx]
216         m, n = Zs.shape                                # m 条文本（对
象），n=22 指标（评价者）
217         rank_sum = np.zeros(m)
218         for c in Zs.columns:
219             rank_sum += stats.rankdata(Zs[c].values, method='
average')
220         S = np.sum((rank_sum - rank_sum.mean()) ** 2)
221         W = 12 * S / max(n ** 2 * (m ** 3 - m), 1)
222         Am = (Zs.values - Zs.values.mean(axis=0)) / (Zs.values
.std(axis=0) + 1e-12)
223         iu = np.triu_indices(n, 1)
224         pairdiff = np.abs(Am[:, iu[0]] - Am[:, iu[1]])
225         rate = float((pairdiff > CONFLICT_TAU).mean())
226         rate_free = float(2 * stats.norm.sf(CONFLICT_TAU / np.
sqrt(2)))
227         chi2 = n * (m - 1) * W
228         pair_rate = pd.Series((pairdiff > CONFLICT_TAU).mean(
axis=0),
229                               index=['%s|%s' % (QUALITY_COLS[i
], QUALITY_COLS[j])
230                                     for i, j in zip(*iu)])
231         return {'dataset': label, 'n': m, 'kendall_W': W, '
chi2': chi2,
232               'p_value': stats.chi2.sf(chi2, m - 1),
233               'conflict_rate': rate, 'indep_baseline':
rate_free,

```

```

234         'conflict_ratio_vs_indep': rate / rate_free},
pair_rate
235
236     ag, pair_table = [], {}
237     for idx, label in [(meta.index[meta.source == 'A1'], 'A1抽
样集'),
238                        (meta.index[meta.source != 'A1'], 'A2+
A3扩展集'),
239                        (meta.index, '全量')]:
240         a, pf = agreement(idx, label)
241         ag.append(a); pair_table[label] = pf
242         save_csv_safe(pd.DataFrame(ag), '质量冲突与一致性.csv')
243         top_pairs = pair_table['全量'].sort_values(ascending=False
).head(20).rename('conflict_rate')
244         save_csv_safe(top_pairs.reset_index().rename(columns={'
index': 'indicator_pair'}),
245                       '冲突指标对_top20.csv')
246
247     # ---- 冲突阈值灵敏度: tau 变动对冲突率与结论的影响 ----
248     thr_rows = []
249     Zall = (norm.values - norm.values.mean(axis=0)) / (norm.
values.std(axis=0) + 1e-12)
250     iu_all = np.triu_indices(22, 1)
251     pd_all = np.abs(Zall[:, iu_all[0]] - Zall[:, iu_all[1]])
252     for tau in [0.5, 1.0, 1.5, 2.0, 2.5]:
253         rc = float((pd_all > tau).mean())
254         r0 = float(2 * stats.norm.sf(tau / np.sqrt(2)))
255         thr_rows.append({'阈值tau': tau, '冲突率': rc, '独立基
准': r0,
256                        '比值': rc / r0, '结论': '指标正相关
(冲突率低于独立基准)'
257                        if rc < r0 else '指标间无正相关'})
258     save_csv_safe(pd.DataFrame(thr_rows), '冲突阈值灵敏度.csv'
)
259
260     # ---- 域级质量: 抽样集与扩展集对照 ----
261     domain = meta.groupby(['source', 'domain']).agg(
262         n=('quality_Q', 'size'), quality_mean=('quality_Q', '

```

```

mean'),
263     quality_std=('quality_Q', 'std')).reset_index()
264     save_csv_safe(domain, '质量域评分_抽样与扩展.csv')
265     comp = domain.pivot(index='domain', columns='source',
values='quality_mean')
266     a1 = comp['A1']
267     ext = comp.drop(columns=[c for c in comp.columns if c == '
A1']).mean(axis=1)
268     comp_out = pd.DataFrame({'domain': comp.index, 'Q_A1': a1.
values, 'Q_扩展集': ext.values})
269     comp_out['差值_A1减扩展'] = comp_out['Q_A1'] - comp_out['
Q_扩展集']
270     save_csv_safe(comp_out, '抽样集与扩展集域级对照.csv')
271     save_csv_safe(meta[['id', 'domain', 'source', 'quality_Q'
]], '样本级质量评分.csv')
272
273     # ---- 高低分原文抽查（验证评分方向） ----
274     checks = []
275     s1 = meta[meta.source == 'A1']
276     for d, g in s1.groupby('domain'):
277         checks.append(g.nsmallest(2, 'quality_Q').assign(group
='低分'))
278         checks.append(g.nlargest(2, 'quality_Q').assign(group=
'高分'))
279     save_csv_safe(pd.concat(checks)[['domain', 'group', '
quality_Q', 'excerpt']], '原文抽查.csv')
280
281     # ---- A16 跨体系映射 ----
282     pooled = meta.groupby('domain')['quality_Q'].mean().
to_dict()
283     mapping = pd.read_csv(DATA('domain_mapping_guide.csv'))
284     global_q = float(meta.quality_Q.mean())
285     mapping['quality_Q'] = [pooled.get(str(q), global_q) if
str(q) != '(none)' else global_q
286                             for q in mapping['quality_domain'
]]
287     mapping['quality_source'] = np.where(mapping.
quality_domain.eq('(none)'),

```

```

288                                     '全局均值推断', 'A1-
A3域映射')
289     save_csv_safe(mapping, '17域质量映射.csv')
290     mapping.to_csv(shared_path('17域质量映射.csv'), index=
False, encoding='utf-8-sig')
291     print('质量全量记录 %s 条; A1=%s, A2+A3=%s' % (f'{len(meta
):,}',
292         f'{int((meta.source == "A1").sum()):,}', f'{int((
meta.source != "A1").sum()):,}'))
293     print(pd.DataFrame(ag).to_string(index=False))
294     print('\n赋权方案对比: ')
295     print(cmp_df.to_string(index=False))
296     print('\nad_en 熵权 = %.4f (单指标权重最大项, 须在敏感性分
析中讨论) '
297         % w_entropy[QUALITY_COLS.index('ad_en')])
298     print('冲突最集中的 5 组指标对: ')
299     print(top_pairs.head(5).to_string())
300     return meta, domain, mapping, norm, w_entropy, pair_table
301
302 # ===== 1. A1-A3 全量质量评价与冲突检验 =====
303 (quality_meta, quality_compare, quality_mapping,
304  quality_norm, quality_weights, quality_pairs) =
    quality_evaluation()
305 quality_domain = quality_meta.groupby('domain').agg(
306     n=('quality_Q', 'size'), quality_mean=('quality_Q', 'mean'
),
307     quality_std=('quality_Q', 'std')).reset_index().
    sort_values('quality_mean')
308 save_csv_safe(quality_domain, '质量域评分_全量.csv')
309
310 # ===== 2. 配比与 Loss 数据加载 =====
311 def load_pair(mix_path, loss_path):
312     m = pd.read_csv(mix_path).sort_values('index').reset_index
(drop=True)
313     l = pd.read_csv(loss_path).sort_values('index').
    reset_index(drop=True)
314     mix_cols = [c for c in m.columns if c.startswith('
train_the_pile_')]

```

```

315     loss_cols = [c for c in l.columns if 'val_loss' in c]
316     return m[mix_cols].values.astype(float), l[loss_cols].
        values.astype(float), m['index'].values
317
318 def short(c):
319     return c.replace('train_the_pile_', '').replace('metric/
        the_pile_', '').replace('_val_loss', '')
320
321 Xtr, Ytr, itr = load_pair(DATA('train_mixture_1m.csv'), DATA('
        train_pile_loss_1m.csv'))
322 Xte1, Yte1, _ = load_pair(DATA('test_mixture_1m.csv'), DATA('
        test_pile_loss_1m.csv'))
323 Xte2, Yte2, _ = load_pair(DATA('test_mixture_60m.csv'), DATA('
        test_pile_loss_60m.csv'))
324 Xte3, Yte3, _ = load_pair(DATA('test_mixture_1B.csv'), DATA('
        test_pile_loss_1B.csv'))
325 Xes10, Yes10, es10_idx = load_pair(DATA('est_mixture_10b.csv')
        , DATA('est_pile_loss_10b.csv'))
326 Xes70, Yes70, es70_idx = load_pair(DATA('est_mixture_70b.csv')
        , DATA('est_pile_loss_70b.csv'))
327
328 tm_df = pd.read_csv(DATA('train_mixture_1m.csv'))
329 MIX_COLS = [c for c in tm_df.columns if c.startswith('
        train_the_pile_')]
330 tl_df = pd.read_csv(DATA('train_pile_loss_1m.csv'))
331 LOSS_COLS = [c for c in tl_df.columns if 'val_loss' in c]
332 DOMAINS = [short(c) for c in MIX_COLS]
333 LOSS_DOMAINS = [short(c) for c in LOSS_COLS]
334 UNMEASURED = ['nih_exporter', 'enron_emails', 'europarl', '
        philpapers'] # 无损失列
335 MEASURED = [d for d in DOMAINS if d not in UNMEASURED]
336
337 # ===== 3. 岭正则线性混合回归 =====
338 def fit_ridge(X, Y, alpha=1e-3):
339     n, K = X.shape
340     Xc = np.hstack([X, np.ones((n, 1))])
341     reg = alpha * np.eye(K + 1); reg[-1, -1] = 0
342     W = np.linalg.solve(Xc.T @ Xc + reg, Xc.T @ Y)

```

```

343     return W[:K], W[-1]
344
345 def predict(X, A, b):
346     return X @ A + b
347
348 def r2(y, yhat):
349     ss_res = np.sum((y - yhat) ** 2, axis=0)
350     ss_tot = np.sum((y - y.mean(axis=0)) ** 2, axis=0)
351     return 1 - ss_res / np.maximum(ss_tot, 1e-12)
352
353 def pearson_col(y, yhat):
354     return np.array([np.corrcoef(y[:, j], yhat[:, j])[0, 1]
355                      for j in range(y.shape[1])])
356
357 A, b = fit_ridge(Xtr, Ytr)
358 r2_tr = r2(Ytr, predict(Xtr, A, b))
359 r2_tel = r2(Ytel, predict(Xtel, A, b))
360 r_te2 = pearson_col(Yte2, predict(Xte2, A, b))
361 r_te3 = pearson_col(Yte3, predict(Xte3, A, b))
362
363 print('\n=== 岭回归训练 R2 (各损失域) ===')
364 for d, v in zip(LOSS_DOMAINS, r2_tr):
365     print(' %-20s %.4f' % (d, v))
366 print('训练平均 R2 = %.4f; 同尺度 1M 检验平均 R2 = %.4f' % (
367     r2_tr.mean(), r2_tel.mean()))
368 print('跨尺度 Pearson: 60M 平均 %.4f, 1B 平均 %.4f' % (r_te2.
369     mean(), r_te3.mean()))
370
371 A_df = pd.DataFrame(A, columns=LOSS_DOMAINS)
372 A_df.insert(0, 'training_domain', DOMAINS)
373 save_csv_safe(A_df, '混合系数矩阵.csv')
374 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, '
375     intercept': b}), '截距.csv')
376
377 lam_rows = []
378 for lam in [1e-4, 1e-3, 1e-2, 1e-1]:
379     A_l, b_l = fit_ridge(Xtr, Ytr, lam)
380     lam_rows.append({'lambda': lam, 'train_mean_R2': r2(Ytr,
381         predict(Xtr, A_l, b_l)).mean(),

```

```

376         'testlm_mean_R2': r2(Ytel, predict(Xtel,
    Al, bl)).mean(),
377         'coef_L2': float(np.linalg.norm(A1)))
378 save_csv_safe(pd.DataFrame(lam_rows), '岭正则敏感性.csv')
379
380 # ===== 4. 质量信息引入配比模型的增量检验 =====
381 qmap = quality_mapping.set_index('mixture_domain')['quality_Q'
    ]
382 qvec = np.array([qmap.get(d, np.nan) for d in DOMAINS], dtype=
    float)
383 qvec = np.where(np.isnan(qvec), float(quality_meta.quality_Q.
    mean()), qvec)
384 Zq = np.hstack([Xtr, (Xtr @ qvec)[:, None]])
385 Az, bz = fit_ridge(Zq, Ytr, 1e-3)
386 r2_z = r2(Ytr, predict(Zq, Az, bz))
387 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, '
    train_R2_仅配比': r2_tr,
388                             'train_R2_追加Qbar': r2_z, 'R2增量
    ': r2_z - r2_tr,
389                             'Qbar系数': Az[-1]}), '质量项增量
    检验.csv')
390
391 diff_mean = Ytr.mean(axis=0)
392 quality_proxy = 1 - (diff_mean - diff_mean.min()) / (diff_mean
    .max() - diff_mean.min())
393 q_df = pd.DataFrame({'domain': LOSS_DOMAINS, 'mean_loss(
    difficulty)': diff_mean,
394                     'quality_score_Q(loss代理)':
    quality_proxy}).sort_values(
395     'quality_score_Q(loss代理)', ascending=False)
396 save_csv_safe(q_df, 'Loss代理领域难度.csv')
397 save_csv_safe(q_df, '领域质量评分.csv')
398
399 link = pd.read_csv(DATA('domain_mapping_guide.csv'))
400 link = link[link.mapping_type.isin(['direct', 'near_direct'])
    ].copy()
401 qd_map = quality_domain.set_index('domain').quality_mean
402 qp_map = q_df.set_index('domain')['quality_score_Q(loss代理)']

```

```

403 link['Q_A1A3'] = [float(qd_map.get(q, np.nan)) for q in link.
    quality_domain]
404 link['Q_loss代理'] = [float(qp_map.get(m, np.nan)) for m in
    link.mixture_domain]
405 rho_link = float(stats.spearmanr(link.Q_A1A3, link.Q_loss代理)
    .statistic)
406 save_csv_safe(link[['mixture_domain', 'quality_domain', '
    mapping_type', 'Q_A1A3', 'Q_loss代理']],
407     '质量与损失难度一致性.csv')
408
409 A_cent = A - A.mean(axis=0, keepdims=True)
410 A_cent_df = pd.DataFrame(A_cent, columns=LOSS_DOMAINS)
411 A_cent_df.insert(0, 'training_domain', DOMAINS)
412 save_csv_safe(A_cent_df, '混合系数矩阵_中心化.csv')
413 save_csv_safe(pd.DataFrame({'training_domain': DOMAINS,
414     'marginal_effect_on_avg_loss':
    A_cent.mean(axis=1)}),
415     '训练域边际效应.csv')
416
417 # ===== 5. 外推与跨尺度稳健性 =====
418 tr_lookup = {i: Ytr[j] for j, i in enumerate(itr)}
419 ratio10 = Yes10 / np.array([tr_lookup[i] for i in es10_idx])
420 ratio70 = Yes70 / np.array([tr_lookup[i] for i in es70_idx])
421 print('\n=== 外推尺度因子 (est/train 损失比) ===')
422 print(' 10B 均值 %.4f (std %.4f)' % (ratio10.mean(), ratio10.
    std()))
423 print(' 70B 均值 %.4f (std %.4f)' % (ratio70.mean(), ratio70.
    std()))
424 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, '
    scale_factor_10B': ratio10.mean(axis=0),
425     'scale_factor_70B': ratio70.mean(
    axis=0)}), '外推尺度因子.csv')
426 ratio_scale = Yte2 / Yte1
427 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS,
428     'loss_ratio_60m_vs_1m':
    ratio_scale.mean(axis=0),
429     'loss_ratio_std': ratio_scale.std(
    axis=0)}),

```

```

430         '跨尺度损失比_60m_vs_1m.csv')
431
432 # ===== 6. 最优配比 (无约束 + 2.5 倍上界推荐)
433     =====
434 from scipy.optimize import minimize
435 ref = Xtr.mean(axis=0)
436 w_ref = ref[[DOMAINS.index(d) for d in LOSS_DOMAINS]]
437 w_ref = w_ref / w_ref.sum()
438 unmeas_sum = ref[[DOMAINS.index(u) for u in UNMEASURED]].sum()
439 meas_idx = [DOMAINS.index(d) for d in MEASURED]
440
441 def build(pm):
442     p = np.zeros(17); p[meas_idx] = pm
443     for u in UNMEASURED:
444         p[DOMAINS.index(u)] = ref[DOMAINS.index(u)]
445     return p / p.sum()
446
447 def obj(pm):
448     return float(w_ref @ predict(build(pm)[None, :], A, b)[0])
449
450 p0m = ref[meas_idx] / ref[meas_idx].sum()
451 cons = {'type': 'eq', 'fun': lambda pm: pm.sum() - (1 -
452         unmeas_sum)}
453 res = minimize(obj, p0m, method='SLSQP', bounds=[(0, 1)] * len
454         (MEASURED), constraints=cons)
455 p_opt = build(res.x)
456 save_csv_safe(pd.DataFrame({'domain': DOMAINS, '
457         reference_mixture': ref,
458         'optimal_mixture': p_opt}).
459         sort_values('optimal_mixture', ascending=False),
460         '最优配比预测.csv')
461
462 bnds = [(0, 2.5 * ref[i]) for i in meas_idx]
463 res_b = minimize(obj, p0m, method='SLSQP', bounds=bnds,
464         constraints=cons)
465 p_b = build(res_b.x)
466 rec_df = pd.DataFrame({'domain': DOMAINS, 'reference_mixture':
467         ref, 'recommended_mixture': p_b,
468         'adjustment': p_b - ref}).sort_values('

```

```

    adjustment', ascending=False)
461 save_csv_safe(rec_df, '推荐配比调整.csv')
462 print('\n无约束最优加权平均损失 %.4f; 有界推荐 %.4f; 参考配方
    %.4f'
463         % (obj(res.x), obj(res_b.x), obj(p0m)))
464 print('有界推荐配比（调整量最大增/减各 5 域）：')
465 print(rec_df.head(5).to_string(index=False))
466 print(rec_df.tail(5).to_string(index=False))
467
468 # ===== 7. 绘图 =====
469 plt.rcParams['font.size'] = 12
470 cmap = safe_cmap('tab20')
471 colors = [cmap(i % 20) for i in range(17)]
472
473 fig, ax = plt.subplots(figsize=(8.5, 4.8))
474 ax.barh(quality_domain['domain'], quality_domain['quality_mean
    '],
475         xerr=quality_domain['quality_std'], color='#3b6ea5',
        alpha=0.85)
476 ax.set_xlabel('全量质量评分 Q（均值±标准差）')
477 ax.set_title('A1 - A3 全量质量信号的域级评分')
478 despine(ax); fig.tight_layout(); save_fig(fig, '图0_A1-A3全量
    质量评分')
479
480 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
481 ax = axes[0]
482 for r in range(8):
483     bottom = 0
484     for k in range(17):
485         ax.bar(r, Xtr[r, k], bottom=bottom, color=colors[k],
            width=0.7)
486         bottom += Xtr[r, k]
487 ax.set_xticks(range(8)); ax.set_xticklabels(['#%d' % (r + 1)
    for r in range(8)])
488 ax.set_ylabel('领域配比 p_k'); ax.set_title('(a) 训练配方示例
    （前 8 组）'); ax.set_ylim(0, 1.05)
489 ax = axes[1]
490 order = np.argsort(-ref)

```

```

491 ax.bar(range(17), ref[order], color=[colors[k] for k in order
    ])
492 ax.set_xticks(range(17)); ax.set_xticklabels([DOMAINS[k] for k
    in order],
493
    rotation=60, ha='
    right', fontsize=8)
494 ax.set_ylabel('平均配比'); ax.set_title('(b) 全部训练配方平均
    配比 (参考配方)')
495 fig.tight_layout(); save_fig(fig, '图1_训练配比结构')
496
497 fig, ax = plt.subplots(figsize=(10, 4.8))
498 x = np.arange(13); w = 0.22
499 ax.bar(x - w, r2_tr, w, label='训练集(1M)', color='#3b6ea5')
500 ax.bar(x, r2_tel, w, label='检验(1M, 同尺度)', color='#e07a3f'
    )
501 ax.bar(x + w, r_te2, w, label='检验(60M, 跨尺度 r)', color='#5
    aa05a')
502 ax.axhline(0, color='k', lw=0.8)
503 ax.set_xticks(x); ax.set_xticklabels(LOSS_DOMAINS, rotation
    =45, ha='right', fontsize=9)
504 ax.set_ylabel('R2 / Pearson r'); ax.set_title('线性混合回归的
    拟合与泛化表现')
505 ax.legend(); despine(ax); fig.tight_layout(); save_fig(fig, '
    图2_线性模型R2')
506
507 fig, ax = plt.subplots(figsize=(6, 6))
508 Yh = predict(Xtel, A, b)
509 all_y, all_h = Ytel.ravel(), Yh.ravel()
510 lo, hi = min(all_y.min(), all_h.min()), max(all_y.max(), all_h
    .max())
511 ax.scatter(all_y, all_h, s=6, alpha=0.35, color='#3b6ea5')
512 ax.plot([lo, hi], [lo, hi], 'r--', lw=1.2, label='$y=\hat{y}$'
    )
513 ax.set_xlabel('实测交叉熵损失'); ax.set_ylabel('模型预测损失')
514 ax.set_title('同尺度检验集(1M): 预测 vs 实测')
515 ax.legend(); despine(ax); fig.tight_layout(); save_fig(fig, '
    图3_预测vs实测')
516

```

```

517 fig, ax = plt.subplots(figsize=(11, 6.5))
518 mesh = safe_heatmap(ax, A_cent.T, row_labels=LOSS_DOMAINS,
    col_labels=DOMAINS,
519                      cmap='RdBu_r', fontsize=9)
520 vmax = np.abs(A_cent).max()
521 mesh.set_clim(-vmax, vmax)
522 cb = fig.colorbar(mesh, ax=ax, shrink=0.85)
523 cb.set_label('中心化混合系数（相对效应）')
524 ax.set_xlabel('训练领域 k（17 域）'); ax.set_ylabel('验证损失
    领域 d（13 域）')
525 ax.set_title('领域配比对交叉熵损失的相对影响（暖=更难，冷=更
    易）')
526 fig.tight_layout(); save_fig(fig, '图4_混合系数热力图')
527
528 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))
529 ax = axes[0]
530 qs = q_df.sort_values('quality_score_Q(loss代理)')
531 ax.barh(qs['domain'], qs['quality_score_Q(loss代理)'],
532         color=plt.colormaps['RdYlGn'](qs['quality_score_Q(loss
    代理)'].values))
533 ax.set_xlabel('Loss 反向映射代理（越高表示越易）')
534 ax.set_title('(a) 13 个验证域的 Loss 难度代理')
535 despine(ax)
536 ax = axes[1]
537 ax.scatter(link.Q_A1A3, link.Q_loss代理, s=45, color='#3b6ea5'
    )
538 for _, r in link.iterrows():
539     ax.annotate(r.mixture_domain, (r.Q_A1A3, r.Q_loss代理),
    fontsize=7,
540                xytext=(3, 3), textcoords='offset points')
541 ax.set_xlabel('A1 - A3 域级质量 Q'); ax.set_ylabel('Loss 难度代
    理质量')
542 ax.set_title('(b) 两套口径排序一致性 Spearman  $\rho$ =%.2f' %
    rho_link)
543 despine(ax); fig.tight_layout(); save_fig(fig, '图5_领域质量评
    分')
544
545 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))

```

```

546 ax = axes[0]
547 tp = quality_pairs['全量'].sort_values(ascending=False).head
    (12)[::-1]
548 ax.barh(range(len(tp)), tp.values, color='#c44e52')
549 ax.set_yticks(range(len(tp))); ax.set_yticklabels(tp.index,
    fontsize=7)
550 ax.set_xlabel('该指标对的冲突样本占比')
551 ax.set_title('(a) 冲突最集中的指标对 ( $|z_j - z_k| > .1f$ ) ' %
    CONFLICT_TAU)
552 despine(ax)
553 ax = axes[1]
554 wi = np.argsort(quality_weights)[::-1]
555 ax.bar(range(22), quality_weights[wi], color='#4c72b0')
556 ax.set_xticks(range(22)); ax.set_xticklabels([QUALITY_COLS[k]
    for k in wi],
557
    rotation=75, ha='
    right', fontsize=7)
558 ax.set_ylabel('熵权  $w_j$ '); ax.set_title('(b) 熵权分布 (客观赋
    权诊断)')
559 despine(ax); fig.tight_layout(); save_fig(fig, '图6_冲突与赋权
    诊断')
560
561 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
562 ax = axes[0]
563 bp = safe_boxplot(ax, [ratio10[:, j] for j in range(13)],
    LOSS_DOMAINS)
564 for box in bp['boxes']:
565     box.set_facecolor('#aec7e8')
566 ax.set_xticklabels(LOSS_DOMAINS, rotation=45, ha='right',
    fontsize=8)
567 ax.axhline(ratio10.mean(), color='r', ls='--', lw=1, label='均
    值  $.3f$ ' % ratio10.mean())
568 ax.set_ylabel('损失比  $est/train$ '); ax.set_title('(a) 10B 外推
    尺度因子'); ax.legend()
569 ax = axes[1]
570 x = np.arange(13)
571 ax.bar(x, ratio_scale.mean(axis=0), 0.5, yerr=ratio_scale.std(
    axis=0),

```

```

572         color='#ff9f68', capsize=2, label='60M vs 1M')
573 ax.axhline(1, color='k', ls='--', lw=0.8)
574 ax.set_xticks(x); ax.set_xticklabels(LOSS_DOMAINS, rotation
    =45, ha='right', fontsize=8)
575 ax.set_ylabel('损失比 60M/1M'); ax.set_title('(b) 跨尺度损失比
    (同配方) '); ax.legend()
576 fig.tight_layout(); save_fig(fig, '图7_外推与跨尺度')
577
578 # ===== 8. 汇总 =====
579 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, '
    train_R2': r2_tr, 'test1m_R2': r2_te1,
580                             'test60m_r': r_te2, 'test1B_r':
    r_te3, 'mean_loss': diff_mean,
581                             'quality_Q(loss代理)':
    quality_proxy}), '问题一_拟合汇总.csv')
582
583 conf = pd.read_csv(os.path.join(OUT_DIR, '质量冲突与一致性.csv
    '))
584 meta_out = {
585     'train_mean_R2': float(r2_tr.mean()), 'test1m_mean_R2':
    float(r2_te1.mean()),
586     'test60m_mean_r': float(r_te2.mean()), 'test1B_mean_r':
    float(r_te3.mean()),
587     'n_quality_records': int(len(quality_meta)),
588     'n_A1': int((quality_meta.source == 'A1').sum()),
589     'n_ext': int((quality_meta.source != 'A1').sum()),
590     'Q0_global_mean': float(quality_meta.quality_Q.mean()),
591     'Q0_mix_weighted': float(ref @ qvec),
592     'kendall_W_full': float(conf.loc[conf.dataset.eq('全量'),
    'kendall_W'].iloc[0]),
593     'conflict_rate_full': float(conf.loc[conf.dataset.eq('全量
    '), 'conflict_rate'].iloc[0]),
594     'conflict_indep_baseline': float(conf.loc[conf.dataset.eq(
    '全量'), 'indep_baseline'].iloc[0]),
595     'spearman_quality_vs_lossproxy': float(rho_link),
596     'Qbar_R2_gain_mean': float((r2_z - r2_tr).mean()),
597 }
598 with open(shared_path('问题一_关键量.json'), 'w', encoding='

```

```

    utf-8') as fh:
599     json.dump(meta_out, fh, ensure_ascii=False, indent=2)
600
601 print('\n===== 问题一求解完成 =====')
602 print('训练平均 R2=%.4f; 同尺度检验 R2=%.4f; 跨尺度相关 60M
    =%.4f, 1B=%.4f'
603         % (r2_tr.mean(), r2_te1.mean(), r_te2.mean(), r_te3.mean
            ()))
604 print('追加混合质量指数 Qbar 的平均 R2 增量 = %.5f' % (r2_z -
    r2_tr).mean())
605 print('全局平均质量 Q0 = %.4f (供问题三基线使用) ' %
    quality_meta.quality_Q.mean())
606 if RC_FAILS:
607     print('注意： 以下结果文件因被占用而改存： ', RC_FAILS)

```

Listing 1: 问题一求解代码（质量综合评价、冲突消解与岭正则配比建模）

问题二：广义标度律（问题二.py）

```

1  # -*- coding: utf-8 -*-
2  """
3  问题二：跨维度数据融合与广义标度律
4  数据：附件 B（B1 主拟合；B2/B3 族外与轨迹验证；B4/B5 跨族与文
    献验证；
5      B6/B7 质量 Q；B8 大规模矩阵（方向异常，仅作可信度边界警
        示）；B9/B10 百亿参数以上外推）
6  模型：经典  $L = E + A N^{-a} + B D^{-b}$ 
7      广义  $L = E + A N^{-a} + B D^{-b} + C (1-Q)^g$  (Q=1 退化为经典
        形式)
8  输出：图片到 图片/, 结果 CSV 到 结果/, 参数共享至 求解/广义标
        度律参数.csv
9  """
10 import sys, os, glob
11 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.
    abspath(__file__))))
12 from _common import * # noqa: F401,F403
13 from scipy.optimize import least_squares
14

```

```

15 DATA = data_path
16
17 def save_csv_safe(df, name_cn):
18     p = os.path.join(OUT_DIR, name_cn)
19     try:
20         df.to_csv(p, index=False, encoding='utf-8-sig')
21     except PermissionError:
22         alt = p.replace('.csv', '_new.csv')
23         df.to_csv(alt, index=False, encoding='utf-8-sig')
24         print('    ! %s 被占用, 已改存为 %s' % (name_cn, os.path
        .basename(alt)))
25
26 pythia = pd.read_csv(DATA('pythia_training_log_existing.csv'))
27 cerebras = pd.read_csv(DATA('cerebras_training_log.csv'))
28 baseline = pd.read_csv(DATA('scaling_baseline.csv'))
29 published = pd.read_csv(DATA('published_scaling_data.csv'))
30 nq6 = pd.read_csv(DATA('supplementary_NQ_experiment.csv'))
31 nq7 = pd.read_csv(DATA('supplementary_NQ_experiment_expanded.
    csv'))
32 nq8 = pd.read_csv(DATA('supplementary_NQ_experiment_large.csv'
    ))
33 large_models = pd.read_csv(DATA('supplementary_large_models.
    csv'))
34 large_loss = pd.read_csv(DATA('supplementary_large_baseline.
    csv'))
35 traj_files = sorted(glob.glob(os.path.join(DATA_DIR, '
    B_scaling_laws',
36
        '
        training_trajectories', '*.csv'))))
37
38 N_p = pythia.N_params_B.to_numpy(float)
39 D_p = pythia.D_tokens_B.to_numpy(float)
40 L_p = pythia.val_loss.to_numpy(float)
41
42 # ===== 1. 经典标度律 (B1 主拟合 + 模型留出验证)
    =====
43 def classic(par, N, D):
44     E, A, a, B, b = par

```

```

45     return E + A * N ** (-a) + B * D ** (-b)
46
47 r0 = least_squares(lambda p: classic(p, N_p, D_p) - L_p, [1.7,
48     .4, .33, 1.2, .28],
49     bounds=([0, 1e-8, .01, 1e-8, .01], [5, 1e3,
50     2, 1e3, 2]),
51     loss='soft_l1', f_scale=.05, max_nfev
52     =40000)
53 E0, A0, a0, B0, b0 = r0.x
54 Lph = classic(r0.x, N_p, D_p)
55 R2_cls = 1 - np.sum((L_p - Lph) ** 2) / np.sum((L_p - L_p.mean
56     ()) ** 2)
57 RMSE_cls = float(np.sqrt(np.mean((L_p - Lph) ** 2)))
58
59 Ns = sorted(pythia.N_params_B.unique())
60 tr_m = pythia[pythia.N_params_B.isin(Ns[:4])]
61 te_m = pythia[pythia.N_params_B.isin(Ns[4:])]
62 rh = least_squares(lambda p: classic(p, tr_m.N_params_B.values
63     , tr_m.D_tokens_B.values)
64     - tr_m.val_loss.values, [1.7, .4, .33, 1.2,
65     .28],
66     bounds=([0, 1e-8, .01, 1e-8, .01], [5, 1e3,
67     2, 1e3, 2]),
68     loss='soft_l1', f_scale=.05, max_nfev
69     =40000)
70 hp = classic(rh.x, te_m.N_params_B.values, te_m.D_tokens_B.
71     values)
72 ym = te_m.val_loss.values
73 hold = {'留出R2': 1 - np.sum((ym - hp) ** 2) / np.sum((ym - ym
74     .mean()) ** 2),
75     '留出RMSE': float(np.sqrt(np.mean((ym - hp) ** 2))),
76     '留出MAPE_%': float(np.mean(np.abs(ym - hp) / ym) *
77     100)}
78
79 print('=== 1. 经典标度律 (B1 Pythia, n=%d) ===' % len(pythia))
80 print('    E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f' % (E0, A0,
81     a0, B0, b0))
82 print('    拟合 R2=%.8f  RMSE=%.6f  MAPE=%.4f%%' % (

```

```

71     R2_cls, RMSE_cls, np.mean(np.abs(L_p - Lph) / L_p) * 100))
72 print(' 留出验证 (4 小模型 -> 4 大模型) : R2=%.8f RMSE=%.6f
      MAPE=%.4f%%' % (
73     hold['留出R2'], hold['留出RMSE'], hold['留出MAPE_%']))
74 print(' 提示: B1 拟合残差标准差仅 %.5f (损失标准差 %.4f), 说
      明该附件近乎无观测噪声, '
75     '其上高 R2 只能确认函数形式自治, 不能替代外部验证。' % (
76         float(np.std(L_p - Lph)), float(np.std(L_p))))
77 save_csv_safe(pd.DataFrame([{'E': E0, 'A': A0, 'alpha': a0, 'B
      ': B0, 'beta': b0,
78                               'R2': R2_cls, 'RMSE': RMSE_cls,
79                               'MAPE_%': float(np.mean(np.abs(
      L_p - Lph) / L_p) * 100),
80                               '残差标准差': float(np.std(L_p -
      Lph))}, **hold])),
81     '经典标度律参数.csv')
82
83 # ===== 2. 质量字段方向审计 =====
84 def group_corr(df, qcol='Q_score'):
85     vals, ng = [], 0
86     for _, g in df.groupby(['N_params_B', 'D_tokens_B']):
87         if g[qcol].nunique() >= 3:
88             vals.append(g[qcol].corr(g.val_loss)); ng += 1
89     return float(np.nanmean(vals)), ng
90
91 audit = []
92 for nm, sub, note in [('B6', nq6, '质量主数据'),
93                       ('B7', nq7, '质量扩展数据 (含 B6 重复
94                           点)'),
95                       ('B8-calibrated', nq8[nq8.data_type.eq('
      calibrated')], '大规模矩阵校准段'),
96                       ('B8-extrapolated', nq8[nq8.data_type.eq
      ('extrapolated')], '大规模矩阵外推段')]:
97     r, ng = group_corr(sub)
98     audit.append({'dataset': nm, 'n': len(sub), 'n_groups': ng
99                 , 'within_ND_corr': r,
100                 'conclusion': ('Q 越大损失越低 -> 质量指标
      (越高越好) ' if r < -0.5 else

```

```

99         ('Q 越大损失越高 -> 方向与 B6
    /B7 相反' if r > 0.5 else
100         '组内无显著单调关系'))))
101 save_csv_safe(pd.DataFrame(audit), '质量字段方向审计.csv')
102 corr6 = audit[0]['within_ND_corr']; corr7 = audit[1]['
    within_ND_corr']
103 corr8 = audit[2]['within_ND_corr']
104 print('\n=== 2. 质量字段方向审计（控制 N、D 的组内相关）===')
105 print(pd.DataFrame(audit).to_string(index=False))
106 print(' 结论：B6/B7 的 Q_score 与损失负相关 (r=%.3f / %.3f
    ), 是“质量”而非“难度”，'
107       '故取 Q = Q_score (Q=1 最优)，与问题一“越高越好”口径
    一致。' % (corr6, corr7))
108 print('  B8 校准段方向相反 (r=%.3f)：两套半合成数据口径不一
    致，B8 不并入拟合。' % corr8)
109
110 # ===== 3. 广义标度律：形式比较与参数估计 =====
111 N_all = np.r_[N_p, nq6.N_params_B]
112 D_all = np.r_[D_p, nq6.D_tokens_B]
113 Q_all = np.r_[np.ones(len(pythia)), nq6.Q_score]      # B1 无
    质量信息，取参考水平 Q=1
114 L_all = np.r_[L_p, nq6.val_loss]
115
116 def g_v1(p, N, D, Q):
117     E, A, a, B, b, C, g = p
118     return E + A * N ** (-a) + B * D ** (-b) + C * np.clip(1 -
    Q, 0, 1) ** g
119
120 def g_v2(p, N, D, Q):
121     E, A, a, B, b, g = p
122     return E + A * N ** (-a) + B * (D * np.clip(Q, 1e-6, 1) **
    g) ** (-b)
123
124 def g_v4(p, N, D, Q):
125     E, A, a, B, b, C, g = p
126     return E + A * N ** (-a) + B * D ** (-b) + C * np.clip(Q,
    1e-6, 1) ** (-g)
127

```

```

128 def g_v5(p, N, D, Q):
129     E, A, a, B, b, g1, g2 = p
130     return (E + A * (N * np.clip(Q, 1e-6, 1) ** g1) ** (-a)
131             + B * (D * np.clip(Q, 1e-6, 1) ** g2) ** (-b))
132
133 FORMS = {
134     '加性质量缺口  $C(1-Q)^g$ ': (g_v1, [E0, A0, a0, B0, b0, .35,
135     1.],
136                                     [0, 1e-8, .01, 1e-8, .01, 1e-8,
137     .05], [5, 1e3, 2, 1e3, 2, 50, 10]),
138     '质量作有效数据乘子': (g_v2, [E0, A0, a0, B0, b0, .5],
139     [0, 1e-8, .01, 1e-8, .01, -10], [5,
140     1e3, 2, 1e3, 2, 30]),
141     '加性质量亏损  $C Q^{-g}$ ': (g_v4, [E0, A0, a0, B0, b0, .2,
142     .5],
143                                     [0, 1e-8, .01, 1e-8, .01, 1e-8,
144     .05], [5, 1e3, 2, 1e3, 2, 50, 10]),
145     '质量同时乘 N 与 D': (g_v5, [E0, A0, a0, B0, b0, .3, .3],
146     [0, 1e-8, .01, 1e-8, .01, -10, -10],
147     [5, 1e3, 2, 1e3, 2, 30, 30]),
148 }
149 nq7_new = nq7[~nq7.experiment_id.astype(str).isin(set(nq6.
150     experiment_id.astype(str)))]
151 form_rows, fit_store = [], {}
152 for nm, (fn, p0, lo, hi) in FORMS.items():
153     rr = least_squares(lambda p: fn(p, N_all, D_all, Q_all) -
154     L_all, p0,
155                         bounds=(lo, hi), loss='soft_l1',
156     f_scale=.05, max_nfev=80000)
157     hh = fn(rr.x, N_all, D_all, Q_all)
158     R2j = 1 - np.sum((L_all - hh) ** 2) / np.sum((L_all -
159     L_all.mean()) ** 2)
160     hq = fn(rr.x, nq6.N_params_B.values, nq6.D_tokens_B.values
161     , nq6.Q_score.values)
162     R2q = 1 - np.sum((nq6.val_loss.values - hq) ** 2) / \
163     np.sum((nq6.val_loss.values - nq6.val_loss.mean()) **
164     2)
165     h7 = fn(rr.x, nq7_new.N_params_B.values, nq7_new.

```

```

D_tokens_B.values, nq7_new.Q_score.values)
154     r7 = float(np.corrcoef(nq7_new.val_loss.values, h7)[0, 1])
155     form_rows.append({'形式': nm, '联合R2': R2j, 'B6拟合R2':
R2q, 'B7新增点r': r7,
156                     '参数个数': len(rr.x),
157                     'AIC': len(N_all) * np.log(np.mean((
L_all - hh) ** 2)) + 2 * len(rr.x)})
158     fit_store[nm] = rr.x
159 sv = pd.DataFrame(form_rows).sort_values('联合R2', ascending=
False)
160 save_csv_safe(sv, '广义标度律形式对比.csv')
161 print('\n=== 3. 广义标度律形式对比 ===')
162 print(sv.to_string(index=False))
163
164 best_name = sv.iloc[0]['形式']
165 fn_best = FORMS[best_name][0]
166 r1x = fit_store[best_name]
167 E, A, alpha, B, beta, Cq, gamma = r1x
168 Lhat = fn_best(r1x, N_all, D_all, Q_all)
169 R2_gen = 1 - np.sum((L_all - Lhat) ** 2) / np.sum((L_all -
L_all.mean()) ** 2)
170 RMSE_gen = float(np.sqrt(np.mean((L_all - Lhat) ** 2)))
171 MAPE_gen = float(np.mean(np.abs(L_all - Lhat) / np.abs(L_all))
* 100)
172 print('\n选定形式: %s' % best_name)
173 print('    E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f C=%.4f
gamma=%.4f R2=%.4f RMSE=%.4f MAPE=%.2f%%'
174       % (E, A, alpha, B, beta, Cq, gamma, R2_gen, RMSE_gen,
MAPE_gen))
175
176 params = pd.DataFrame({'param': ['E', 'A', 'alpha', 'B', 'beta
', 'C', 'gamma'],
177                        'value': [E, A, alpha, B, beta, Cq,
gamma]})
178 save_csv_safe(params, '广义标度律参数.csv')
179 params.to_csv(shared_path('广义标度律参数.csv'), index=False,
encoding='utf-8-sig')
180

```

```

181 # ===== 4. 分层验证 =====
182 def metrics(df, Q=1.0):
183     q = np.full(len(df), Q) if np.isscalar(Q) else np.asarray(
184         Q)
185     ph = fn_best(r1x, df.N_params_B.to_numpy(float), df.
186         D_tokens_B.to_numpy(float), q)
187     y = df.val_loss.to_numpy(float)
188     return ph, float(np.corrcoef(y, ph)[0, 1]), float(np.sqrt(
189         np.mean((y - ph) ** 2))), \
190         float(np.mean(np.abs(y - ph) / np.maximum(np.abs(y), 1
191             e-9)) * 100)
192
193 nq8c = nq8[nq8.data_type.eq('calibrated')]
194 validation, pred_sets = [], []
195 for name, df, q, kind in [('B2_Cerebras(半合成,族外)',
196     cerebras, 1., '半合成'),
197     ('B4_跨族收敛点(真实)', baseline,
198     1., '真实'),
199     ('B5_已发表标度律(真实)', published,
200     1., '真实'),
201     ('B7_质量扩展新增点(半合成)',
202     nq7_new, nq7_new.Q_score, '半合成'),
203     ('B8_校准段(Q 反向重定向)', nq8c, 1
204     - nq8c.Q_score, '半合成'),
205     ('B10_百亿参数以上(估算)',
206     large_loss, 1., '估算')]:
207     ph, r, rm, mp = metrics(df, q)
208     validation.append({'dataset': name, 'n': len(df), '性质':
209         kind, 'r': r,
210         'RMSE': rm, 'MAPE_%': mp})
211     pred_sets.append((name.split('_')[0], df.val_loss.to_numpy(
212         float), ph))
213
214 val_df = pd.DataFrame(validation)
215 save_csv_safe(val_df, '广义标度律分层验证.csv')
216 print('\n=== 4. 广义标度律分层验证 ===')
217 print(val_df.to_string(index=False))
218
219 save_csv_safe(large_loss.assign(

```

```

207     pred=fn_best(r1x, large_loss.N_params_B.values, large_loss
.D_tokens_B.values,
208                 np.ones(len(large_loss))), '百亿参数以上外推
.csv')
209
210 # ===== 5. 弹性、边际效用与替代关系 =====
211 def marg(p, N, D, Q):
212     E_, A_, a_, B_, b_, C_, g_ = p
213     return (-a_ * A_ * N ** (-a_ - 1), -b_ * B_ * D ** (-b_ -
1),
214            -g_ * C_ * np.clip(1 - Q, 1e-9, None) ** (g_ - 1))
215
216 el_rows = []
217 for N_ref, D_ref, Q_ref in [(1., 100., 0.6), (12., 300., 0.6)
]:
218     L_ref = float(fn_best(r1x, N_ref, D_ref, Q_ref))
219     dN, dD, dQ = marg(r1x, N_ref, D_ref, Q_ref)
220     sub = abs(dQ) / abs(dN)
221     el_rows.append({'参考点N(B)': N_ref, 'D(B)': D_ref, 'Q':
Q_ref, 'L': L_ref,
222                   '弹性e_N': dN * N_ref / L_ref, '弹性e_D':
dD * D_ref / L_ref,
223                   '弹性e_Q': dQ * Q_ref / L_ref, '替代率dN/
dQ(B)': sub,
224                   'dQ=0.1等价dN(B)': sub * 0.1})
225 el_df = pd.DataFrame(el_rows)
226 save_csv_safe(el_df, '弹性与替代关系.csv')
227 print('\n=== 5. 弹性与质量—参数替代关系 ===')
228 print(el_df.round(4).to_string(index=False))
229
230 N_ref, D_ref, Q_ref = 1., 100., 0.6
231 dN, dD, dQ = marg(r1x, N_ref, D_ref, Q_ref)
232 subst = abs(dQ) / abs(dN)
233 save_csv_safe(pd.DataFrame({'质量提升dQ': [.05, .1, .2, .3],
234                             '局部等价参数增量dN(B)': subst *
np.array([.05, .1, .2, .3]),
235                             '等价数据增量dD(B)': abs(dQ) / abs
(dD) * np.array([.05, .1, .2, .3])})),

```

```

236         '质量参数替代关系.csv')
237
238 Qs = np.linspace(0.30, 0.99, 80)
239 subs = np.array([abs(marg(r1x, N_ref, D_ref, q)[2]) / abs(marg
240         (r1x, N_ref, D_ref, q)[0])
241         for q in Qs])
242 save_csv_safe(pd.DataFrame({'Q': Qs, 'dN_per_dQ(B)': subs,
243         'dN_for_dQ_0.1(B)': subs * 0.1}),
244         '替代率随质量变化.csv')
245
246 save_csv_safe(pd.DataFrame({'Q': Qs, '不可约损失下限': E + Cq
247         * (1 - Qs) ** gamma}),
248         '质量与损失下限.csv')
249
250 # ===== 6. 领域间的替代/互补关系（基于问题一系数向量）
251 # =====
252 p1_dir = os.path.join(SOLVE_DIR, '问题一', '结果')
253 coef = pd.read_csv(os.path.join(p1_dir, '混合系数矩阵.csv')).
254     set_index('training_domain')
255 inter = pd.read_csv(os.path.join(p1_dir, '截距.csv')).
256     set_index('loss_domain').intercept
257 mix = pd.read_csv(DATA('train_mixture_lm.csv'))
258 mix_cols = [c for c in mix if c.startswith('train_the_pile_')]
259 domains = [c.replace('train_the_pile_', '') for c in mix_cols]
260 p_ref = mix[mix_cols].mean().to_numpy(float)
261 loss_domains = list(coef.columns)
262 w_ref = np.array([p_ref[domains.index(d)] for d in
263     loss_domains]); w_ref /= w_ref.sum()
264
265 def mix_obj(p):
266     return float(w_ref @ (np.asarray(p) @ coef.loc[domains].
267         to_numpy()
268         + inter.loc[loss_domains].to_numpy()
269     ))
270
271 ref_obj = mix_obj(p_ref)
272 rec = pd.read_csv(os.path.join(p1_dir, '推荐配比调整.csv')).
273     set_index('domain').loc[domains]
274 p_rec = rec.recommended_mixture.to_numpy(float)

```

```

264 mix_delta_rec = mix_obj(p_rec) - ref_obj
265 save_csv_safe(pd.DataFrame([{'mixture': '参考配比', '
    predicted_weighted_loss': ref_obj, 'M_p': 0.},
266                               {'mixture': '问题一有界推荐', '
    predicted_weighted_loss': mix_obj(p_rec),
267                               'M_p': mix_delta_rec}]), '配比项
    校准.csv')
268
269 Cn = coef.loc[domains].to_numpy()
270 Cc = Cn - Cn.mean(axis=0, keepdims=True)
271 nm_ = np.linalg.norm(Cc, axis=1, keepdims=True)
272 sim = (Cc @ Cc.T) / np.maximum(nm_ @ nm_.T, 1e-12)
273 pd.DataFrame(sim, index=domains, columns=domains).to_csv(
274     os.path.join(OUT_DIR, '领域替代互补矩阵.csv'), encoding='
    utf-8-sig')
275 tri = np.triu_indices(len(domains), 1)
276 pair_sim = pd.DataFrame({'domain_i': [domains[i] for i in tri
    [0]],
277                          'domain_j': [domains[j] for j in tri
    [1]],
278                          'coefficient_similarity': sim[tri]}).
    sort_values(
279     'coefficient_similarity', ascending=False)
280 save_csv_safe(pd.concat([pair_sim.head(8).assign(关系='可替代'
    ),
281                          pair_sim.tail(8).assign(关系='互补或
    冲突')]), '领域替代互补_top.csv')
282 print('\n=== 6. 领域替代与互补（系数向量余弦相似度）===')
283 print('最可替代的 4 组: '); print(pair_sim.head(4).to_string(
    index=False))
284 print('最互补的 4 组: '); print(pair_sim.tail(4).to_string(
    index=False))
285 print('推荐配比带来的加权平均损失变化  $M(p) = %.4f$  %
    mix_delta_rec)
286
287 # ===== 7. B3 插值轨迹验证 =====
288 traj_rows = []
289 for f in traj_files:

```

```

290     d = pd.read_csv(f)
291     n_par = float(d.N_params_B.iloc[0])
292     base = E0 + A0 * n_par ** (-a0)
293     y = np.clip(d.val_loss.values - base, 1e-9, None)
294     sl = np.polyfit(np.log(d.D_tokens_B.values), np.log(y), 1)
295     traj_rows.append({'file': os.path.basename(f), 'N(B)':
296         n_par,
297         '轨迹内D幂指数': float(sl), '经典beta':
298         float(-b0)})
299     traj_df = pd.DataFrame(traj_rows)
300     save_csv_safe(traj_df, 'B3轨迹验证.csv')
301     print('\n=== 7. B3 插值轨迹验证 (轨迹内损失对 D 的幂指数 vs 经
302         典 beta) ===')
303     print(traj_df.round(4).to_string(index=False))
304
305     # ===== 8. 汇总 =====
306     save_csv_safe(pd.DataFrame({
307         '指标': ['E', 'A', 'alpha', 'B', 'beta', 'C', 'gamma', '经
308             典R2', '经典残差标准差',
309             '留出R2', '广义R2', '广义RMSE', '广义MAPE%', 'B6
310             方向相关r', 'B7方向相关r', 'B8方向相关r',
311             '参考点弹性e_N', '参考点弹性e_D', '参考点弹性e_Q'
312             , '替代率dN/dQ', '推荐配比M(p)'],
313         '数值': [E, A, alpha, B, beta, Cq, gamma, R2_cls, float(np
314             .std(L_p - Lph)),
315             hold['留出R2'], R2_gen, RMSE_gen, MAPE_gen, corr6
316             , corr7, corr8,
317             el_rows[0]['弹性e_N'], el_rows[0]['弹性e_D'],
318             el_rows[0]['弹性e_Q'],
319             subst, mix_delta_rec]}), '问题二_拟合汇总.csv')
320
321     # ===== 9. 绘图 =====
322     from 重绘图表 import render_all
323     render_all()
324
325     print('\n===== 问题二求解完成 =====')
326     print('经典 R2=%.8f, 留出 R2=%.8f; 广义 R2=%.4f; 替代率 dN/dQ

```

```

318     =%.3f B/单位质量'
        % (R2_cls, hold['留出R2'], R2_gen, subst))

```

Listing 2: 问题二求解代码（经典与广义标度律拟合、方向审计、弹性与质量—参数替代分析）

问题三：算力约束联合优化（问题三.py）

```

1  # -*- coding: utf-8 -*-
2  """
3  问题三：算力约束下的多维资源联合优化与结构性转移
4  输入：问题二广义标度律参数  $L(N,D,Q)=E+A/N^a+B/D^b+C(1-Q)^g$ ；
5      问题一的质量基线  $Q_0$  与配比修正  $M(p)$ ；附件 C7 的上下文长
      度可行取值。
6  成本（严格按赛题附录 B 与正文公式）：
7       $C_{total} = C_{train} + C_Q + C_{attn}$ 
8       $= 6ND + D * [g(Q) - g(Q_0)]_+ + \eta * N * D * L_{ctx}$ ,  $\eta = 2e-4$ 
9       $g(Q)$  取附录 B 三型（指数型  $1e7 * \exp(6Q)$ 、幂函数型  $5e9 * Q^4$ 、
      对数渐进型  $2e9 * \ln(1+10Q)$ ）
10 临界上下文长度：注意力开销 = 基础训练开销  $\Leftrightarrow \eta * N * D * L = 6 * N * D \Leftrightarrow L_{crit} = 6/\eta = 30000$ 
11 方法：三档预算联合优化（SLSQP，对数参数化）；预算扫略 + 结构性
      转移识别； $L_{ctx}$  敏感性。
12 输出：图片到 图片/，结果 CSV 到 结果/。
13  """
14  import sys, os, json
15  sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
16  from _common import * # noqa: F401,F403
17  from scipy.optimize import minimize
18
19  DATA = data_path
20
21  def save_csv_safe(df, name_cn):
22      p = os.path.join(OUT_DIR, name_cn)
23      try:
24          df.to_csv(p, index=False, encoding='utf-8-sig')

```

```

25     except PermissionError:
26         alt = p.replace('.csv', '_new.csv')
27         df.to_csv(alt, index=False, encoding='utf-8-sig')
28         print('    ! %s 被占用, 已改存为 %s' % (name_cn, os.path
        .basename(alt)))
29
30 # ===== 0. 读取前两问输出 =====
31 params = pd.read_csv(shared_path('广义标度律参数.csv'),
        index_col='param')['value']
32 E, A, alpha, B, beta, Cq, gamma = (params['E'], params['A'],
        params['alpha'],
33                                     params['B'], params['beta']
        ], params['C'], params['gamma'])
34
35 with open(shared_path('问题一_关键量.json'), encoding='utf-8')
        as fh:
36     k1 = json.load(fh)
37 Q0_sample = float(k1['Q0_global_mean'])
38
39 mix_map = pd.read_csv(shared_path('17域质量映射.csv'))
40 ref_mix = pd.read_csv(data_path('train_mixture_lm.csv'))
41 ref_cols = [c for c in ref_mix if c.startswith('
        train_the_pile_')]
42 ref_p = ref_mix[ref_cols].mean().to_numpy(float)
43 ref_dom = [c.replace('train_the_pile_', '') for c in ref_cols]
44 qmap = mix_map.set_index('mixture_domain')['quality_Q']
45 q_vec = np.array([float(qmap.get(d, Q0_sample)) for d in
        ref_dom])
46 Q0 = float(ref_p @ q_vec)          # 基线质量 = 参考语料配比加权
        平均质量
47 print('=== 0. 质量基线与模型参数 (来自问题一、二) ===')
48 print('    参考语料配比加权基线质量 Q0 = %.4f (全量样本均值 %.4f
        ) ' % (Q0, Q0_sample))
49 print('    标度律: E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f C
        =%.4f gamma=%.4f'
50         % (E, A, alpha, B, beta, Cq, gamma))
51
52 def loss(N_abs, D_abs, Q, M_p=0.0):

```

```

53     """验证损失：参数以 B 为单位代入问题二标度律，M_p 为配比项
    修正。"""
54     return (E + A * (N_abs / 1e9) ** (-alpha) + B * (D_abs / 1
    e9) ** (-beta)
55             + Cq * (1 - Q) ** gamma + M_p)
56
57 # ===== 1. 附录 B 的三型质量成本函数 =====
58 GB = {
59     '指数型': dict(gamma=1e7, lam=6.0),
60     '幂函数型': dict(gamma=5e9, lam=4.0),
61     '对数渐进型': dict(gamma=2e9, lam=10.0),
62 }
63 def g_of(Q, form):
64     par = GB[form]
65     if form == '指数型':
66         return par['gamma'] * np.exp(par['lam'] * Q)
67     if form == '幂函数型':
68         return par['gamma'] * Q ** par['lam']
69     return par['gamma'] * np.log(1 + par['lam'] * Q)
70
71 def c_quality(D_abs, Q, form):
72     return D_abs * max(g_of(Q, form) - g_of(Q0, form), 0.0)
73
74 for form in GB:
75     print(' g(%) 在 Q0=%.3f 处为 %.4e, 在 Q=1 处为 %.4e (单
    位算力/token) '
76           % (form, Q0, g_of(Q0, form), g_of(1.0, form)))
77
78 # ===== 2. 临界上下文长度 =====
79 ETA = 2e-4
80 L_CRIT = 6.0 / ETA
81 arch = pd.read_csv(DATA('model_architecture_metadata.csv'))
82 L_FEAS = sorted(arch['max_position_embeddings'].dropna().
    unique().astype(int).tolist())
83 print('\n=== 1. 临界上下文长度 (解析解) ===')
84 print(' C_attn = eta*N*D*L_ctx, C_train = 6ND -> L_crit = 6/
    eta = %.0f tokens' % L_CRIT)
85 print(' C7 给出的可行上下文长度 (max_position_embeddings) : %

```

```

    s' % L_FEAS)
86 print(' 可行取值中跨越临界值的一对: 8192 (0.2f 倍) 与 32768 (
    0.2f 倍) '
87         % (ETA * 8192 / 6, ETA * 32768 / 6))
88
89 # ===== 3. 总成本与单点最优 =====
90 def total_cost(N_abs, D_abs, Q, L_ctx, form):
91     return 6.0 * N_abs * D_abs + c_quality(D_abs, Q, form) +
        ETA * N_abs * D_abs * L_ctx
92
93 def optimize(C_budget, L_ctx=2048, form='幂函数型', M_p=0.0,
    n_starts=8, Qcap=1.0):
94     """SLSQP 对数参数化联合优化 (N, D, Q)。"""
95     def obj(x):
96         return loss(np.exp(x[0]), np.exp(x[1]), x[2], M_p)
97     def con(x):
98         return C_budget - total_cost(np.exp(x[0]), np.exp(x
100 [1]), x[2], L_ctx, form)
101     best = None
102     for k in range(n_starts):
103         frac = (k + 1) / (n_starts + 1)
104         x0 = [np.log(1e9) + 4 * (frac - .5), np.log(1e11) + 4
105 * (.5 - frac),
106             Q0 + (Qcap - Q0) * frac]
107         try:
108             res = minimize(obj, x0, method='SLSQP',
109                             bounds=[(np.log(1e5), np.log(1e15))
110 ,
111                                     (np.log(1e6), np.log(1e16))
112 , (Q0, Qcap)],
113                             constraints={'type': 'ineq', 'fun':
114 con},
115                             options={'maxiter': 1500, 'ftol': 1
116 e-14})
117         except Exception:
118             continue
119         if res.success and (best is None or res.fun < best.fun
120 ):

```

```

113         best = res
114     if best is None:
115         raise RuntimeError('优化失败')
116     N_abs, D_abs, Q = np.exp(best.x[0]), np.exp(best.x[1]),
best.x[2]
117     c_base = 6.0 * N_abs * D_abs
118     c_qual = c_quality(D_abs, Q, form)
119     c_attn = ETA * N_abs * D_abs * L_ctx
120     return dict(N_B=N_abs / 1e9, D_B=D_abs / 1e9, D_over_N=
D_abs / N_abs, Q=Q,
121                 L=loss(N_abs, D_abs, Q, M_p), c_base=c_base,
c_qual=c_qual,
122                 c_attn=c_attn, c_tot=c_base + c_qual + c_attn,
123                 f_base=c_base / C_budget, f_qual=c_qual /
C_budget,
124                 f_attn=c_attn / C_budget, form=form, L_ctx=
L_ctx, C=C_budget)
125
126 BUDGETS = [1e19, 1e22, 1e24]
127 print('\n=== 2. 三档预算最优配置（幂函数型质量成本, L_ctx=2048
) ===')
128 rows = []
129 for Cb in BUDGETS:
130     r = optimize(Cb, 2048, '幂函数型')
131     rows.append(r)
132     print('  C=%.0e: N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f  '
133           '[base %.1f%% qual %.1f%% attn %.1f%%]'
134           % (Cb, r['N_B'], r['D_B'], r['Q'], r['L'],
135              r['f_base'] * 100, r['f_qual'] * 100, r['f_attn']
* 100))
136 save_csv_safe(pd.DataFrame(rows), '三档预算最优配置.csv')
137
138 # ===== 4. 三种质量成本函数对比 =====
139 print('\n=== 3. 质量成本函数对最优解的影响 ===')
140 comp = []
141 for Cb in BUDGETS:
142     for form in GB:
143         r = optimize(Cb, 2048, form)

```

```

144         comp.append(r)
145 comp_df = pd.DataFrame(comp)
146 save_csv_safe(comp_df, '成本函数对比.csv')
147 for Cb in BUDGETS:
148     sub = comp_df[comp_df.C.eq(Cb)]
149     print('  C=%.0e' % Cb)
150     for _, r in sub.iterrows():
151         print('      %-6s N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f 质量
投入占比 %.1f%%'
152               % (r['form'], r['N_B'], r['D_B'], r['Q'], r['L'
], r['f_qual'] * 100))
153
154 # ===== 4b. 质量基线 Q0 口径的灵敏度 =====
155 _Q0_main = Q0
156 Q0 = Q0_sample
157 print('\n=== 3b. 质量基线口径灵敏度 (Q0 由配比加权 %.4f 改为样
本均值 %.4f) ==='
158       % (_Q0_main, Q0_sample))
159 q0_rows = []
160 for Cb in BUDGETS:
161     r = optimize(Cb, 2048, '幂函数型')
162     q0_rows.append(r)
163     print('  C=%.0e: N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f 质量投
入占比 %.1f%%'
164           % (Cb, r['N_B'], r['D_B'], r['Q'], r['L'], r['f_qual
'] * 100))
165 save_csv_safe(pd.DataFrame(q0_rows), '质量基线灵敏度.csv')
166 Q0 = _Q0_main
167
168 # ===== 5. 领域配比 M(p) 的算力等价效果 =====
169 mix_cal = pd.read_csv(os.path.join(SOLVE_DIR, '问题二', '结果'
, '配比项校准.csv'))
170 M_p = float(mix_cal.loc[mix_cal.mixture.eq('问题一有界推荐'),
'M_p'].iloc[0])
171 print('\n=== 4. 领域配比修正的算力等价效果 (M(p)=%.4f) ===' %
M_p)
172 r_mix = optimize(1e22, 2048, '幂函数型', M_p=M_p)
173 r_ref = optimize(1e22, 2048, '幂函数型', M_p=0.0)

```

```

174 print('    C=1e22: 参考配比 L*=% .4f (N*=% .3fB D*=% .1fB Q*=% .4f)
      ,
175         % (r_ref['L'], r_ref['N_B'], r_ref['D_B'], r_ref['Q']))
176 print('        推荐配比 L*=% .4f (N*=% .3fB D*=% .1fB Q*=% .4f)
      ,
177         % (r_mix['L'], r_mix['N_B'], r_mix['D_B'], r_mix['Q']))
178
179 def budget_for_loss(L_target, L_ctx=2048, form='幂函数型'):
180     lo, hi = 1e17, 1e25
181     for _ in range(60):
182         mid = np.sqrt(lo * hi)
183         try:
184             r = optimize(mid, L_ctx, form, n_starts=3)
185             if r['L'] > L_target:
186                 lo = mid
187             else:
188                 hi = mid
189         except Exception:
190             lo = mid
191     return np.sqrt(lo * hi)
192
193 C_eq = budget_for_loss(r_mix['L'])
194 print('    推荐配比在 C=1e22 时得到的损失 %.4f, 若改用参考配比实
      现同一损失需要'
195       ' C≈%.3e FLOPs, 即配比优化的效果等价于约 %.1f 倍算力扩张
      ,
196       % (r_mix['L'], C_eq, C_eq / 1e22))
197 print('    (说明: M(p) 是损失水平上的固定平移, 与算力无关, 故换
      算为算力收益会被'
198       '损失的算力弹性放大, 此倍数宜作量级参考。) ')
199 save_csv_safe(pd.DataFrame([{'mixture': '参考配比', 'M_p':
      0.0, **r_ref},
200                             {'mixture': '问题一有界推荐', 'M_p'
      ': M_p, **r_mix}]),
201               '配比项算力等价.csv')
202
203 # ===== 6. 预算扫略与结构性转移识别 =====
204 print('\n=== 5. 预算扫略与结构性转移 ===')

```

```

205 C_sweep = np.logspace(18, 25, 141)
206 sweep = []
207 for Cb in C_sweep:
208     try:
209         sweep.append(optimize(Cb, 2048, '幂函数型', n_starts
210                               =5))
211     except Exception:
212         sweep.append(dict(N_B=np.nan, D_B=np.nan, Q=np.nan, L=
213                           np.nan, c_base=np.nan,
214                           c_qual=np.nan, c_attn=np.nan, c_tot=
215                           np.nan, f_base=np.nan,
216                           f_qual=np.nan, f_attn=np.nan, form='
217                           幂函数型', L_ctx=2048, C=Cb))
218 sw = pd.DataFrame(sweep)
219 save_csv_safe(sw, '预算扫略.csv')
220
221 Qc = sw['Q'].to_numpy(float)
222 logC = np.log10(C_sweep)
223 ok = np.isfinite(Qc)
224 # 结构性转移的定量定义:
225 # 以“质量投入份额 f_qual 的相对变化方向反转 + 最优质量对预
226 # 算的弹性改变符号/量级”为标志。
227 inQ = (Qc - Q0) / (1 - Q0) # 质量提升完成
228 度
229 d_inQ = np.gradient(inQ[ok], logC[ok])
230 fQ = sw['f_qual'].to_numpy(float)
231 d_fQ = np.gradient(fQ[ok], logC[ok])
232 shift = {'质量提升完成度弹性 dln(Theta)/dlnC 的最大值点预算':
233         float(C_sweep[ok][np.argmax(d_inQ)]),
234         '质量投入启动边界（份额由零转正的拐点预算）': float(
235             C_sweep[ok][np.argmax(d_fQ)]),
236         '质量投入份额峰值预算': float(C_sweep[ok][np.argmax(
237             fQ[ok])]),
238         '质量投入份额峰值': float(np.nanmax(fQ[ok]))}
239 # 以“质量提升完成度”  $\Theta = (Q^* - Q_0) / (1 - Q_0)$  报告门槛，避免 Q0 本
240 # 身已高于固定阈值时失去意义
241 for frac in [0.25, 0.5, 0.75, 0.99]:
242     hit = Qc[ok] >= Q0 + frac * (1 - Q0)

```

```

233     shift['完成度Theta首次达到%d%%的预算' % int(frac * 100)] =
        (
234         float(C_sweep[ok][np.argmax(hit)]) if hit.any() else
        float('nan'))
235 for k, v in shift.items():
236     print(' %s = %.3e FLOPs' % (k, v))
237
238 # N/D 比随预算的演化（规模结构）
239 sw['D_over_N'] = sw['D_B'] / sw['N_B']
240 print(' D/N 比: C=1e19 时 %.1f, C=1e22 时 %.1f, C=1e24 时 %.1
        f'
241         % tuple(sw.loc[(sw.C - c).abs().idxmin(), 'D_over_N']
        for c in [1e19, 1e22, 1e24]))
242 save_csv_safe(pd.DataFrame([shift]), '结构性转移识别.csv')
243
244 # ===== 7. 上下文长度敏感性 (C7 可行取值 + 临界值)
        =====
245 print('\n=== 6. 上下文长度敏感性 (C=1e22, 幂函数型) ===')
246 sens = []
247 for L in L_FEAS:
248     r = optimize(1e22, L, '幂函数型')
249     r['attn_over_train'] = ETA * L / 6.0
250     r['is_above_crit'] = L > L_CRIT
251     sens.append(r)
252     print(' L=%6d (%.2f 倍训练开销) : N*=%.3fB D*=%.1fB Q
        *=%.4f L*=%.4f 注意力占比 %.1f%%s'
253           % (L, r['attn_over_train'], r['N_B'], r['D_B'], r['Q
        '], r['L'],
254             r['f_attn'] * 100, ' [超临界]' if L > L_CRIT
        else ''))
255 sens_df = pd.DataFrame(sens)
256 save_csv_safe(sens_df, '上下文长度敏感性.csv')
257
258 # 连续 L 扫描（含临界点）
259 L_grid = np.logspace(np.log10(1024), np.log10(200000), 40)
260 cont = []
261 for L in L_grid:
262     try:

```

```

263         r = optimize(1e22, float(L), '幂函数型', n_starts=3)
264         r['attn_over_train'] = ETA * L / 6.0
265         cont.append(r)
266     except Exception:
267         pass
268 save_csv_safe(pd.DataFrame(cont), '上下文长度连续扫描.csv')
269
270 # ===== 8. 绘图 =====
271 plt.rcParams['font.size'] = 12
272 C1_, C2_, C3_ = '#3b6ea5', '#e07a3f', '#5aa05a'
273
274 opt_df = pd.DataFrame(rows)
275 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))
276 x = np.arange(len(BUDGETS))
277 axes[0].plot(x, opt_df['N_B'], 'o-', lw=2, ms=8, color=C1_,
278             label='参数量 N* (B)')
279 axes[0].plot(x, opt_df['D_B'], 's--', lw=2, ms=8, color=C2_,
280             label='数据量 D* (B tokens)')
281 for xi, (nb, db) in enumerate(zip(opt_df['N_B'], opt_df['D_B']
282                                 )):
283     axes[0].annotate('%.3g' % nb, (xi, nb), fontsize=8, xytext
284                     =(0, 8),
285                     textcoords='offset points', ha='center',
286                     color=C1_)
287     axes[0].annotate('%.4g' % db, (xi, db), fontsize=8, xytext
288                     =(0, -15),
289                     textcoords='offset points', ha='center',
290                     color=C2_)
291 axes[0].set_yscale('log'); axes[0].set_xticks(x)
292 axes[0].set_xticklabels(['1019', '1022', '1024'])
293 axes[0].set_xlabel('算力预算 C (FLOPs)'); axes[0].set_ylabel('
    最优规模（对数尺度）')
294 axes[0].set_title('(a) 最优参数量与数据量'); axes[0].legend(
295     fontsize=9); despine(axes[0])
296 axes[1].plot(x, opt_df['Q'], 'o-', lw=2, ms=8, color=C3_,
297             label='最优质量 Q*')
298 axes[1].axhline(Q0, color='gray', ls=':', lw=1.2, label='基线
    质量 Q0=%.3f' % Q0)

```

```

290 for xi, qv in enumerate(opt_df['Q']):
291     axes[1].annotate('%0.3f' % qv, (xi, qv), fontsize=9, xytext
292         =(0, 8),
293         textcoords='offset points', ha='center',
294         color=C3_)
295 axes[1].set_xticks(x); axes[1].set_xticklabels(['1019', '1022',
296         '1024'])
297 axes[1].set_ylim(0, 1.12); axes[1].set_xlabel('算力预算 C (
298     FLOPs)')
299 axes[1].set_ylabel('最优质量 Q*'); axes[1].set_title('(b) 最优
300     质量随预算的变化')
301 axes[1].legend(fontsize=9); despine(axes[1])
302 fig.tight_layout(); save_fig(fig, '图1_三档预算最优配置')
303
304 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
305 axes[0].plot(C_sweep, Qc, lw=2, color=C3_)
306 axes[0].axhline(Q0, color='gray', ls=':', lw=1.2, label='基线
307     质量 Q0=%0.3f' % Q0)
308 axes[0].axvline(shift['质量投入启动边界（份额由零转正的拐点预
309     算）'], color='#d62728', ls='--', lw=1.2,
310     label='投入份额拐点')
311 axes[0].set_xscale('log'); axes[0].set_xlabel('算力预算 C (
312     FLOPs)')
313 axes[0].set_ylabel('最优质量 Q*'); axes[0].set_ylim(0, 1.05)
314 axes[0].set_title('(a) 最优质量随预算的演化'); axes[0].legend(
315     fontsize=8); despine(axes[0])
316 axes[1].plot(C_sweep, sw['f_base'] * 100, lw=2, color=C1_,
317     label='基础训练 6ND')
318 axes[1].plot(C_sweep, sw['f_qual'] * 100, lw=2, color=C2_,
319     label='质量提升 D[g(Q)-g(Q0)]')
320 axes[1].plot(C_sweep, sw['f_attn'] * 100, lw=2, color='#d62728',
321     label='长文本注意力')
322 axes[1].set_xscale('log'); axes[1].set_xlabel('算力预算 C (
323     FLOPs)')
324 axes[1].set_ylabel('预算份额 (%)'); axes[1].set_title('(b) 预
325     算份额的结构性演化')
326 axes[1].legend(fontsize=8); despine(axes[1])
327 fig.tight_layout(); save_fig(fig, '图2_预算份额结构性转移')

```

```

314
315 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
316 axes[0].plot([r['attn_over_train'] for r in cont], [r['N_B']
    for r in cont], 'o-', ms=3, color=C1_)
317 axes[0].set_xscale('log'); axes[0].set_yscale('log')
318 axes[0].axvline(1.0, color='#d62728', ls='--', lw=1.2,
319                 label='临界  $L* = \eta L / 6$  (注意力=训练)' % L_CRIT)
320 axes[0].set_xlabel('注意力开销 / 训练开销 =  $\eta L / 6$ '); axes[0].
    set_ylabel('N* (B)')
321 axes[0].set_title('(a) 上下文长度对最优规模的影响'); axes[0].
    legend(fontsize=8); despine(axes[0])
322 # 右图: 不同预算档下  $Q*$  随注意力开销倍数的变化 (低预算档才有内
    点解, 故三档并列)
323 L_small = np.logspace(np.log10(1024), np.log10(150000), 14)
324 _qL_rows = []
325 for Cb, col, lbl in [(1e19, C3_, '1019'), (1e20, C2_, '1020'),
    (1e21, C1_, '1021')]:
326     xs, ys = [], []
327     for L in L_small:
328         try:
329             rr = optimize(Cb, float(L), '幂函数型', n_starts
    =3)
330             xs.append(ETA * L / 6.0); ys.append(rr['Q'])
331             _qL_rows.append({'C': Cb, 'L_ctx': float(L), '
    attn_over_train': ETA * L / 6.0,
332                             'N_B': rr['N_B'], 'D_B': rr['D_B']
    }, {'Q': rr['Q'], 'L': rr['L']})
333         except Exception:
334             pass
335     axes[1].plot(xs, ys, 'o-', ms=3, color=col, label='预算 %s
    FLOPs' % lbl)
336 save_csv_safe(pd.DataFrame(_qL_rows), '上下文长度与最优质量.
    csv')
337 axes[1].axvline(1.0, color='#d62728', ls='--', lw=1.2, label='
    临界  $L*$ ')
338 axes[1].set_xscale('log'); axes[1].set_xlabel('注意力开销 / 训
    练开销 =  $\eta L / 6$ ')
339 axes[1].set_ylabel('Q*'); axes[1].set_ylim(0, 1.05)

```

```

340 axes[1].yaxis.get_major_formatter().set_useOffset(False)
341 axes[1].set_title('(b) 上下文长度对质量投入的影响')
342 axes[1].legend(fontsize=8); despine(axes[1])
343 fig.tight_layout(); save_fig(fig, '图3_上下文长度敏感性')
344
345 fig, ax = plt.subplots(figsize=(7, 5))
346 Q_g = np.linspace(Q0, 1.0, 300)
347 for form, lbl in [('幂函数型', '幂函数型  $5 \times 10^9 Q^4$ '),
348                  ('指数型', '指数型  $10^7 e^{(6Q)}$ '),
349                  ('对数渐进型', '对数渐进型  $2 \times 10^9 \ln(1+10Q)$ ')
350                  ]:
351     ax.plot(Q_g, [g_of(q, form) - g_of(Q0, form) for q in Q_g], lw=2, label=lbl)
352 ax.set_xlabel('数据质量 Q'); ax.set_ylabel('单位 token 质量成本 g(Q)-g(Q0)')
353 ax.set_title('附录 B 三种质量成本函数 (Q0=%.3f)' % Q0)
354 ax.legend(fontsize=9); despine(ax); fig.tight_layout();
355     save_fig(fig, '图4_质量成本函数对比')
356
357 # ===== 9. 汇总 =====
358 print('\n===== 问题三求解完成 =====')
359 print('  临界上下文长度 L_crit = 6/eta = %.0f tokens; C7 可行取值 %s' % (L_CRIT, L_FEAS))
360 print('  三档预算（幂函数型）： ' +
361       '; '.join('C=%.0e -> N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f' % (r['C'], r['N_B'], r['D_B'], r['Q'], r['L'])
362               for r in rows))

```

Listing 3: 问题三求解代码（赛题成本模型、临界上下文长度、预算约束联合优化与结构性转移识别）

问题四：技术演进与前沿预测（问题四.py）

```

1  # -*- coding: utf-8 -*-
2  """
3  问题四：技术演进分析与前沿预测
4  数据：C1/C2 排行榜、C3 时序、C4 Epoch 宏观元数据、C5/C6 Loss-
        Benchmark 桥接、C7 架构、C8 逐任务 JSON

```

```

5  流程：1) 明确口径：时间轴(提交日期)、开源口径(开源权重+许可允许研究复现)、模型类型(pretrained/chat)
6      2) C8 逐任务聚合（必做），并与 C1 六维交叉校验
7      3) C6 桥接（按可比性分层）+ C5 对照，建立 Loss -> Benchmark 映射
8      4) 面板回归（含年份固定效应）分离规模扩张与非规模技术进步贡献
9      5) 情景设定下 logistic 饱和外推 12/24 个月能力前沿 + 残差 bootstrap 不确定性
10     6) C4 宏观算力/数据量/开源权重证据，支撑"算力增长放缓"情景
11 输出：图片到 图片/，结果 CSV 到 结果/
12  """
13  import sys, os, json, glob, re, collections
14  sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
15  from _common import *    # noqa: F401,F403
16  from scipy.optimize import curve_fit
17
18  DATA = data_path
19  C_DIR = os.path.join(DATA_DIR, 'C_efficiency_evolution')
20
21  def save_csv_safe(df, name_cn):
22      p = os.path.join(OUT_DIR, name_cn)
23      try:
24          df.to_csv(p, index=False, encoding='utf-8-sig')
25      except PermissionError:
26          alt = p.replace('.csv', '_new.csv')
27          df.to_csv(alt, index=False, encoding='utf-8-sig')
28          print('    ! %s 被占用，已改存为 %s' % (name_cn, os.path.basename(alt)))
29
30  TASKS = ['IFEval', 'BBH', 'MATH Lvl 5', 'GPQA', 'MUSR', 'MMLU-PRO']
31
32  # ===== 1. C1 排行榜：口径定义与筛选 =====
33  lb = pd.read_csv(os.path.join(C_DIR, 'leaderboard_cleaned.csv'))

```

```

34 lb['P'] = pd.to_numeric(lb['#Params (B)'], errors='coerce')
35 lb['date'] = pd.to_datetime(lb['Submission Date'], errors='
    coerce')
36 lb['Year'] = lb['date'].dt.year
37 lb['avg'] = pd.to_numeric(lb['Average ↑'], errors='coerce')
38 for t in TASKS:
39     lb[t] = pd.to_numeric(lb[t], errors='coerce')
40
41 TYPE_PRE = ['□ pretrained', '□ continuously pretrained']
42 TYPE_CHAT = ['□ chat models (RLHF, DPO, IFT, ...)',
43             '□ fine-tuned on domain-specific datasets']
44 TYPE_EXCL = ['□ base merges and moerges', '□ multimodal', '□
    other']
45
46 PERMISSIVE = {'apache-2.0', 'mit', 'gpl-3.0', 'cc-by-4.0', '
    mpl-2.0', 'bsd-3-clause',
47             'llama2', 'llama3', 'llama3.1', 'llama3.2', '
    llama3.3', 'gemma',
48             'cc-by-sa-4.0', 'openrail'}
49
50 def license_ok(x):
51     """许可是否允许研究与复现（非商用/受限/未知一律排除）。"""
52     return str(x).strip().lower() in PERMISSIVE
53
54 lb['license_ok'] = lb['Hub License'].map(license_ok)
55 lb['type_group'] = np.where(lb['Type'].isin(TYPE_PRE), '
    pretrained',
56                             np.where(lb['Type'].isin(TYPE_CHAT
    ), 'chat/finetuned', '其他'))
57 lb['six_complete'] = lb[TASKS].notna().all(axis=1)
58
59 funnel = [{'层级': 'C1 全量记录', 'n': len(lb)},
60           {'层级': '六维得分完整', 'n': int(lb.six_complete.
    sum())}]
61 core = lb[lb.six_complete & lb.date.notna() & lb.P.notna()]
62 funnel.append({'层级': '时间/参数/六维齐备', 'n': len(core)})
63 core_o = core[core.license_ok]
64 funnel.append({'层级': '许可允许研究复现（开源口径）', 'n':

```

```

        len(core_o))}
65 core_pt = core_o[core_o.type_group.eq('pretrained')]
66 core_ch = core_o[core_o.type_group.eq('chat/finetuned')]
67 funnel.append({'层级': '    其中 pretrained', 'n': len(core_pt)
    })
68 funnel.append({'层级': '    其中 chat/finetuned', 'n': len(
    core_ch)})
69 save_csv_safe(pd.DataFrame(funnel), '筛选漏斗.csv')
70 print('=== 1. 数据口径与筛选漏斗 ===')
71 print(pd.DataFrame(funnel).to_string(index=False))
72 print('    时间轴口径：以 C1 的 Submission Date（提交日期）为基
    准。')
73 print('    开源口径：权重可获取（pretrained/持续预训练/chat/微
    调）且 Hub License 允许研究复现。')
74 print('    模型类型口径：pretrained 与 chat/finetuned 分组统计；
    base 合并模型（%d 条）、'
75         '多模态（%d 条）不并入主分析。'
76         % (int(lb['Type'].eq(TYPE_EXCL[0]).sum()), int(lb['Type'
    ].eq(TYPE_EXCL[1]).sum())))
77 print('    提交日期范围：%s ~ %s' % (str(lb.date.min())[:10],
    str(lb.date.max())[:10]))
78
79 # ===== 2. C8 逐任务聚合（必用） =====
80 def norm_name(s):
81     return re.sub(r'^a-z0-9', '', str(s).lower())
82
83 C8_DIR = os.path.join(C_DIR, 'detailed_results')
84 dirs = sorted(d for d in os.listdir(C8_DIR) if os.path.isdir(
    os.path.join(C8_DIR, d)))
85 METRIC_PREF = ['acc_norm,none', 'acc,none', '
    prompt_level_loose_acc,none',
86                 'inst_level_loose_acc,none', 'exact_match,none'
    ]
87 recs, broken = [], 0
88 for d in dirs:
89     fl = sorted(glob.glob(os.path.join(C8_DIR, d, '*.json')))
90     if not fl:
91         continue

```

```

92     ok = False
93     for f in reversed(fl):
94         try:
95             o = json.load(open(f, encoding='utf-8'))
96             res = o.get('results')
97             if not isinstance(res, dict) or not res:
98                 continue
99         except Exception:
100             broken += 1
101             continue
102         rec = {'dir': d, 'json_model_name': o.get('model_name'
103             , ''),
104             'file': os.path.basename(f)}
105         for k, v in res.items():
106             if not isinstance(v, dict):
107                 continue
108             for mk in METRIC_PREF:
109                 if isinstance(v.get(mk), (int, float)):
110                     rec[k] = float(v[mk]) * 100.0
111                     break
112             recs.append(rec)
113             ok = True
114             break
115         if not ok:
116             broken += 1
117
118     c8 = pd.DataFrame(recs)
119     print('\n=== 2. C8 逐任务聚合（必做） ===')
120     print('  模型目录 %d 个；成功解析 %d 个；跳过损坏/无有效记录 %d 个' % (len(dirs), len(c8), broken))
121     save_csv_safe(c8.fillna(''), 'C8逐任务解析结果.csv')
122
123     C8_MAP = {'IFEval': 'leaderboard_ifeval', 'BBH': '
124         leaderboard_bbh',
125         'MATH Lvl 5': 'leaderboard_math_hard', 'GPQA': '
126         leaderboard_gpqa',
127         'MUSR': 'leaderboard_musr', 'MMLU-PRO': '
128         leaderboard_mmlu_pro'}

```

```

125 for t, k in C8_MAP.items():
126     c8[t] = c8[k] if k in c8.columns else np.nan
127 c8['key'] = c8['dir'].map(norm_name)
128
129 lb['key'] = lb['Model'].map(norm_name)
130 core_o = core_o.assign(key=core_o['Model'].map(norm_name))
131 mrg = lb[['key'] + TASKS].merge(c8[['key'] + TASKS], on='key',
    suffixes=('_C1', '_C8'))
132 chk_rows = []
133 for t in TASKS:
134     a = pd.to_numeric(mrg[t + '_C1'], errors='coerce')
135     bq = pd.to_numeric(mrg[t + '_C8'], errors='coerce')
136     okm = a.notna() & bq.notna()
137     chk_rows.append({'任务': t, '匹配模型数': int(okm.sum()),
138                     'Pearson_r': float(np.corrcoef(a[okm], bq
139 [okm])[0, 1]) if okm.sum() > 2 else np.nan,
140                     'Spearman_rho': float(stats.spearmanr(a[
141 okm], bq[okm]).statistic)
142                     if okm.sum() > 2 else np.nan,
143                     '平均绝对偏差': float(np.mean(np.abs(a[
144 okm] - bq[okm]))) if okm.sum() else np.nan})
145 chk = pd.DataFrame(chk_rows)
146 save_csv_safe(chk, 'C8与C1一致性校验.csv')
147 print(' C8 与 C1 逐维一致性（规范化模型名匹配，命中 %d 个模
    型）：' % len(mrg))
148 print(chk.to_string(index=False))
149 print(' 口径说明：C8 存的是各任务的原始指标（acc/acc_norm/
    exact_match），'
150       'C1 存的是官方按随机基线归一化后的六维得分（例如 GPQA=(
    raw-0.25)/0.75、'
151       'MMLU-PRO=(raw-0.10)/0.90、IFEval=(prompt_strict+
    inst_strict)/2），'
152       '故两者存在系统性水平差但秩一致性很高。因此：跨模型得分
    分析用 C1，'
153       '逐任务结构分析在 C8 的原始指标空间内进行。')
154 # 逐任务（子任务）聚合：BBH 24 项、MATH 7 项、GPQA 3 项、MUSR
    3 项

```

```

153 SUB_PREFIX = {'BBH子任务': 'leaderboard_bbh_', 'MATH子任务': '
      leaderboard_math_',
154               'GPQA子集': 'leaderboard_gpqa_', 'MUSR子任务': '
      leaderboard_musr_' }
155 sub_cols = {}
156 for grp, pre in SUB_PREFIX.items():
157     cols = [c for c in c8.columns if c.startswith(pre) and c.
      count('_') >= 2]
158     cols = [c for c in cols if c not in ('leaderboard_bbh', '
      leaderboard_musr',
159                                           'leaderboard_gpqa', '
      leaderboard_math_hard')]
160     sub_cols[grp] = cols
161 print(' C8 子任务维度数: ' + ', '.join('%s %d 项' % (k, len(v
      )), for k, v in sub_cols.items()))
162
163 # 子任务前沿与剩余空间（限定在开源筛选后的模型集合上）
164 screened_keys = set(core_o['key'])
165 c8s = c8[c8.key.isin(screened_keys)]
166 sub_rows = []
167 for grp, cols in sub_cols.items():
168     for c in cols:
169         s = pd.to_numeric(c8s[c], errors='coerce').dropna()
170         if len(s) < 10:
171             continue
172         sub_rows.append({'类别': grp, '子任务': c.replace('
      leaderboard_', ''), '样本数': len(s),
173                           '最优得分': float(s.max()),
174                           '前5%均值': float(s.nlargest(max(1,
      int(len(s) * 0.05))).mean()),
175                           '中位数': float(s.median()),
176                           '剩余空间_%': float((100 - s.max()) /
      100 * 100)})
177 sub_df = pd.DataFrame(sub_rows)
178 save_csv_safe(sub_df, 'C8逐任务聚合.csv')
179 print('\n 子任务层面最弱 6 项（剩余空间最大）: ')
180 print(sub_df.nlargest(6, '剩余空间_%')[['类别', '子任务', '样
      本数', '最优得分', '前5%均值'],

```

```

181                                     '剩余空间_%']]].
    to_string(index=False))
182
183 CAT_ROWS = []
184 for grp, cols in sub_cols.items():
185     s = pd.to_numeric(c8s[cols].stack(), errors='coerce').
    dropna()
186     CAT_ROWS.append({'类别': grp, '维度数': len(cols), '平均最
    优得分': float(
187         np.mean([pd.to_numeric(c8s[c], errors='coerce').max()
    for c in cols])),
188         '整体前5%均值': float(s.quantile(0.95)), '整体中位数':
    float(s.median())})
189 save_csv_safe(pd.DataFrame(CAT_ROWS), 'C8子任务类别汇总.csv')
190 print('\n 子任务类别汇总: ')
191 print(pd.DataFrame(CAT_ROWS).to_string(index=False))
192
193 # ===== 3. Loss - Benchmark 桥接 (C6 主用, 按可比性分
    层; C5 对照) =====
194 br = pd.read_csv(os.path.join(C_DIR, '
    loss_benchmark_bridge_expanded.csv'))
195 br['cmp'] = np.where(br['Loss_Comparability'].str.startswith('
    High'), '高可比',
196                     '中可比')
197 print('\n=== 3. Loss-Benchmark 桥接 (C6 扩展桥接) ===')
198 bridge_rows, fits = [], {}
199 for lvl in ['高可比', '中可比', '全样本']:
200     d = br if lvl == '全样本' else br[br.cmp.eq(lvl)]
201     sl, ic = np.polyfit(d.Val_Loss, d.LB_Average, 1)
202     pred = ic + sl * d.Val_Loss
203     fits[lvl] = (sl, ic)
204     bridge_rows.append({'可比性': lvl, 'n': len(d),
205                         'Pearson_r': float(np.corrcoef(d.
    Val_Loss, d.LB_Average)[0, 1]),
206                         'Spearman_rho': float(stats.spearmanr(
    d.Val_Loss, d.LB_Average).statistic),
207                         '斜率': float(sl), '截距': float(ic),
208                         'MAPE_%': float(np.mean(np.abs(pred -

```

```

d.LB_Average) / d.LB_Average) * 100),
209         'R2': float(1 - np.sum((d.LB_Average -
    pred) ** 2)
210                               / np.sum((d.LB_Average - d
    .LB_Average.mean()) ** 2)))
211 bridge_df = pd.DataFrame(bridge_rows)
212 save_csv_safe(bridge_df, 'Loss_Benchmark映射.csv')
213 print(bridge_df.to_string(index=False))
214 br['pred'] = fits['全样本'][1] + fits['全样本'][0] * br.
    Val_Loss
215 save_csv_safe(br[['Model', 'N_params_B', 'D_tokens_B', '
    Val_Loss', 'LB_Average', 'pred',
216                 'Loss_Comparability']], 'Loss_Benchmark映射
    明细.csv')
217
218 try:
219     b5 = pd.read_csv(os.path.join(C_DIR, '
    loss_benchmark_bridge.csv'))
220     s5, i5 = np.polyfit(b5.Val_Loss, b5.LB_Average, 1)
221     print(' C5 (原始桥接, n=%d) 对照: r=%.3f, 斜率=%.3f'
222           % (len(b5), float(np.corrcoef(b5.Val_Loss, b5.
    LB_Average)[0, 1]), s5))
223 except Exception as e:
224     print(' C5 对照读取失败: ', e)
225 print(' 说明: 全样本 |r|=%.2f (R2=%.2f), 映射误差较大; 高可
    比子集仅 %d 条且落在 '
226       'Pythia 小模型区间, 得分变化幅度极小 (%.2f-%.2f), 故桥
    接误差对能力结论的影响'
227       '必须显式讨论。 '
228       % (abs(bridge_df.iloc[2]['Pearson_r']), bridge_df.iloc
    [2]['R2'],
229          int(bridge_df.iloc[0]['n']),
230          br[br.cmp.eq('高可比')].LB_Average.min(), br[br.cmp.
    eq('高可比')].LB_Average.max()))
231
232 # ===== 4. 面板回归与贡献分解 (规模扩张 vs 非规模技术进
    步) =====
233 # 思路: 规模系数 bP 在“同一年内的横截面”上识别最稳健 (年份效

```

```

    应吸收了当年的技术水平)，
234 #      再由 bP 与前沿模型的  $\Delta \log_{10} P$  做 shift-share 分解，残差
    部分归为非规模技术进步。
235 panel = core_o.copy()
236 lb_o = core_o.copy()
237 panel['logP'] = np.log10(panel.P)
238 panel['t'] = panel.Year - 2019
239 panel = panel[panel.t.between(0, 12)]
240
241 years = sorted(panel.t.unique())
242 def fit_model(sub, label, fe=True):
243     if fe:
244         Dm = np.column_stack([(sub.t.values == yy).astype(
245 float) for yy in years])
246         X = np.column_stack([Dm, sub.logP.values])
247     else:
248         X = np.column_stack([np.ones(len(sub)), sub.logP.
249 values, sub.t.values])
250     b, *_ = np.linalg.lstsq(X, sub.avg.values, rcond=None)
251     pred = X @ b
252     R2 = 1 - np.sum((sub.avg.values - pred) ** 2) / \
253         np.sum((sub.avg.values - sub.avg.values.mean()) ** 2)
254     return (b[-1], b[:-1], R2) if fe else (b[1], b[2], R2)
255
256 bP_fe, alpha_t, R2_fe = fit_model(panel, 'FE', True)
257 bP_ols, bt_ols, R2_ols = fit_model(panel, 'OLS', False)
258 panel['alpha_t'] = panel.t.map(dict(zip(years, alpha_t)))
259 save_csv_safe(pd.DataFrame({'Year': [y + 2019 for y in years],
    '年效应alpha_t': alpha_t}),
260                 '年份效应.csv')
261 save_csv_safe(panel[['Model', 'P', 'logP', 't', 'avg', '
    type_group', 'Year']],
262                 '面板回归数据.csv')
263 print('\n=== 4. 规模系数估计 (C1 开源筛选样本, %d 条, %d-%d
    年) ==='
264         % (len(panel), int(panel.Year.min()), int(panel.Year.max
    ())))
265 print('  年份固定效应模型: bP=%0.3f 分/10 倍参数, R2=%0.3f' % (

```

```

    bP_fe, R2_fe))
264 print('  线性时间 OLS 模型: bP=%.3f 分/10 倍参数, bt=%.3f 分/
    年, R2=%.3f'
265         % (bP_ols, bt_ols, R2_ols))
266 for tg in ['pretrained', 'chat/finetuned']:
267     sub = panel[panel.type_group.eq(tg)]
268     if len(sub) > 30:
269         bP_t, bt_t, R2_t = fit_model(sub, tg, False)
270         print('  %s 子样本 (n=%d): bP=%.3f 分/10 倍参数, bt
    =%.3f 分/年, R2=%.3f'
271               % (tg, len(sub), bP_t, bt_t, R2_t))
272
273 # 口径分层: C3 全期序列 (2019–2025, 跨年标准混合) 用于长周期分
    解;
274 #          C1 开源筛选样本 (2024–2025, 标准一致) 用于近端稳健
    性检验。
275 ts = pd.read_csv(os.path.join(C_DIR, '
    leaderboard_extended_timeseries.csv'))
276 ts['avg'] = pd.to_numeric(ts['Average'], errors='coerce')
277 ts['P'] = pd.to_numeric(ts['Params_B'], errors='coerce')
278 six_ts = ['IFEval', 'BBH', 'MATH_Lvl5', 'GPQA', 'MUSR', '
    MMLU_PRO']
279 for c in six_ts:
280     ts[c] = pd.to_numeric(ts[c], errors='coerce')
281 ts['mean6'] = ts[six_ts].mean(axis=1)
282 ts['gap'] = (ts.avg - ts.mean6).abs()
283 bad = ts[ts.gap > 5]
284 ts_ok = ts[(ts.gap <= 5) & ts.P.notna() & ts.avg.notna() & (ts
    .P > 0)].copy()
285 ts_ok['logP'] = np.log10(ts_ok.P)
286 ts_ok = ts_ok[np.isfinite(ts_ok.logP) & np.isfinite(ts_ok.avg)
    ].copy()
287 ts_ok['t'] = ts_ok.Year - 2019
288 save_csv_safe(bad[['Model', 'Year', 'Average', 'mean6', 'gap'
    ]], 'C3口径异常行.csv')
289 print('\n  C3 自洽性审计: |Average - 六维均值| > 5 的记录 %d
    条 (跨口径混入, 已剔除), '
290         '最大差值 %.1f (%)' % (len(bad), bad.gap.max()),

```

```

291         bad.loc[bad.gap.idxmax(), 'Model
    ']))
292
293 Xl = np.column_stack([np.ones(len(ts_ok)), ts_ok.logP.values,
    ts_ok.t.values])
294 bl, *_ = np.linalg.lstsq(Xl, ts_ok.avg.values, rcond=None)
295 b0_l, bP_l, bt_l = bl
296 pred_l = Xl @ bl
297 R2_l = 1 - np.sum((ts_ok.avg.values - pred_l) ** 2) / \
298     np.sum((ts_ok.avg.values - ts_ok.avg.values.mean()) ** 2)
299
300 front_year = ts_ok.groupby('Year').agg(avg=('avg', 'max'), P=(
    'P', 'max')).reset_index()
301 front_year['cum_avg'] = front_year.avg.cummax()           # 前沿边
    界按定义单调不减
302 front_year['t'] = front_year.Year - 2019
303 front_year['logP'] = np.log10(front_year.P)
304 save_csv_safe(front_year, '前沿序列.csv')
305
306 f1 = lb_o.groupby('Year').agg(avg=('avg', 'max'), P=('P', 'max
    ')).reset_index()
307 dF = front_year.cum_avg.iloc[-1] - front_year.cum_avg.iloc[0]
308 dLP = front_year.logP.iloc[-1] - front_year.logP.iloc[0]
309 dec = pd.DataFrame([
310     {'口径': 'C3 长周期（跨年标准混合）', '前沿区间': '%d→%d'
311         % (front_year.Year.iloc[0], front_year.Year.iloc[-1]),
312         '前沿提升分': dF, 'Δlog10P': dLP, 'bP': bP_l, 'bt(分/年)'
313         : bt_l, 'R2': R2_l,
314         '规模贡献分': bP_l * dLP, '非规模贡献分': dF - bP_l * dLP
315         ,
316         '规模占比%': bP_l * dLP / dF * 100, '非规模占比%': (dF -
317         bP_l * dLP) / dF * 100}])
318 save_csv_safe(dec, '规模时间分解_长周期.csv')
319 print('\n 前沿序列（年度最高分 + 累计最大, %d-%d）: '
320     % (front_year.Year.iloc[0], front_year.Year.iloc[-1]))
321 print(front_year.round(2).to_string(index=False))
322 print(' 贡献分解: ')
323 print(dec.round(2).to_string(index=False))

```

```

321 print(' 提示: C3 早期年份为公开报告值 (每 1-7 条), 跨年评测
      标准不完全一致 (C3 标注'
322       '为“混合”口径), 故长周期占比视为量级参考; C1 近端口径
      标准一致但窗口仅约 1 年。')
323
324 # ===== 5. 能力前沿的 logistic 外推 =====
325 # 前沿定义: 每个时间窗内开源筛选模型平均分的上侧分位 (月度取前
      1%, 历史年度取最高分)
326 lb_o['ym'] = lb_o.date.dt.to_period('M')
327 mon_top1 = lb_o.groupby('ym').apply(
328     lambda g: g.avg.nlargest(max(1, int(len(g) * 0.01))).mean
      (), include_groups=False)
329 mon_top1 = mon_top1[mon_top1.index >= pd.Period('2024-01')]
330 t_mon = np.array([(pd.Timestamp(str(p)).year - 2019) + (pd.
      Timestamp(str(p)).month - .5) / 12
331                   for p in mon_top1.index])
332
333 # 前沿边界单调不减: 历史段取 C3 累计最大, 近端取月度前 1% 的累
      计最大
334 hist_rows = front_year[front_year.Year <= 2023]
335 t_h = hist_rows.t.values + .5
336 s_h = np.maximum.accumulate(hist_rows.cum_avg.values)
337 t_front = np.r_[t_h, t_mon]
338 s_front = np.r_[s_h, np.maximum.accumulate(mon_top1.values)]
339 idx = np.argsort(t_front)
340 t_front, s_front = t_front[idx], np.maximum.accumulate(s_front
      [idx])
341
342 # 近端口径 (一致窗口) 内的前沿分解: 从首月到峰值月
343 def month_stats(period):
344     g = lb_o[lb_o.date.dt.to_period('M').eq(period)]
345     g = g.nlargest(max(1, int(len(g) * 0.01)), 'avg')
346     return float(g.avg.mean()), float(g.P.max())
347
348 p0, pk = mon_top1.index[0], mon_top1.idxmax()
349 F0_, P0_ = month_stats(p0)
350 Fk_, Pk_ = month_stats(pk)
351 dF1 = Fk_ - F0_

```

```

352 dLP1 = float(np.log10(Pk_ / P0_))
353 dec1 = pd.DataFrame([
354     '口径': 'C1 近端 (一致窗口 %s→%s, 前沿模型 %sB→%sB) '
355     % (str(p0), str(pk), ('%.1f' % P0_), ('%.1f' % Pk_)),
356     '前沿区间': '%s→%s' % (str(p0), str(pk)),
357     '前沿提升分': dF1, 'Δlog10P': dLP1, 'bP': bP_fe, 'bt(分/年)': bt_ols, 'R2': R2_fe,
358     '规模贡献分': bP_fe * dLP1, '非规模贡献分': dF1 - bP_fe * dLP1,
359     '规模占比%': bP_fe * dLP1 / dF1 * 100 if dF1 != 0 else np.nan,
360     '非规模占比%': (dF1 - bP_fe * dLP1) / dF1 * 100 if dF1 != 0 else np.nan]])
361 dec_all = pd.concat([dec, dec1], ignore_index=True)
362 save_csv_safe(dec_all, '规模时间分解.csv')
363 print('\n 近端口径 (一致窗口) 分解: 前沿 %s→%s 由 %.2f 升至 %.2f (+%.2f 分), '
364       '前沿模型规模 %sB→%sB (Δlog10P=%.3f) '
365       % (str(p0), str(pk), F0_, Fk_, dF1, ('%.1f' % P0_), ('%.1f' % Pk_), dLP1))
366 print(' -> 规模贡献 %.2f 分, 非规模贡献 %.2f 分; 该窗口内前沿提升基本来自非规模因素。 '
367       % (bP_fe * dLP1, dF1 - bP_fe * dLP1))
368
369 def logistic(t, K, r, t0):
370     return K / (1 + np.exp(-r * (t - t0)))
371
372 t_last = float(np.max(t_mon))
373 t_pred = np.array([t_last + 1.0, t_last + 2.0]) # 严格对
374     应未来 12 / 24 个月
375 try:
376     popt, _ = curve_fit(logistic, t_front, s_front, p0=[55, .8, 3.5],
377                         bounds=([20, .01, 0.5], [100, 5, 8]),
378                         maxfev=40000)
379     K_fit, r_fit, t0_fit = popt
380     pred_fit = logistic(t_front, *popt)
381     R2_log = 1 - np.sum((s_front - pred_fit) ** 2) / np.sum((

```

```

    s_front - s_front.mean()) ** 2)
380 except Exception as e:
381     K_fit, r_fit, t0_fit, R2_log = 52., .3, 3., 0.
382     print('logistic 拟合失败: ', e)
383 s_pred = logistic(t_pred, K_fit, r_fit, t0_fit)
384
385 resid = s_front - logistic(t_front, K_fit, r_fit, t0_fit)
386 rng = np.random.default_rng(42)
387 boot = []
388 for _ in range(600):
389     s_b = logistic(t_front, K_fit, r_fit, t0_fit) + rng.choice
390         (resid, size=len(resid), replace=True)
391     try:
392         pb, _ = curve_fit(logistic, t_front, s_b, p0=[K_fit,
393             r_fit, t0_fit], maxfev=40000)
394         boot.append(logistic(t_pred, *pb))
395     except Exception:
396         pass
397 boot = np.array(boot)
398 lo, hi = np.percentile(boot, 5, axis=0), np.percentile(boot,
399     95, axis=0)
400
401 # 情景分析：算力增长放缓 -> 技术增速减半
402 def logistic_slow(t, K, r, t0, halve_from):
403     te = np.minimum(t, halve_from)
404     return K / (1 + np.exp(-r * (te - t0)))
405 s_slow = logistic_slow(t_pred, K_fit, r_fit, t0_fit, t_pred[0]
406     - 1)
407
408 fut = pd.DataFrame({'t(2019起算年)': t_pred, '对应日历时点':
409     ['%d-%02d' % (2019 + int(np.floor(tt)),
410         int(round((tt % 1) * 12)) + 1)
411         for tt in t_pred],
412     'frontier_pred': s_pred,
413     'CI_low_5%': lo, 'CI_high_95%': hi, '情景_
414     技术增速减半': s_slow})
415 save_csv_safe(fut, '前沿预测.csv')

```

```

411 # 结构外推:  $F(t+\Delta) = F(t) + [bP \cdot \Delta \log_{10} P + bt] \cdot \Delta$  (把分解式直接
    用作预测方程)
412 F_last = float(np.max(s_front))
413 scen = []
414 for yrs in [1, 2]:
415     for dlogP, bt_use, lbl in [(0.0, bt_ols, '基准 (前沿模型规
        模持平, 非规模增速不变)'),
416                               (0.0, bt_ols / 2, '算力增长放缓
        (非规模增速减半)'),
417                               (0.3, bt_ols, '规模继续扩张 (前
        沿 +0.3 dex/年)')]:
418         scen.append({'情景': lbl, '前瞻年数': yrs,
419                     'Δlog10P假设': dlogP, 'bt假设(分/年)':
        bt_use,
420                     '结构外推前沿': F_last + (bP_fe * dlogP +
        bt_use) * yrs})
421 scen_df = pd.DataFrame(scen)
422 save_csv_safe(scen_df, '前沿预测_结构外推情景.csv')
423 save_csv_safe(pd.DataFrame([{'K': K_fit, 'r': r_fit, 't0(2019
    起)': t0_fit, 'R2': R2_log,
424                             '数据截止': str(max(lb_o.date))
    [:10], 'n前沿点': len(t_front),
425                             't_last': t_last}])),
426               'logistic参数.csv')
427 print('\n=== 5. 能力前沿 logistic 外推 ===')
428 print(' F(t)=K/(1+exp(-r(t-t0))): K=%.2f, r=%.3f, t0=%.2f (
    2019 起算), R2=%.3f'
429       % (K_fit, r_fit, t0_fit, R2_log))
430 print(fut.round(2).to_string(index=False))
431
432 # ===== 6. C4 宏观证据 (算力、数据量、开源权重)
    =====
433 ep = pd.read_csv(os.path.join(C_DIR, 'epoch_all_ai_models.csv'
    ), low_memory=False)
434 ep['compute'] = pd.to_numeric(ep['Training compute (FLOP)'].
    astype(str).str.replace(',', ''),
435                             errors='coerce')
436 ep['dsize'] = pd.to_numeric(ep['Training dataset size (total)']

```

```

].astype(str).str.replace(',', ''),
437         errors='coerce')
438 ep['P'] = pd.to_numeric(ep['Parameters'].astype(str).str.
    replace(',', ''), errors='coerce')
439 ep['year'] = pd.to_datetime(ep['Publication date'], errors='
    coerce').dt.year
440 ep['open_w'] = ep['Open model weights?'].astype(str).str.strip
    ().eq('Yes')
441
442 c4 = ep[(ep.Domain.astype(str).str.contains('Language', na=
    False)) &
443         ep.year.between(2018, 2026)].copy()
444 g = c4.groupby('year').agg(n=('Model', 'size'),
445                             n_compute=('compute', 'count'),
446                             前沿算力_max=('compute', 'max'),
447                             前沿算力_中位=('compute', 'median')
448                             ,
449                             最大参数量_B=('P', 'max'),
450                             最大数据量_tokens=('dsize', 'max'),
451                             开源权重占比=('open_w', 'mean')).
    reset_index()
452 save_csv_safe(g, 'C4宏观趋势.csv')
453 print('\n=== 6. C4 宏观证据 (Epoch AI 全模型元数据, Language
    域) ===')
454 print(g.round(3).to_string(index=False))
455 gc = g.dropna(subset=['前沿算力_max'])
456 gc = gc[gc.前沿算力_max > 0]
457 sl_c, ic_c = np.polyfit(gc.year, np.log10(gc.前沿算力_max), 1)
458 print('  前沿训练算力的年增长率: log10(C) 每年 +%.3f 个数量级
    (即约 %.2f 倍/年) '
459       % (sl_c, 10 ** sl_c))
460 ov = ep['Open model weights?'].astype(str).str.strip()
461 print('  C4 开源权重字段分布: ' + ';' .join('%s=%d' % (k, v)
    for k, v in ov.value_counts().head(4).items()))
462 save_csv_safe(pd.DataFrame([{'log10前沿算力年增速': sl_c, '年
    倍数': 10 ** sl_c,
463                             '开源权重_Yes': int(ov.eq('Yes')).
    sum()),

```

```

464         '开源权重_No': int(ov.eq('No').
sum()),
465         '有算力记录行数': int(ep.compute.
notna().sum()),
466         '有数据量记录行数': int(ep.dsize.
notna().sum()))]],
467         'C4宏观摘要.csv')
468
469 # ===== 7. 六维 Benchmark 前沿与剩余空间（开源筛选集）
=====
470 task_rows = []
471 for tk in TASKS:
472     s = lb_o[tk].dropna()
473     task_rows.append({'task': tk, 'n': len(s), 'max': float(s.
max()),
474                       'top1_mean': float(s.nlargest(max(1, int
(len(s) * 0.01))).mean()),
475                       'median': float(s.median()),
476                       'headroom_%': float((100 - s.max()))})
477 task_df = pd.DataFrame(task_rows).sort_values('max')
478 save_csv_safe(task_df, '逐任务聚合.csv')
479 print('\n=== 7. 六维 Benchmark 前沿与剩余空间（开源筛选集, n=%
d) ===' % len(lb_o))
480 print(task_df.to_string(index=False))
481
482 # ===== 8. 绘图 =====
483 plt.rcParams['font.size'] = 12
484 C1_, C2_, C3_ = '#3b6ea5', '#e07a3f', '#5aa05a'
485
486 fig, ax = plt.subplots(figsize=(6.8, 5))
487 hi_pt = br[br.cmp.eq('高可比')]; me_pt = br[br.cmp.eq('中可比'
)]
488 ax.scatter(hi_pt.Val_Loss, hi_pt.LB_Average, s=30, color=C1_,
489            label='高可比 (n=%d)' % len(hi_pt))
490 ax.scatter(me_pt.Val_Loss, me_pt.LB_Average, s=20, color='#
aec7e8',
491            label='中可比 (n=%d)' % len(me_pt))
492 Ls = np.linspace(br.Val_Loss.min(), br.Val_Loss.max(), 50)

```

```

493 ax.plot(Ls, fits['全样本'][1] + fits['全样本'][0] * Ls, '--',
        color='#d62728', lw=1.6,
494         label='全样本 r=%.2f' % bridge_df.iloc[2]['Pearson_r']
        ])
495 ax.set_xlabel('验证交叉熵损失 L'); ax.set_ylabel('Leaderboard
        均分 B')
496 ax.set_title('Loss 与 Benchmark 得分的桥接（按可比性分层）')
497 ax.legend(fontsize=9); despine(ax); fig.tight_layout();
        save_fig(fig, '图1_Loss_Benchmark映射')
498
499 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
500 ax = axes[0]
501 sc = ax.scatter(panel.logP, panel.avg, s=8, c=panel.Year, cmap
        ='viridis', alpha=.5)
502 cb = fig.colorbar(sc, ax=ax, shrink=.85); cb.set_label('提交年
        份')
503 Pg = np.linspace(panel.logP.min(), panel.logP.max(), 50)
504 for yy in [2023, 2024, 2025]:
505     k = yy - 2019
506     if k in list(years):
507         off = float(panel.loc[panel.t.eq(k), 'alpha_t'].iloc
        [0])
508         ax.plot(Pg, off + bP_fe * Pg, lw=1.6, label='%d 年拟合
        （含年效应）' % yy)
509 ax.set_xlabel('log10 参数量 (B)'); ax.set_ylabel('Leaderboard
        均分')
510 ax.set_title('(a) 同一年内的规模效应'); ax.legend(fontsize=8);
        despine(ax)
511 ax = axes[1]
512 d0 = dec.iloc[0]
513 ax.bar(['规模扩张', '非规模技术进步'], [d0['规模贡献分'], d0['
        非规模贡献分']],
514        color=[C1_, C2_], width=.5)
515 for i, v in enumerate([d0['规模贡献分'], d0['非规模贡献分']]):
516     ax.text(i, v + .3, '%.1f%%' % ([d0['规模占比%'], d0['非规
        模占比%']][i]), ha='center')
517 ax.set_ylabel('对前沿提升的贡献（分）')
518 ax.set_title('(b) %s 前沿提升来源分解' % d0['前沿区间']);

```

```

    despine(ax)
519 fig.tight_layout(); save_fig(fig, '图2_规模时间分解')
520
521 fig, ax = plt.subplots(figsize=(8, 5))
522 ax.plot(t_front, s_front, 'o', color=C1_, ms=4, label='历史前
    沿（年度锚点+月频前1%）')
523 tt = np.linspace(0, t_pred[-1] + 1, 300)
524 ax.plot(tt, logistic(tt, K_fit, r_fit, t0_fit), '-', color='#
    d62728', lw=2, label='logistic 饱和外推')
525 ax.plot(t_pred, s_pred, 's', color=C2_, ms=8, label='预测前沿'
    )
526 ax.fill_between(t_pred, lo, hi, color=C2_, alpha=.2, label='
    90% 置信区间（残差 bootstrap）')
527 ax.plot(t_pred, s_slow, 'v--', color='gray', ms=7, label='情
    景：技术增速减半')
528 ax.set_xlabel('年份（2019 起算）'); ax.set_ylabel('能力前沿（
    Leaderboard 均分）')
529 ax.set_ylim(0, 100); ax.legend(fontsize=8); despine(ax)
530 ax.set_title('开源大语言模型能力前沿预测（未来 12/24 个月）')
531 fig.tight_layout(); save_fig(fig, '图3_能力前沿预测')
532
533 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))
534 ax = axes[0]
535 td = task_df.sort_values('max')
536 ax.barh(td['task'], td['max'], color=C1_, label='任务最优得分'
    )
537 ax.barh(td['task'], td['top1_mean'], color='#aec7e8', label='
    前 1% 均值')
538 ax.axvline(100, color='gray', ls=':', lw=1)
539 ax.set_xlabel('Benchmark 得分（满分 100）'); ax.set_title('(a)
    六维前沿与饱和程度')
540 ax.legend(fontsize=9); despine(ax)
541 ax = axes[1]
542 top = sub_df.nlargest(10, '剩余空间_%').sort_values('剩余空间_
    %')
543 ax.barh(top['子任务'], top['剩余空间_%'], color=C2_)
544 ax.set_xlabel('相对满分的剩余空间 (%)'); ax.set_title('(b) C8
    子任务剩余空间最大 10 项')

```

```

545 despine(ax); fig.tight_layout(); save_fig(fig, '图4_逐任务聚合
    ')
546
547 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
548 ax = axes[0]
549 gg = g.dropna(subset=['前沿算力_max'])
550 ax.plot(gg.year, gg.前沿算力_max, 'o-', color=C1_, label='年度
    前沿训练算力')
551 ax.plot(gg.year, gg.前沿算力_中位, 's--', color=C2_, label='年
    度中位训练算力')
552 ax.set_yscale('log'); ax.set_xlabel('发布年份'); ax.set_ylabel
    ('训练算力 (FLOP)')
553 ax.set_title('(a) C4: 训练算力的年度演化'); ax.legend(fontsize
    =8); despine(ax)
554 ax = axes[1]
555 ax.bar(g.year, g.开源权重占比 * 100, color=C3_)
556 ax.set_xlabel('发布年份'); ax.set_ylabel('开源权重模型占比 (%)
    ')
557 ax.set_title('(b) C4: 开源权重比例'); despine(ax)
558 fig.tight_layout(); save_fig(fig, '图5_C4宏观趋势')
559
560 # ===== 9. 汇总 =====
561 print('\n===== 问题四求解完成 =====')
562 print('  桥接: 全样本 r=%.3f; 分解 (%s): 规模 %.1f%% / 非规模
    %.1f%%; '
563       'OLS 版: 规模 %.1f%% / 非规模 %.1f%%'
564       '% (bridge_df.iloc[2]['Pearson_r'], d0['前沿区间'], d0['
    规模占比%'], d0['非规模占比%'],
565       dec_all.iloc[1]['规模占比%'], dec_all.iloc[1]['非规模
    占比%'])))
566 print('  前沿预测 (+12/+24 个月): %s %.2f (90%CI %.2f-%.2f
    ); %s %.2f (90%CI %.2f-%.2f); '
567       '技术增速减半情景 %.2f / %.2f'
568       '% (fut['对应日历时点'].iloc[0], fut.frontier_pred.iloc
    [0], fut['CI_low_5%'].iloc[0],
569       fut['CI_high_95%'].iloc[0], fut['对应日历时点'].iloc
    [1], fut.frontier_pred.iloc[1],
570       fut['CI_low_5%'].iloc[1], fut['CI_high_95%'].iloc[1],

```

```

571         fut['情景_技术增速减半'].iloc[0], fut['情景_技术增速
        减半'].iloc[1]))
572 print('\n 结构外推情景（基准前沿 F=%.2f）： ' % F_last)
573 print(scen_df.round(2).to_string(index=False))
574 print(' 两条路线互为校验：曲线外推（logistic）给出约 %.0f-%.0
        f，结构外推给出约 '
575         '%.0f-%.0f；差异来自“前沿是否已饱和”的判断，故以区间而
        非点值作为结论。'
576         % (min(fut.frontier_pred.min(), fut['CI_low_5%'].min()),
        fut['CI_high_95%'].max(),
577         scen_df['结构外推前沿'].min(), scen_df['结构外推前沿'
        ].max()))

```

Listing 4: 问题四求解代码（口径筛选、C8 逐任务聚合、Loss 桥接、规模/时间分解与能力前沿外推）